



Arab International University

Faculty of Informatics and Communication Engineering

Senior Project Report on

**An IoT and AI-Based Smart Home Security and
Safety System**

Submitted to

Department of Informatics Engineering

in partial fulfillment of the requirement for the Degree of Bachelor in

Informatics Engineering

Submitted by

Mohammad Naim Ayrourd Mohammad Hussein Hamed

Abdulkareem Shaheen

Anas Saudi

Islam Alabd

Under the Supervision of

Dr. Ahmad Mohsen

Eng. Abd-Allah Al-Sardi

Jan 2026

© **AIU** Arab International University

All Rights Reserved

Jan 2026

Faculty of Informatics & Communication Engineering

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the Department of Informatics Engineering for acceptance, a project report entitled “**An IoT and AI-Based Smart Home Security and Safety System**” Submitted by: **Mohammad Naim Ayroud, Mohammad Hussein Hamed, Abdulkareem Shaheen, Anas Saudi, Islam Alabd** in partial fulfilment for the degree of Bachelor of Engineering in Informatics.

Supervisor

Dr. Ahmad Mohsen

Head of Department

Dr. Rami Yared

Arab International University

The Arab International University (AIU) is a private Syrian university established in 2005. Its academic plans and the documents issued by it are approved and certified by the Ministry of Higher Education in the Syrian Arab Republic.

The university works to achieve the following goals:

- Preparing a distinguished generation of university graduates who are able to meet and advance the specific needs of society.
- Contributing to theoretical and applied scientific research that serves the purposes of national development. Work is being done to urge professors and academic staff to scientific research and participate in conferences and seminars that organize research.
- Achieving partnership with prestigious Arab and foreign universities with the aim of continuous development and modernization of academic work and conducting joint scientific research.
- Attracting distinguished academic and research competencies by providing the appropriate environment for their work.

The Arab International University is one of the first Syrian universities that have been established and inaugurated. It has been able to attract distinguished educational, research and administrative competencies, to create an integrated edifice from the academic, organizational and administrative aspects. It was able to graduate cadres of distinguished innovators by providing an educational environment based on unique qualitative and material ingredients, including:

- Modern and advanced study plans based on the credit hour system.
- Carefully selected educational cadres.
- Modern scientific laboratories and a laboratory for electronic libraries.
- Physical and moral incentives for students.
- Application of interactive teaching methods.
- Academic and educational guidance and counseling.
- A wide range of scientific cooperation agreements with local, regional and international universities of reputable reputation.
- Multiple agreements and memoranda of understanding with many civil society organizations.
- A proper campus with all the facilities of science, sports and entertainment, which we encourage you to visit and learn about their features.
- Student activities and clubs of all kinds: athletic, cultural, scientific and social.

The Arab International University life years are a time to invest in a student's future. The knowledge and experience that students acquire in the lecture hall and laboratories will help them in developing themselves. They will provide them with reasons for success in the chosen specialty. The student activities will help in expanding students' horizon. The activities of training, clubs and sports will enable students to develop their talents, and may even help in discovering new talents.

Students could invest time, mind and spirit in our university in order to reap the benefits of works and the time devoted in the coming years. We will be by our students in every step of their way.

To my father

My mother

My brothers ...

.....

Student1 name

To my father

My mother

My brothers ...

.....

Student2 name

.....

.....

Acknowledgements

It gives us immense pleasure to express our deepest sense of gratitude and sincere thanks to our highly respected and esteemed guide Dr. **Ahmad Mohsen** for her valuable guidance, encouragement and help for completing this work, her useful suggestions for this whole work and cooperative behavior are sincerely acknowledged.

We are also grateful to Dr. **Tarek Barhoum** for his constant support and guidance.

At the end we would like to express our sincere thanks to all our families, friends and others who helped us directly or indirectly during this project work.

Abstract

AuraHome is an IoT-based smart home security and safety platform that integrates access control, intrusion monitoring, hazard sensing, and real-time notifications within a single architecture. The system combines ESP32-based nodes with a Raspberry Pi 5 edge gateway for local processing, and a cloud layer that includes a Laravel backend, MySQL database, and a Mosquitto MQTT broker to enable secure publish–subscribe communication. A Flutter mobile application provides remote monitoring and control, while Firebase Cloud Messaging is used for near real-time alert delivery.

Several AI components were integrated and evaluated independently. The Speech Emotion Recognition module achieved **77.45%** accuracy on **MSP-IMPROV** (F1 **0.775**) and **31.59%** on **emoDB** (F1 **0.27**), highlighting limited cross-lingual generalization. Gun detection, tested on **1,076 images**, achieved precision **0.838**, recall **0.835**, mAP@0.5 **0.872**, and mAP@0.5:0.95 **0.521**. Masked-people detection, tested on **399 images**, achieved precision **0.935**, recall **0.740**, mAP@0.5 **0.856**, and mAP@0.5:0.95 **0.614**. Face anti-spoofing was implemented using DeepFace without custom training and achieved **95.2%** accuracy on around **150** diverse samples under varied lighting and spoofing attempts. Server-side load testing showed stable performance under moderate concurrency, while stress testing revealed capacity limits that motivate future scalability improvements.

Keywords

- Smart Home Security
- Internet of Things (IoT)
- Real-time Monitoring
- MQTT Broker
- Firebase Cloud Messaging (FCM)
- Mobile Application (Flutter)
- Backend System (Laravel)
- REST API
- Computer Vision
- YOLOv8
- Gun Detection
- Masked Person Detection
- Face Recognition
- Face Anti-Spoofing (Liveness Detection)
- Speech Emotion Recognition (SER)

Abbreviations

AI Artificial Intelligence

IoT Internet of Things

MQTT Message Queuing Telemetry Transport

FCM Firebase Cloud Messaging

API Application Programming Interface

REST Representational State Transfer

UI User Interface

YOLO You Only Look Once

mAP Mean Average Precision

IoU Intersection over Union

SER Speech Emotion Recognition

TLS Transport Layer Security

Contents

Introduction	2
Chapter 1: Project Description	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Project Objective	4
1.4 Project Scope.....	5
1.5 Project Features	5
1.6 Project Feasibility	6
1.6.1 Technical Feasibility	6
1.6.2 Commercial Feasibility	7
1.6.3 Operational Feasibility	7
1.7 System Requirements.....	7
1.7.1 Hardware Requirements	7
1.7.2 Software Requirements	8
Chapter 2: Theoretical Study.....	10
2.1 IoT in Smart Home Security.....	10
2.2 MQTT Publish/Subscribe Messaging Protocol (Over TLS).....	10
2.3 Speech Emotion Recognition (SER) using CNN.....	11
2.4 Object Detection using YOLO in Computer Vision.....	11
Chapter 3: Related Work.....	13
3.1 Similar Applications	13
3.1.1 Key Features Explained	13
3.1.2 Comparison with Similar Applications	14
3.2 Literature Review	15
3.2.1 Weapons and Masked Detection Core	15
3.2.1.1 Weapon Detection with FMR-CNN and YOLOv8 for Enhanced Crime Prevention and Security	15
3.2.1.2 Deep Learning and YOLOv8 Utilized in an Accurate Face Mask Detection System.....	16

3.2.2 Speech Emotion Recognition Core	17
3.2.2.1 Improved Speech Emotion Recognition Using Transfer Learning on Spectrograms	17
3.2.2.2 Speech Emotion Recognition Using Deep 1D & 2D CNN LSTM Networks ..	18
3.2.3 Face Recognition Core	19
3.2.3.1 Measuring the Re-identification Risk of Imaging Participants Using DeepFace	19
3.2.3.2 Real-Time Facial Emotion Recognition Using YOLOv8 and DeepFace	20
Chapter 4: System Analysis	22
4.1 Requirements Specification	22
4.2 Functional Requirements.....	25
4.3 Non-Functional Requirements	27
4.3.1 Security.....	27
I. Data Protection.....	28
II. Authentication & Authorization.....	28
III. MQTT Security.....	28
IV. Audit Trail.....	29
V. Network Security	29
4.3.2 Performance	29
4.3.3 Efficiency	30
4.3.4 Usability.....	30
4.3.5 Maintainability.....	31
4.4 Use Case Diagram	32
Chapter 5: System Design	33
5.1 System Architecture	33
5.2 Entity Relationship Diagram.....	34
5.3 Sequence Diagram.....	35
5.3.1 User Creation with Automatic MQTT Namespace Provisioning.....	35
5.3.2 Device Creation with Product-Type Based Auto-Provisioning	36
5.3.3 Email Verification Reset on Admin Edit	37
5.3.4 Async User Deletion with MQTT Cleanup & Retry Logic.....	38
5.4 Activity Diagram	39

5.4.1 Real-Time Sensor Data Flow	39
5.5 Hardware Design	40
5.5.1 ESP32 Node Architecture Overview.....	40
5.5.2 Local AI Deployment on Raspberry Pi 5 (Performance and Privacy)	42
5.5.3 Per-Node Circuit Design and Component Selection	42
5.5.3.1 Door Lock Product (DLP) Circuit.....	42
LCD 16×2 (I2C)	42
4×3 Matrix Keypad.....	43
Push Button (Start Trigger).....	43
Buzzer (Audio Feedback).....	44
Relay Module (Electronic Lock Control)	44
5.5.3.2 Door Guard Product (DGP) Circuit	45
Ultrasonic Distance Sensor.....	45
PIR Motion Sensor	46
LDR (Light-Dependent Resistor)	46
Two Relay Modules (12 V LED Control).....	47
5.5.3.3 Gas Temperature Product (GTP) Circuit.....	47
DS18B20 Temperature Sensor	47
MQ-4 Gas Sensor	48
Buzzer (Hazard Alert)	48
Status LEDs (2 × 3.3 V).....	49
5.5.3.4 Trigger Emergency Product (TEP) Circuit	49
INMP441 Microphone (I2S).....	49
Buzzer (Wake Word Feedback)	50
5.6.1 Security Objectives and Protected Assets.....	50
5.6.1.1 Protection of device credentials (device_token and MQTT credentials)	50
5.6.1.2 Protection of control and telemetry traffic (MQTT messages).....	51
5.6.1.3 Securing user access to the mobile application (2FA/OTP and optional biometric login)	51
5.6.1.4 Topic isolation and authorization (per-user namespace + ACL rules).....	51
5.6.1.5 Privacy-first local storage and processing (Raspberry Pi 5).....	52
5.6.2 Replay Attack Mitigation.....	52
Chapter 6: Implementation.....	53

6.0 Common Hardware Implementation Concepts	53
6.1 Door Lock Product (DLP) Implementation	54
6.1.1 AI Models Used in DLP	54
6.1.2 Initial Dataset-Based Experiments and Limitations	55
6.1.3 DeepFace Adoption as a Practical Implementation Solution	56
6.1.4 Source-Code Adaptations for Real-Time Decisions and Returned Outputs	56
6.1.5 Anti-Spoofing Decision Mechanism in DeepFace	57
6.1.6 Selected Models and Backends in This Implementation	57
6.1.7 Hardware Connections of the DLP Circuit	58
6.1.8 DLP Workflow Algorithm.....	59
6.2 Door Guard Product (DGP) Implementation (Timed Suspicious-Activity Detection)	60
6.2.1 Overview and Timed Activation Logic.....	60
6.2.2 Sensor-Gated “Suspicious Movement” Confirmation	60
6.2.3 Masked Person Detection Model Training and Evaluation	61
6.2.4 Weapon (Gun) Detection Model Training and Evaluation	63
6.2.5 DGP’s AI Models Local Inference	65
6.2.6 Hardware Connections of the DGP Circuit	65
6.2.7 DGP Workflow and Algorithm	66
6.3 Gas and Temperature Product (GTP) Implementation.....	67
6.3.1 Overview of GTP	67
6.3.2 Hardware Connections.....	68
6.3.3 Workflow and Decision Logic	69
6.4.1 Overview	69
6.4.2 Hardware and I2S Microphone Interface	70
6.4.3 Wake-Word Detection	70
6.4.4 MQTT Integration	70
6.4.5 SER Datasets and Preprocessing.....	71
6.4.6 SER Models and Experiments	71

6.4.7 End-to-End Workflow.....	73
6.5 Backend Development.....	74
6.5.1 Technologies Used	74
6.5.2 Architectural Design.....	75
Three-Layer Architecture	75
Dependency Injection and Interface Binding	76
6.5.3 Security Measures	77
Multi-Token Authentication System.....	77
Two-Factor Authentication (2FA)	77
Dual-Key Encryption Architecture	78
Role-Based Access Control (RBAC)	79
Password Security.....	79
Audit Logging.....	80
6.5.4 MQTT Integration	80
Mosquitto Dynamic Security Plugin	80
User Namespace Architecture	81
MQTT Credential Lifecycle	81
MosquittoDynamicSecurityService.....	82
MQTT Topic Generation.....	82
6.5.5 Deployment and Production Setup	83
Server Configuration.....	83
Laravel Queue System	83
6.6 Mobile Application implementation.....	84
6.6.1 Technologies Used	84
6.6.2 Architectural Design.....	86
Feature-Based Modular Structure.....	86
Three-Layer Architecture	87
Dependency Injection System	88
6.6.3 Security Implementation.....	89
Multi-Token Authentication Flow.....	89
Secure Token Storage	89
Two-Factor Authentication Support	90

6.6.4 Real-Time Device Communication.....	90
MQTT Integration Architecture	90
Real-Time Sensor Updates	91
6.6.5 Routing and Navigation.....	91
Chapter 7: Results & Testing	92
7.1 Speech Emotion Recognition (SER) Testing and Results.....	92
7.1.1 Datasets Description	92
MSP-IMPROV (English, Training Dataset):	92
emoDB (German, Cross-Lingual Evaluation):	92
7.1.2 Results	93
7.1.3 Discussion	93
7.2 Gun Detection Testing	94
7.2.1 Dataset and Testing Setup	94
7.2.2 Quantitative Results.....	94
7.2.3 Discussion	95
7.3 Masked People Detection Testing	96
7.3.1 Dataset and Testing Setup	96
7.3.2 Quantitative Results.....	97
7.3.3 Discussion	98
7.4 Face Anti-Spoofing Testing.....	99
7.4.1 DeepFace Library Adoption and Baseline Performance	99
7.4.2 Test Dataset and Experimental Validation	100
7.4.3 Results and Discussion	100
7.5 Server Testing.....	101
7.5.1 Server Specifications	101
7.5.2 JMeter Tool Details	102
7.5.3 Baseline Testing	102
7.5.4 Load Testing	104
7.5.5 Stress Testing.....	105

7.5.6 Performance Testing Summary	106
Chapter 8: Conclusion and Future Work	107
8.1 Conclusion.....	107
8.2 Challenges.....	107
8.3 Limitations.....	108
8.4 Future Work	108
References	111

List of Figures

Figure 1 :MQTT Publish/Subscribe Architecture Using Mosquitto Broker [1]	11
Figure 2: Example of YOLO Object Detection in a Street Scene [2]	12
Figure 3. Use Case Diagram	32
Figure 4: System Architecture.....	33
Figure 5: Entity Relationship Diagram.....	34
Figure 6: User Creation with Automatic MQTT Namespace Provisioning.....	35
Figure 7: Device Creation with Product-Type Based Auto-Provisioning	36
Figure 8: Email Verification Reset on Admin Edit	37
Figure 9: Async User Deletion with MQTT Cleanup & Retry Logic.....	38
Figure 10: Real-time Sensor Data Flow	39
Figure 11. Roboflow dataset sample showing limited diversity	55
Figure 12. YOLOv8 validation/testing results (20 epochs) showing possible fit	56
Figure 13. DLP circuit diagram	58
Figure 14. Masked Detection, Experiment 1	61
Figure 15. Masked Detection, Experiment 2	62
Figure 16. Masked Detection, Experiment 3	62
Figure 17. Gun Detection, Experiment 1	63
Figure 18. Gun Detection, Experiment 2	64
Figure 19. Gun Detection, Experiment 3	64
Figure 20. DGP Circuit Diagram.....	65
Figure 21. GTP Circuit Diagram	68
Figure 22. Balanced CNN Accuracy and Loss over Training.....	72
Figure 23. Balanced CNN Confusion Matrix.....	72
Figure 24. TEP end-to-end workflow showing system boot, Wi-Fi and MQTT initialization ...	73
Figure 25. Singleton Binding	76
Figure 26. Service-Repository Pattern.....	77
Figure 27. MQTT Topic Structure Example	81
Figure 28. Clean Architecture Design Pattern [14]	88
Figure 29. Sample detections from Gun Detection test dataset	94
Figure 30. Precision–Recall curve for Gun Detection Core (test set)	95
Figure 31. Normalized Confusion Matrix for Gun Detection Core.....	96
Figure 32. Sample detections from Masked People Detection test dataset	97
Figure 33. Precision-Recall Curve for Masked People Detection Core (test set)	98
Figure 34. Normalized Confusion Matrix for Masked People Detection Core	99
Figure 35. anti-spoofing test cases under different lighting and presentation conditions. ...	101
Figure 36. Response Time over Test Duration for Baseline Test	103
Figure 37. Response Time over Test Duration for Load Test.....	104
Figure 38. Response Time over Test Duration for Stress Test	105

List of Tables

Table 1: Similar Applications.....	15
Table 2: AuraHome ESP32 nodes, deployment locations, and functional roles.....	41
Table 3. SER datasets summary	71
Table 4. SER performance metrics on MSP-IMPROV and emoDB.....	93
Table 5. Baseline Test Metrics	103
Table 6. Load Test Metrics.....	104
Table 7. Stress Test Metrics	105

Introduction

In recent years, home security and safety have become increasingly vital aspects of daily life. This shift is primarily driven by the growing frequency of home burglaries and the broader accessibility of firearms among the general population. According to recent global crime statistics, residential burglary rates have seen a consistent rise, particularly in urban areas where unmonitored properties are frequent targets. These incidents not only result in property loss but often carry emotional and psychological consequences for homeowners.

At the same time, household hazards such as gas leaks and fires continue to pose serious and sometimes fatal risks. These threats, though preventable, often go undetected in homes lacking integrated safety systems. Unfortunately, most commercially available solutions tend to address these issues in a fragmented manner. For instance, fire detection systems, intrusion alarms, and biometric locks are often marketed as independent units, which limits their collective effectiveness.

Aura Home introduces a unified and intelligent solution to this problem. By leveraging the capabilities of Internet of Things (**IoT**) infrastructure and advanced Artificial Intelligence (**AI**) models, the system aims to provide comprehensive protection through proactive monitoring, threat detection, and real-time notifications. Its goal is to ensure that homeowners are not only protected against intrusions and environmental hazards but also kept informed and empowered through timely alerts sent directly to their mobile devices.

Chapter 1: Project Description

1.1 Background

In recent years, smart home systems have become more common due to the growth of Internet of Things (IoT) technologies. These systems allow people to control and monitor their homes remotely, making life more comfortable and efficient. One important area where smart systems are becoming more useful is home security and safety. With the increase in break-ins and household accidents such as fires and gas leaks, there is a real need for better solutions. Many current products on the market focus on only one part of the problem, like fire alarms or smart locks, but do not provide full protection. At the same time, new technologies like Artificial Intelligence (AI) have made it possible to build smarter systems that can detect danger and send alerts in real time. This has created an opportunity to design complete smart home solutions that combine different features into one connected system.

1.2 Problem Statement

Home protection is no longer a luxury—it has become a basic need. However, many smart home security products do not cover all aspects of protection with high quality and reliability. One of the main issues is that AI models are not always accurate and can make mistakes.

These errors can lead to serious consequences, such as property theft or even loss of life in dangerous situations. This creates a need for more research and testing to improve the accuracy of AI models and their response time, in order to increase safety levels in smart home environments.

1.3 Project Objective

The main objective of this project is to build a smart home system that improves both security and safety using IoT devices and AI models. The system will use ESP32 nodes to monitor the environment, detect possible threats, and send real-time alerts to the user's mobile app.

Key goals include secure door access using face and fingerprint recognition, detecting motion and masked people at night, gas and temperature monitoring, and recognizing emotions from speech after a wake word is detected. The system will communicate using MQTT over TLS and use Firebase Cloud Messaging (FCM) to deliver instant notifications.

1.4 Project Scope

This project includes **four** separate ESP32-based circuits (nodes), each responsible for a specific task. The first node handles secure door access using biometric factors (face and fingerprint) combined with a keypad password. The second node monitors motion at night using ultrasonic and PIR sensors to detect suspicious activity, including masked individuals or weapons. Motion detection begins after a specific time in the evening, which can be controlled remotely by the homeowner through a dedicated MQTT control topic from the mobile app.

The third node is responsible for gas and fire detection using appropriate environmental sensors. The fourth node focuses on voice input, using an INMP441 microphone to detect a wake word, record short audio clips, and analyze speech emotion.

All AI models, including face recognition, weapon detection, and emotion analysis, are deployed locally on a **Raspberry Pi 5** installed inside the home. This ensures that data remains private and is not stored or processed in the cloud.

The project does not include smart appliance control of electrical devices. It also does not integrate with payment gateways; any payments for system setup or support are handled manually in person.

1.5 Project Features

The **AuraHome** system offers several integrated features that work together to improve home security and safety. Each feature is implemented through a dedicated ESP32 node and supported by AI models running locally. The main features include:

1) Secure Door Access (MFA + Anti-spoofing):

A smart door lock system that uses multiple factors for authentication, including

fingerprint, face recognition, and a keypad password. To prevent spoofing, the face recognition system checks for signs of fake attempts using printed images or videos. All access attempts, whether successful or not, are sent to the mobile app as real-time notifications.

2) Night Security Mode (Motion → Mask/Gun Detection):

At a configurable time during the evening, the system activates night security mode. The ESP32 node uses PIR, ultrasonic, and LDR sensors to detect real motion while reducing false alarms. If movement is confirmed, AI models running on the Raspberry Pi analyze the video feed to detect weapons (e.g., handgun, rifle) or masked individuals. Alerts are sent instantly to the user.

3) Gas and Temperature Safety Monitoring:

A dedicated node monitors gas levels and temperature to detect potentially dangerous situations such as gas leaks or overheating. When a threshold is passed, the system sends an immediate alert to the user.

4) Wake Word Detection and Speech Emotion Recognition (SER):

A microphone (INMP441) connected to an ESP32 listens for a wake word. Once detected, it records a 5-second audio clip and sends it to the SER model. The model analyzes the emotion in the voice (e.g., fear, anger) and reports the result to the backend and mobile app.

5) Real-Time Mobile Notifications and Alerts through User App:

The system sends real-time notifications to the user's mobile phone using Firebase Cloud Messaging (FCM). The Flutter app displays full event details, including time, type, and detection results, along with any available image or audio attachments.

1.6 Project Feasibility

1.6.1 Technical Feasibility

The hardware components used in this project, such as ESP32 boards, Raspberry Pi 5 and many other sensors, are widely available and well-supported. All AI models used in the system are ready for deployment and can run efficiently on the Raspberry Pi. MQTT communication between nodes and the backend is reliable, and there are many online resources that demonstrate its implementation. Additionally, integrating the

mobile app with Firebase services using Flutter and Cubit is straightforward and well-documented.

1.6.2 Commercial Feasibility

As of 2025, the prices of electronic components used in this project are reasonable and accessible in local and online markets. The total cost of the system remains affordable, especially when compared to commercial smart home systems, making AuraHome a competitive and cost-effective solution.

1.6.3 Operational Feasibility

The system is designed to be easy to install and operate by regular users without requiring technical expertise. The mobile app features a simple and user-friendly interface (UI/UX), and the interactions—such as receiving notifications or controlling security modes—are intuitive and easy to manage.

1.7 System Requirements

System requirements are the software and hardware resources needed to deploy the system in a real-world usage environment.

1.7.1 Hardware Requirements

The system requires the following hardware components:

- 4x ESP32 development boards (for separate sensor nodes)
- 1x Raspberry Pi 5 (to host and run AI models locally)
- 1x Gas sensor (e.g., MQ-2)
- 1x Temperature sensor (e.g., DHT22)
- 1x LCD 16x2 screen
- 3x Relay modules (for activating external outputs)
- 1x 12V LED (for output indicators)
- 1x Ultrasonic sensor (for distance/motion)
- 1x PIR sensor (for infrared motion detection)

- 1x INMP441 microphone (for audio input)
- 1x Keypad (for password entry)
- 1x Fingerprint sensor (e.g., R305)
- 1x LDR sensor (light detection)
- 1x Buzzer (for audio feedback)
- Assorted resistors, Jumper wires, and breadboards/PCBs
- Official 27W USB-C power supply for Raspberry Pi 5 (recommended for stability)
- Stable Wi-Fi router for MQTT communication
- Android smartphone (for running the mobile app)

1.7.2 Software Requirements

The software tools and platforms required for development, deployment, and usage include:

● On Raspberry Pi:

- Raspberry Pi OS (64-bit Lite) – based on Debian
- Python 3.8 or 3.10 – compatible with AI libraries
- Python libraries:
 - OpenCV, YOLOv8 (Ultralytics), DeepFace, Keras, TensorFlow, numpy, paho-mqtt, etc.

● On ESP32 Devices:

- Arduino IDE with ESP32 board support
- Required libraries for sensors, LCD, fingerprint, MQTT, etc.

● Backend and Communication:

- MQTT Broker (e.g., Mosquitto) with TLS support

- Properly configured ACLs and certificates for secure messaging

● Mobile Application:

- Flutter SDK (v3.10+)
- Dart language
- Cubit for state management
- Firebase Cloud Messaging (FCM) for push notifications
- Android device running Android 6.0 (Marshmallow) or higher

Chapter 2: Theoretical Study

This chapter presents the theoretical background and key technologies used in the development of the AuraHome system.

2.1 IoT in Smart Home Security

The Internet of Things (IoT) has transformed the way homes are secured by enabling devices to communicate, collect data, and respond in real time. In smart home systems, IoT allows various sensors—such as motion detectors, cameras, gas sensors, and temperature sensors—to work together under one integrated platform. These devices can detect unusual activity, environmental risks, and trigger alerts to the user.

IoT-based security systems are also scalable and can be customized based on the needs of each home. The use of low-cost microcontrollers like ESP32 makes it possible to build distributed systems that are reliable and power-efficient. In the AuraHome project, IoT forms the backbone of the system by connecting all sensor nodes and enabling communication between devices and the backend server.

2.2 MQTT Publish/Subscribe Messaging Protocol (Over TLS)

MQTT (Message Queuing Telemetry Transport) is a lightweight publish/subscribe protocol designed for reliable communication between IoT devices. It is suitable for resource-constrained environments like ESP32 microcontrollers due to its low bandwidth usage and minimal overhead.

To ensure secure communication, AuraHome uses MQTT over TLS (port 8883), which provides encryption and authentication. TLS certificates protect the messages from interception or tampering. Topics are structured to allow each ESP32 node to publish events and receive control messages from the backend. Access to topics is restricted using ACL (Access Control Lists), and Quality of Service (QoS) levels are configured based on the importance of each message.



Figure 1 .MQTT Publish/Subscribe Architecture Using Mosquitto Broker [1]

2.3 Speech Emotion Recognition (SER) using CNN

Speech Emotion Recognition (SER) is the process of analyzing audio to identify the emotional state of the speaker. In the AuraHome system, this task is performed using a Convolutional Neural Network (CNN) trained on labeled emotional datasets. The system detects a wake word, then records a short audio clip (5 seconds), which is passed to the CNN model.

The model classifies the speech into emotions such as fear, anger, or happiness, helping the system understand potential threats or emergency situations. SER adds an extra layer of intelligence by interpreting voice tone, regardless of the spoken language. The model runs on a Raspberry Pi 5 for local processing, which ensures privacy and low response time.

2.4 Object Detection using YOLO in Computer Vision

YOLO (You Only Look Once) is a real-time object detection algorithm widely used in computer vision tasks. It processes an entire image in a single forward pass, making it fast and efficient. In AuraHome, YOLOv8 is used to detect threats such as weapons (e.g., guns, rifles) or masked persons in surveillance footage.

When suspicious motion is detected at night, an image is sent to the YOLO model running on a Raspberry Pi. The model returns bounding boxes, object labels, and confidence scores. These results are then used to notify the user through the mobile app. YOLO's ability to detect multiple objects in one frame makes it suitable for real-time home security applications.

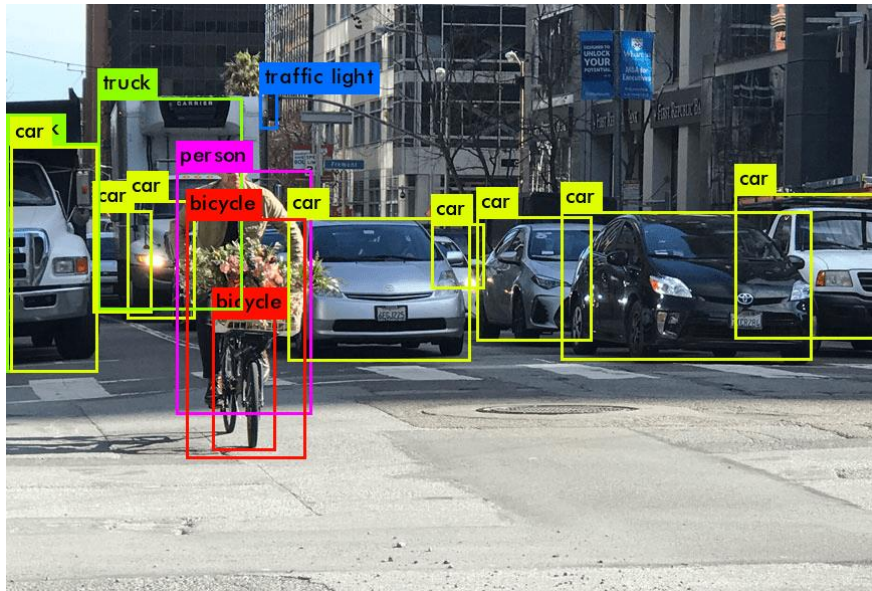


Figure 2: Example of YOLO Object Detection in a Street Scene [2]

Chapter 3: Related Work

3.1 Similar Applications

This section provides a comparison between AuraHome and similar smart security applications by analyzing essential features such as biometric access, motion detection, and AI-powered threat detection.

3.1.1 Key Features Explained

In this section, we provide a brief explanation of each key feature that will be used to compare AuraHome with other similar security applications.

1. **Biometrical Door Locking (Face, Finger):**

This feature refers to securing the main entrance using biometric authentication methods such as fingerprint and facial recognition. It ensures that only authorized individuals can access the home. In AuraHome, this system is enhanced with anti-spoofing detection to prevent unauthorized access using printed or digital photos.

2. **Motion & Alarm Detection (Timed Activation):**

This refers to the system's ability to detect motion after a specific time during the night. AuraHome activates this feature based on a scheduled command sent through MQTT from the mobile app, allowing the system to monitor the home perimeter only during selected hours to reduce false alarms.

3. **Voice/Emotional Detection (Emergency Trigger):**

AuraHome supports voice-based emergency activation by listening for a wake word and analyzing the speaker's emotional state using Speech Emotion Recognition (SER). If fear or distress is detected, the system can trigger emergency alerts automatically.

4. **Gas/Smoke Detection:**

This feature includes sensors that monitor the environment for gas leaks or abnormal temperature increases. When thresholds are exceeded, the system notifies the user instantly, helping to prevent fires or hazardous situations.

5. **AI Weapon/Masked Detection:**

AuraHome uses computer vision models (YOLOv8) to detect firearms and masked individuals. This adds an extra layer of security by identifying threats that may not be detected by traditional motion sensors alone.

6. **Voice Assistant Integration:**

This feature allows the system to be connected with popular smart assistants like Alexa, Siri, or Google Assistant. While some commercial products support this integration, AuraHome does not currently include it, as the system focuses on local control and privacy.

3.1.2 Comparison with Similar Applications

A comparison was conducted between AuraHome and several existing smart security solutions to evaluate the key features provided by each system. The comparison focused on six essential functionalities:

- 1) Biometrical door locking,
- 2) Timed motion and alarm detection,
- 3) Voice/emotional emergency activation,
- 4) Gas and smoke detection,
- 5) AI-based detection of weapons and masked individuals, and
- 6) Voice assistant integration.

AuraHome integrates biometric authentication through fingerprint and facial recognition, combined with anti-spoofing mechanisms. It features smart activation of motion detection at night, triggered by a scheduled MQTT command. Additionally, it supports emergency mode activation through voice and emotion recognition using speech-based AI models. Other capabilities include gas and smoke detection, and real-time object detection for weapons and masks using YOLOv8. Notably, the system prioritizes local privacy by deploying all AI models on a Raspberry Pi 5 without cloud storage.

In contrast, most compared products offer only partial support for these features. For example, **Lockly Visage Zeno** focuses on biometric access and supports voice assistant integration but lacks AI-based detection and environmental monitoring. **Eufy**

Smart Lock E330 includes motion sensors and voice command functionality, yet does not support emotion analysis or gas detection. **Nest Protect** and **Minut Smart Alarm** specialize in environmental alerts, while **Ambient.ai** is designed solely for AI-based weapon detection. AuraHome combines these functionalities into a unified, customizable, and locally deployed system, providing a more comprehensive approach to smart home security.

The table below presents a detailed comparison of the key features across all systems.

Table 1: Similar Applications

Product Name	Biometrical door locking (Face, Finger)	Motion & alarm detection	Voice/emotional detection	Gas/smoke detection	AI weapon/masked detection	Voice-assistant integration
AuraHome	both	✓	✓	✓	✓	X
Lockly Visage Zeno [3]	both	X	X	X	X	✓ (Alexa/Siri/Google)
Eufy Video Smart Lock E330 [4]	Fingerprint only	✓	X	X	X	✓ (Alexa/Google)
Nest Protect [5]	X	X	X	✓	X	X
Minut Point Smart Alarm [6]	X	✓	X	✓	X	✓ (Alexa/Google)
Ambient.ai Weapon Detection [7]	X	X	X	X	✓	X

3.2 Literature Review

3.2.1 Weapons and Masked Detection Core

3.2.1.1 Weapon Detection with FMR-CNN and YOLOv8 for Enhanced Crime Prevention and Security [8]

Summarized Abstract:

This paper proposes a hybrid deep learning model that combines FMR-CNN and YOLOv8 to improve real-time weapon detection accuracy in surveillance environments. The model is designed for urban safety systems with a focus on fast inference and minimal false positives.

Goal:

To enhance weapon detection performance by combining the speed of YOLOv8 with the focused feature extraction capabilities of FMR-CNN.

Methodology:

- YOLOv8 is used for initial object localization.
- FMR-CNN refines detection by extracting region-specific features.

- The architecture applies dual-stage processing to reduce misclassification of non-threatening objects.

Dataset:

- Public surveillance and weapon datasets including Open Image Dataset V6 and custom-labeled firearm video clips.

Models/Algorithms:

- YOLOv8 as the base detector.
- FMR-CNN (Feature-Mapped Region CNN) for post-processing and verification.

Results/Evaluations:

- Achieved a 92.4% mAP score for weapon detection.
- Significant reduction in false positives compared to using YOLOv8 alone.
- Real-time performance maintained at 25 FPS on mid-range GPUs.

Limitations/Obstacles:

- Slight latency due to dual-stage processing.
- Requires high-quality video for optimal performance.

3.2.1.2 Deep Learning and YOLOv8 Utilized in an Accurate Face Mask Detection System [9]

Summarized Abstract:

This study presents a deep learning-based system that leverages YOLOv8 to identify whether individuals are wearing face masks in public spaces. The system is optimized for both indoor and outdoor conditions using various real-world datasets.

Goal:

To provide a fast and accurate face mask detection system to support public health and security measures in crowded environments.

Methodology:

- Training YOLOv8 on annotated datasets with labeled mask/no-mask faces.
- Data augmentation applied to simulate diverse lighting and angle conditions.
- Real-time detection pipeline implemented for live video surveillance.

Dataset:

- Face Mask Detection Dataset (FMDD), RMFD, and MAFA datasets.
- Augmented with surveillance footage for real-world validation.

Models/Algorithms:

- YOLOv8 for detection with customized anchor box optimization.
- Preprocessing includes histogram equalization and image normalization.

Results/Evaluations:

- Achieved over 95% accuracy in distinguishing masked from unmasked faces.
- Maintained 30+ FPS in real-time deployment.
- Robust against partial occlusion and head tilts.

Limitations/Obstacles:

- Reduced accuracy in extreme lighting conditions.
- Less effective on low-resolution video inputs.

3.2.2 Speech Emotion Recognition Core

3.2.2.1 Improved Speech Emotion Recognition Using Transfer Learning on Spectrograms (Li et al., 2022) [10]

Summarized Abstract:

This paper investigates the application of transfer learning for speech emotion recognition (SER) by converting audio signals into spectrogram representations and utilizing pre-trained image classification models such as ResNet and VGG. The approach achieves a notable improvement in classification accuracy across several benchmark datasets, particularly IEMOCAP.

Goal:

- Apply transfer learning using image-based CNNs to improve SER performance.
- Achieve competitive results on established emotion recognition benchmarks.

Methodology:

- Audio signals are converted into spectrograms (log-mel).
- Pre-trained ResNet and VGG models from ImageNet are adopted as CNN backbones.
- Final classification layers are fine-tuned for emotion categories.
- Spectrogram augmentation and careful tuning are applied to optimize performance.
- Evaluation includes cross-dataset validation with a focus on 4-class IEMOCAP.

Dataset:

- IEMOCAP, RAVDESS, EMO-DB, and TESS.
- Acted emotional speech recordings under controlled conditions.
- IEMOCAP used with the standard four-emotion subset: angry, happy, sad, neutral.

Models / Algorithms:

- ResNet and VGG with ImageNet pretraining.
- Spectrogram input with customized output layers for emotion classification.
- Cross-entropy loss and transfer-learning optimizations applied.

Results / Evaluations:

- Achieved 84.3% accuracy on IEMOCAP (4-class).
- Transfer learning outperformed traditional CNN-from-scratch approaches.
- Consistent improvements observed across multiple datasets.

Limitations / Obstacles:

- Limited generalization due to domain mismatch and acted dataset bias.
- High computational demand due to large CNN models.
- Class imbalance and lack of spontaneous speech limit broader applicability.
- Accuracy not directly comparable across studies using different class subsets.

3.2.2.2 Speech Emotion Recognition Using Deep 1D & 2D CNN LSTM Networks

(Zhao et al., 2019) [11]

Summarized Abstract:

This paper presents a hybrid deep learning architecture for speech emotion recognition that combines time-domain features (MFCC) with time–frequency representations (log-mel spectrograms). The model integrates 1D and 2D convolutional layers followed by LSTM units to capture temporal dependencies.

Goal:

- Combine temporal and frequency-based features to improve SER performance.
- Leverage complementary CNN streams and LSTM units to enhance context modeling.

Methodology:

- Audio preprocessing includes segmentation and normalization.
- MFCCs are processed through 1D CNN layers; spectrograms through 2D CNNs.
- Features are fused and fed into LSTM layers.
- Emotion classification is performed via SoftMax.
- Training conducted with cross-entropy loss and regularization techniques.

Dataset:

- IEMOCAP (4-emotion subset), RAVDESS.
- Standard data splits with speaker-independent evaluation.
- Acted emotional data used for training and testing.

Models / Algorithms:

- 1D CNN + 2D CNN for dual-feature extraction.
- LSTM for modeling temporal dependencies.
- Fusion via concatenation prior to LSTM processing.

Results / Evaluations:

- Achieved 71.8% accuracy on IEMOCAP (4-class).
- Outperformed individual CNN stream baselines.
- Analysis included confusion matrices and per-class metrics.

Limitations / Obstacles:

- Increased model complexity and inference time.
- Generalization limited by dataset size and emotion distribution.
- Fusion strategy and parameter tuning impact performance.
- Performance degradation possible in speaker-independent scenarios.

3.2.3 Face Recognition Core

3.2.3.1 Measuring the Re-identification Risk of Imaging Participants Using DeepFace
(Prakaashana et al., 2025) [12]

Summarized Abstract:

This study examines the potential privacy risks associated with open-source face recognition software. It demonstrates how DeepFace can be utilized to re-identify research participants by comparing MRI renders to real photographs using face embeddings and SVM classification.

Goal:

- Assess the ability of face recognition systems to re-identify individuals in medical imaging data.
- Evaluate the privacy vulnerabilities introduced by open-source facial recognition tools.

Methodology:

- Extraction of face embeddings using DeepFace.
- Application of Support Vector Machines (SVM) to match MRI-rendered images with corresponding facial photographs.
- Controlled dataset of imaging participants used to evaluate match likelihood.

Dataset:

- MRI scans and associated facial photographs of study participants.
- Ground-truth labels for evaluating match accuracy.

Models / Algorithms:

- DeepFace for facial feature embedding.
- SVM for classification and similarity matching.

Results / Evaluations:

- Achieved re-identification accuracy between 51% and 59%.
- Demonstrated a tangible privacy threat using publicly available tools.

Limitations / Obstacles:

- Accuracy varies significantly depending on image quality and rendering technique.
- Re-identification risk depends on availability of external facial image databases.
- Ethical and legal implications must be considered for real-world usage.

3.2.3.2 Real-Time Facial Emotion Recognition Using YOLOv8 and DeepFace (Ihsan et al., 2025) [13]

Summarized Abstract:

This paper proposes a real-time intelligent surveillance system that integrates YOLOv8 for object detection with DeepFace for facial emotion recognition. The system is designed to detect suspicious behaviors and recognize emotional states in real-time video streams.

Goal:

- Develop an integrated system for real-time facial emotion detection in surveillance settings.
- Enhance public safety through behavioral and emotional awareness.

Methodology:

- YOLOv8 is employed for face detection in live video streams.
- DeepFace performs facial emotion classification on detected faces.
- The system analyzes emotions in real time to identify abnormal or suspicious activities.

Dataset:

- Likely utilizes standard facial emotion datasets (e.g., FER2013 or custom surveillance datasets).
- Real-time video feeds used for validation under practical conditions.

Models / Algorithms:

- YOLOv8 for object and face localization.
- DeepFace for facial emotion analysis.
- Fusion of detection and classification components in a live processing pipeline.

Results / Evaluations:

- Achieved an accuracy of 93.4% in facial emotion recognition.
- Demonstrated effective performance in real-time conditions.

Limitations / Obstacles:

- Performance may degrade under low lighting or occlusions.
- Emotion misclassification due to subtle expressions or cultural differences.
- Computational demands for real-time processing in edge devices.

Chapter 4: System Analysis

4.1 Requirements Specification

This section defines the functional and non-functional requirements of the AuraHome IoT smart home management system. Functional requirements are categorized by user roles and system components, while non-functional requirements address system performance, security, and usability.

Manage Users: Create, update, enable/disable users with different roles (Admin, Technician, Customer).

Manage Devices: Add devices to customer accounts, configure controls and sensors, set up MQTT topics.

Manage Controls: Create and configure control entities for devices with MQTT topic assignments.

Manage Sensors: Create and configure sensor entities for devices with MQTT topic assignments.

Manage Subscription Plans: Create subscription plans with pricing tiers, device limits, and subscription periods.

Manage Subscriptions: Create customer subscriptions manually, cancel subscriptions, and monitor expiration.

Manage Payment Methods: Add and configure payment method options for customers.

View Audit Logs: Access comprehensive activity logs with filters by user, action, date, or event type to track system modifications.

Send Bulk Notifications: Send push notifications simultaneously to all customers. Compose notification title, message body, and optional data payload.

Generate Reports: Create comprehensive reports for audit logs, devices, users, and subscriptions. And export reports as PDF documents for documentation, compliance, and analysis purposes.

View Dashboard Statistics: Monitor system metrics including total users, devices, subscriptions, and usage analytics.

Publish Sensor Data: Collect and publish sensor readings to MQTT broker at configured intervals.

Execute Control Commands: Receive control commands from MQTT, execute commands, and publish state updates.

Send Emergency Alerts: Detect emergency conditions and send FCM notifications.

Request AI Processing: Send requests from ESP32 devices to Raspberry Pi via local LAN for specific AI Model.

Response with AI Analysis Results: Return processed results from AI Models to ESP32 devices for action execution.

Authenticate MQTT Users: Verify user credentials for MQTT connections.

Route MQTT Messages: Forward messages between devices, mobile apps, and backend services.

Authentication: Login with email/password. Verifying email and changing password is mandatory for the first login.

View Profile: Access personal account information including username, email, phone number, address, account creation date, and current subscription status.

Change Password: Update account password by providing current password for verification and entering new password that meets security requirements (minimum 12 characters, uppercase, lowercase, numbers, and special characters).

Reset Password (Forgot Password): Request password reset when unable to login. Requires admin approval flag to be enabled on account. System sends OTP code to registered email for verification before allowing password reset.

Enable Two-Factor Authentication: Activate additional security layer using either email-based OTP (6-digit code sent to registered email) or Authenticator app (Google Authenticator, Authy) for login verification, it is need current password before enabling.

Saving Backup Codes: After enabling Two-Factor Authentication via an Authenticator app the customer can copy or download PDF containing 10 backup codes.

Consume Backup Code: If the customer does not have access to the Authenticator app, customer can use one of the saved backup codes to complete login.

Disable Two-Factor Authentication: Deactivate two-factor authentication by providing current password before disabling.

View Available Subscription Plans: Browse all active subscription plans with pricing, device limits, and subscription periods.

View Current Subscription Status: Check active subscription details including plan name, start/end dates, days remaining, and device limit.

View Subscription History: Access complete subscription history with all previous subscriptions (active, expired, cancelled).

Check Device Limit: Verify remaining device slots and whether additional devices can be added to account.

Manage Owned Devices: View all owned devices with their controls, sensors, Update device location.

Control Devices: Send control commands to ESP32 devices via MQTT to execute actions (lock/unlock, set-time, etc.).

View Sensor Data: Monitor real-time sensor readings from devices (temperature, gas level, etc.).

View Notifications: View push notifications from devices for alerts, emergencies, and system.

Interact with Support Chatbot: Access AI-powered support chatbot to ask questions about device setup, troubleshooting, subscription management, and general system usage. Receive instant automated responses and guidance for common inquiries.

Access Local Dashboard: Interact with local web-based dashboard accessible on home LAN Hosted on the “Raspberry Pi”.

Manage Known Faces: Add, view, and delete authorized face profiles for home entry authentication. Customers can register family members and trusted individuals by capturing their facial biometric data, which is stored locally on the Raspberry Pi for privacy. Each face profile includes name, and photo for identification purposes.

View Face Recognition History: Access comprehensive logs of all face detection events at the door, categorized by recognition status. Customers can review known faces (authorized individuals), unknown faces (unregistered persons), and spoofed faces (attempted attacks using photos or masks). Each entry includes timestamp, captured image, confidence score, and access decision.

View Gun & Masked Detection History: Monitor security alerts for potential threats detected by AI models at the door entry point. The system logs all instances where guns or masked people are identified, providing timestamp, captured image, threat type, and confidence level. This enables customers to review suspicious activities and take appropriate security measures.

4.2 Functional Requirements

The **Admin** holds the highest level of authority within the system. This role is assigned to system administrators. They should be able to do the following:

- 1- Manage Users
- 2- Manage Devices
- 3- Manage Controls
- 4- Mange Sensors
- 5- Manage Subscription Plans
- 6- Manage Subscriptions
- 7- Manage Payment Methods

- 8- View Audit Logs
- 9- Send Bulk Notifications
- 10- Generate Reports
- 11- View Dashboard Statistics

The **Technician** role is responsible for technical support and device management. This role focuses on device configuration. They should be able to do the following:

- 1- Manage Devices
- 2- Manage Controls
- 3- Mange Sensors
- 4- View Users
- 5- View Subscriptions
- 6- Generate Reports

The **Customer** is the end-user who manages their smart home devices. They should be able to do the following:

- 1- Authentication
- 2- View Profile
- 3- Change Password
- 4- Reset Password (Forgot Password)
- 5- Saving Backup Codes
- 6- Consume Backup Code
- 7- Enable Two-Factor Authentication
- 8- Disable Two-Factor Authentication
- 9- View Available Subscription Plans
- 10- View Current Subscription Status
- 11- View Subscription History
- 12- Check Device Limit
- 13- Manage Owned Devices
- 14- Control Devices
- 15- View Sensor Data
- 16- View Notifications
- 17- Interact with Support Chatbot

- 18- Access Local Dashboard
- 19- Manage Known Faces
- 20- View Face Recognition History
- 21- View Gun & Masked Detection History

The **ESP32** communicate via MQTT. It should be able to do the following:

- 1- Authenticate MQTT Users
- 2- Publish Sensor Data
- 3- Execute Control Commands
- 4- Send Emergency Alerts
- 5- Request AI Processing

The **Raspberry Pi** operates entirely on the local home LAN without internet connectivity. It provides AI processing and local dashboard capabilities. It should be able to do the following:

- 1- Host The Local Dashboard
- 2- Save The Local Data
- 3- Response with AI Analysis Results

The **MQTT Broker** acts as the central cloud-based message router and communication hub. It serves as the intermediary between the system edges. It should be able to do the following:

- 1- Authenticate MQTT Users
- 2- Route MQTT Messages

4.3 Non-Functional Requirements

Non-functional requirements define the quality attributes, system constraints, and operational characteristics that govern how the AuraHome IoT platform performs, behaves, and meets user expectations.

4.3.1 Security

To ensure data integrity, privacy, and system reliability, the system implements the following safeguards:

I. Data Protection

Encryption:

- **At Rest:**
 - Sensitive data (device tokens, MQTT passwords, Two-factor authentication secrets, device metadata) encrypted using a separate ENCRYPTION_KEY (distinct from Laravel's APP_KEY)
 - Custom Encryptor service ensures secure storage
- **In Transit:**
 - All API communication secured via HTTPS/TLS 1.3
 - MQTT communication uses TLS encryption

Hashing:

- User passwords hashed using bcrypt with salting
- MQTT credentials stored securely

II. Authentication & Authorization

- **Bearer Tokens:** Sanctum-based authentication with separate access and refresh tokens
- **Two-Factor Authentication:**
 - Email-based OTP using random 6-digit codes
 - Authenticator app support (Google Authenticator, Authy)
- **Role-Based Access Control (RBAC):** Spatie Permission package manages roles (admin, technician, customer)
- **Password Reset Security:** Requires password_reset_allowed flag set by admin to prevent unauthorized resets
- **API Rate Limiting:** Throttling on authentication endpoints to prevent brute force attacks

III. MQTT Security

- **Mosquitto Dynamic Security:**

- User-specific namespaces prevent cross-user access
 - Dynamic user/role creation via Admin panel
 - Automatic credential cleanup on user deletion
- **Topic-Level ACL:** Devices restricted to their own namespaces
- **Encrypted Credentials:** MQTT passwords encrypted separately from application keys

IV. Audit Trail

- **Activity Logging:**
 - All Admin/Technician actions logged via HasAuditLog trait
 - Tracks user actions, timestamps, and modified data
 - Immutable audit records for compliance
- **User Actions:** Login attempts, password changes, 2FA events logged
- **Device Events:** Device additions, deletions, control commands tracked

V. Network Security

- **HTTPS Enforcement:** All API endpoints require HTTPS in production
- **CORS Configuration:** Restrict API access to authorized domains
- **CSRF Protection:** Laravel's built-in CSRF tokens for web routes
- **SQL Injection Prevention:** Eloquent ORM and prepared statements

4.3.2 Performance

- **Real-Time Communication:** MQTT protocol ensures low-latency device communication (<100ms typical)
- **Notification Delivery:** FCM push notifications delivered within 1-2 seconds
- **API Response Time:** Average response time < 400ms for authenticated requests
- **Database Optimization:**
 - Eager loading to prevent N+1 queries
 - Indexed columns for fast searches
 - Query caching for frequently accessed data

- **Edge Computing:** Raspberry Pi processes AI models locally, reducing cloud latency
- **Background Processing:** Queue system for background jobs

4.3.3 Efficiency

- **Query Optimization:**
 - Eager loading relationships to reduce database queries
 - Local edge storage via Raspberry Pi (no cloud sync)
 - Lazy loading for non-critical data
- **Response Payload Optimization:**
 - API Resources for consistent JSON formatting
 - Only essential data returned per endpoint
 - Compressed responses for large payloads
- **Caching:**
 - Query result caching for static data
- **Background Jobs:**
 - Asynchronous processing for email sending
 - Queue-based MQTT namespace cleanup
 - Scheduled tasks for subscription expiration checks

4.3.4 Usability

- **Mobile App:**
 - Intuitive and easy-to-navigate Flutter UI
 - Support for Android and iOS
- **Admin Panel:**
 - Filament 4 provides modern, responsive interface
 - Grouped navigation for related resources
 - Color-coded status indicators
 - Quick action buttons for common tasks
 - Search and filter capabilities on all tables
- **Local Dashboard (On Raspberry Pi):**
 - Responsive web design accessible from any browser on home LAN

- Simple controls for non-technical users
- Works without internet connection

4.3.5 Maintainability

- **Clean Architecture:**
 - Service-Repository pattern separates concerns
 - Controllers handle HTTP layer only
 - Services contain business logic
 - Repositories manage data access
- **Design Patterns:**
 - Dependency Injection via interfaces
 - Singleton binding for services
 - Observer pattern for model events
 - Job pattern for asynchronous tasks
- **Code Organization:**
 - Clear folder structure
 - Interfaces in app/Contracts/
 - Reusable traits in app/Traits/
 - Custom validation rules in app/Rules/
- **Documentation:**
 - Comprehensive inline comments
 - API documentation via Postman collections
 - Architecture diagrams
 - Setup guides and README files

4.4 Use Case Diagram

This case diagram clearly shows how user roles interact with key system functions in the AuraHome IoT platform. It provides a simple, visual overview of each actor's responsibilities and how they connect with the overall system.

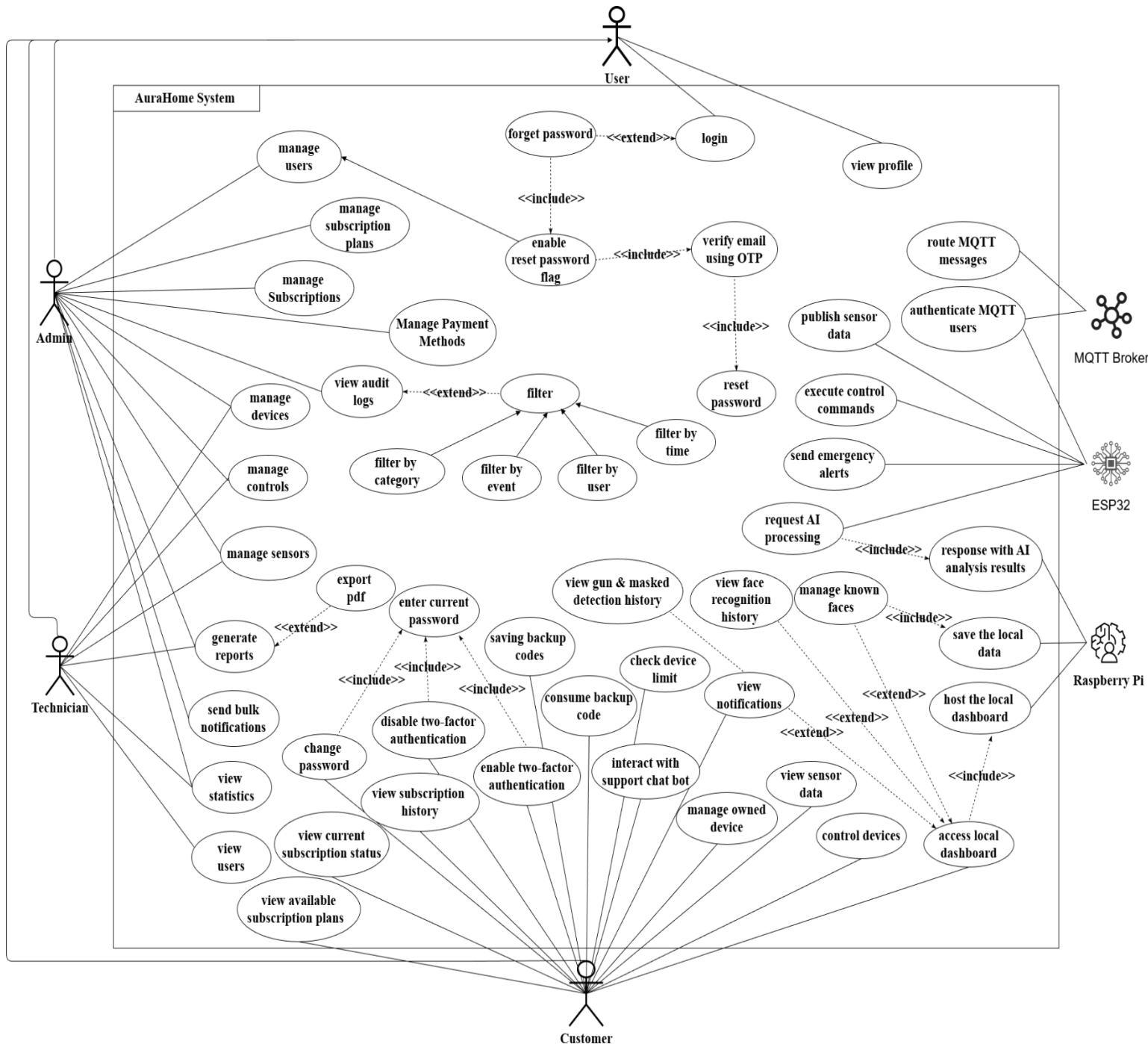


Figure 3. Use Case Diagram

Chapter 5: System Design

5.1 System Architecture

The system consists of three main layers: Hostinger KVM4 cloud infrastructure hosting the Laravel backend with Filament admin panel, MySQL database, and Mosquitto MQTT broker for real-time communication. Edge & IoT Device Layer (LAN) where ESP32 smart devices (door guards, door locks, gas and temperature sensors, and emergency triggers) communicate locally with a Raspberry Pi 5 gateway that runs AI models and maintains a local SQLite database with management dashboard; and client applications including a Flutter mobile app that connects via REST API and MQTT for remote monitoring or control. Third-party services (Sentry for monitoring, Firebase for push notifications, and Hostinger SMTP for emails) integrate with the Laravel backend to provide comprehensive system observability and user notifications.

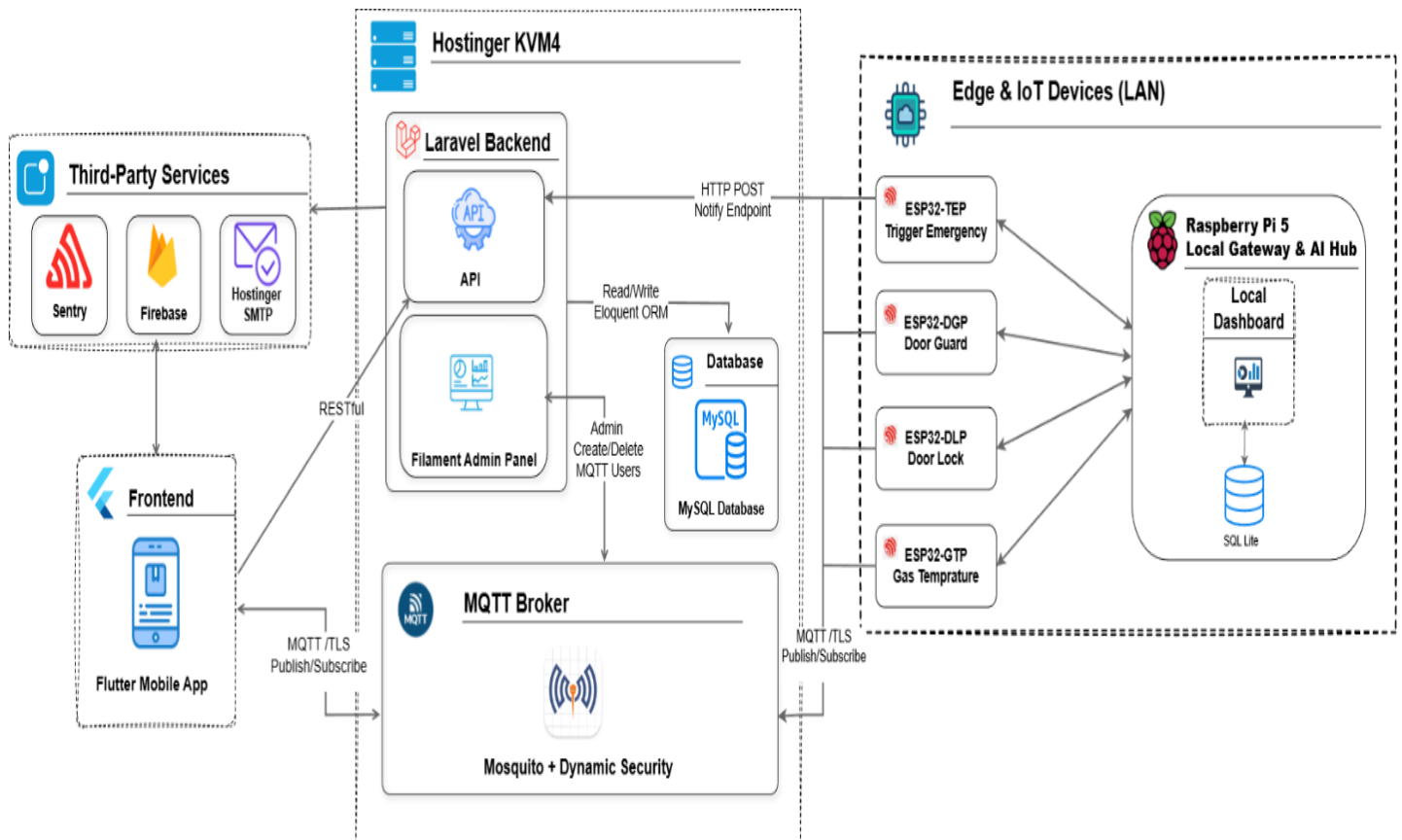


Figure 4: System Architecture

5.2 Entity Relationship Diagram

This ERD delineates the comprehensive relational schema underpinning a sophisticated IoT smart home management system. It methodically maps key entities—users, devices, sensors, controls, audit logs, and role-based access control components—and explicates their intricate interdependencies through cascading foreign key relationships and junction tables. The schema ensures robust data integrity through encrypted credential storage (device_token, mqtt_password), namespace isolation via MQTT username mapping, and systematic governance through comprehensive audit trails that capture all administrative actions with IP tracking and state versioning.

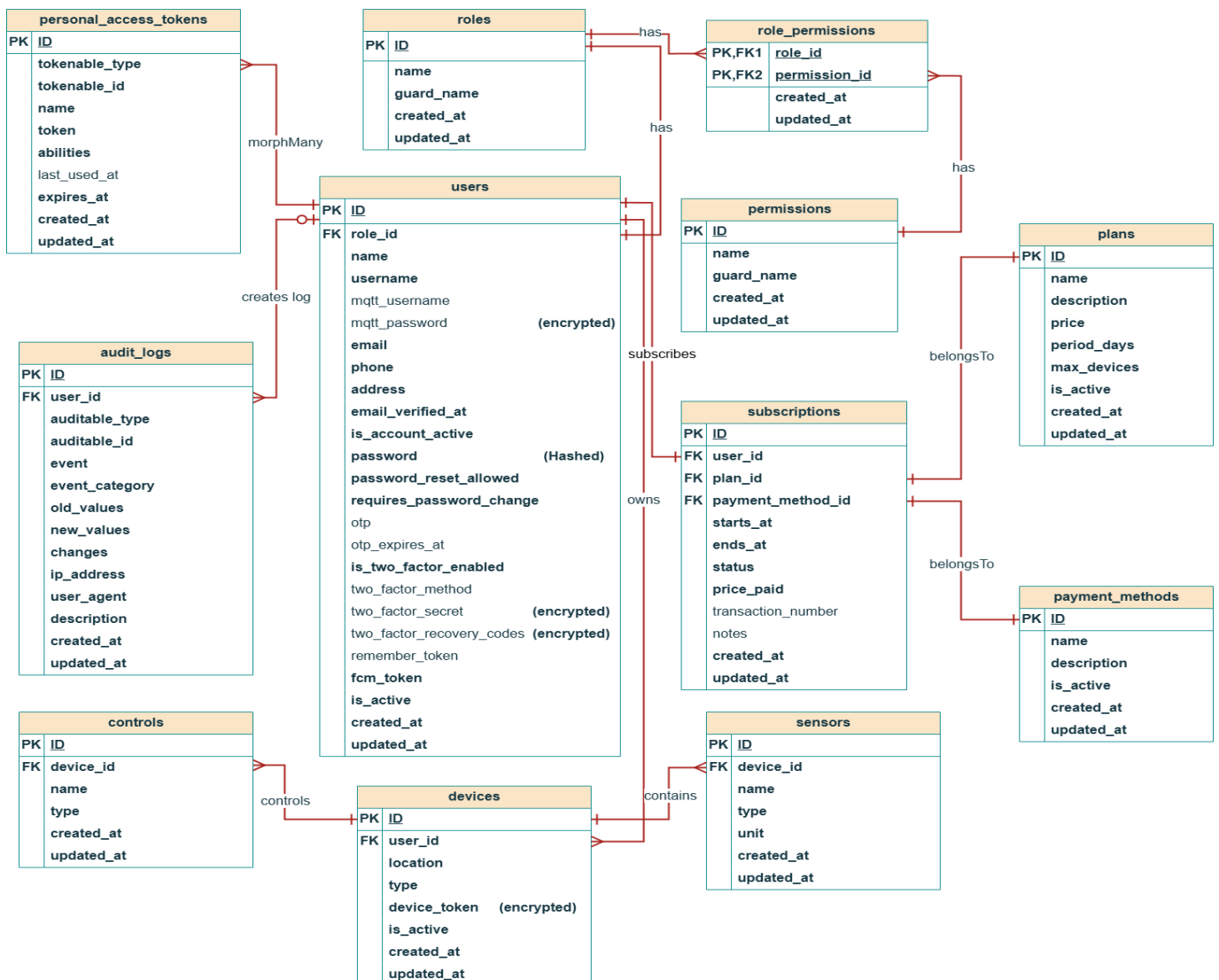


Figure 5: Entity Relationship Diagram

5.3 Sequence Diagram

This section details critical system workflows through UML sequence diagrams, visualizing interactions between system components and actors to ensure secure operations.

5.3.1 User Creation with Automatic MQTT Namespace Provisioning

When an admin creates a new user through Filament, the system automatically generates unique MQTT credentials, creates an isolated namespace in the Mosquitto broker using Dynamic Security Plugin, assigns topic-based permissions through roles, and stores encrypted credentials in the database - enabling secure device communication without manual MQTT configuration.

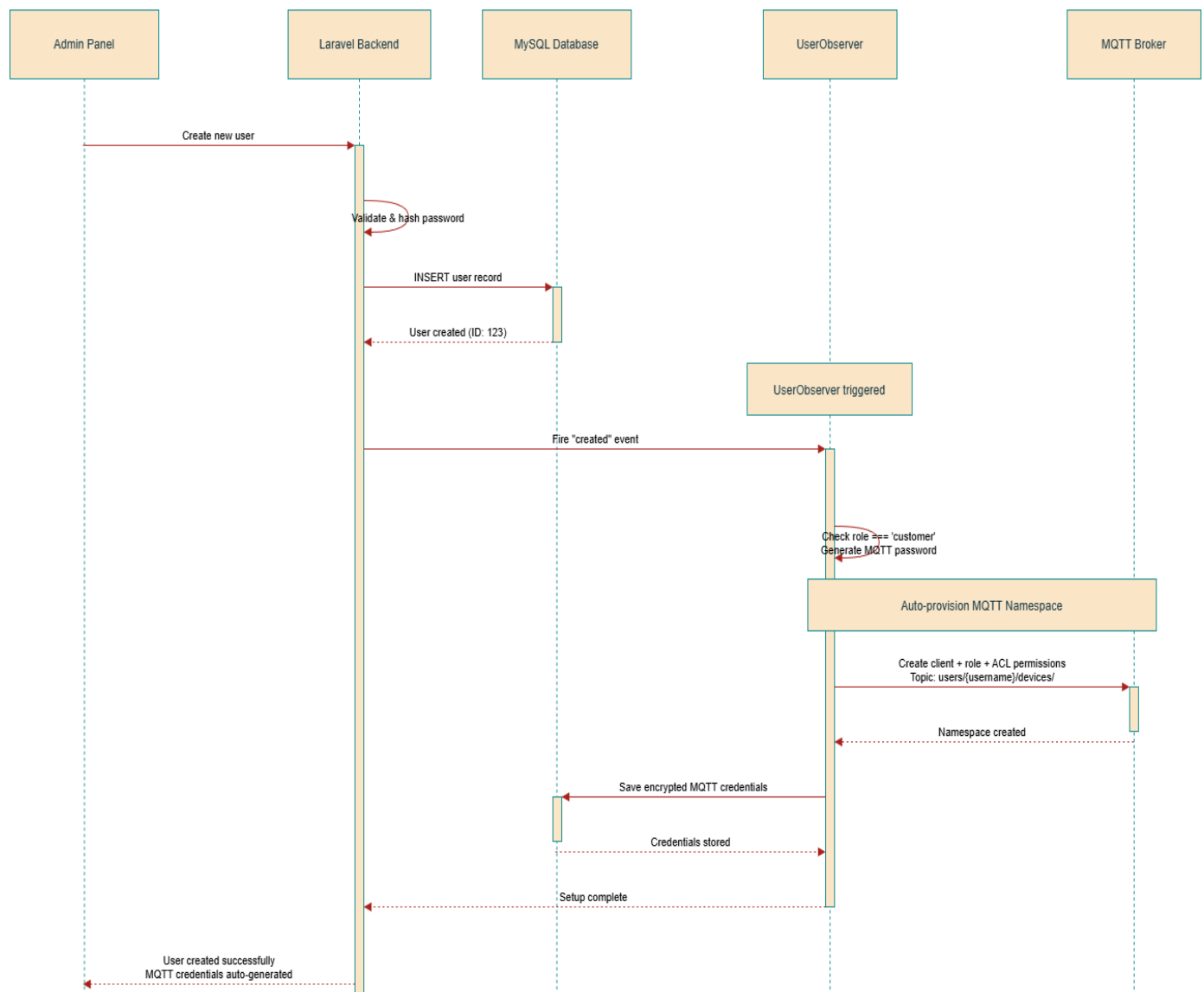


Figure 6: User Creation with Automatic MQTT Namespace Provisioning

5.3.2 Device Creation with Product-Type Based Auto-Provisioning

When creating a device (either from main Devices page or User relation manager), the system automatically generates appropriate sensors and controls based on product type while encrypting device tokens and constructing MQTT topic structures - eliminating manual sensor or control setup.

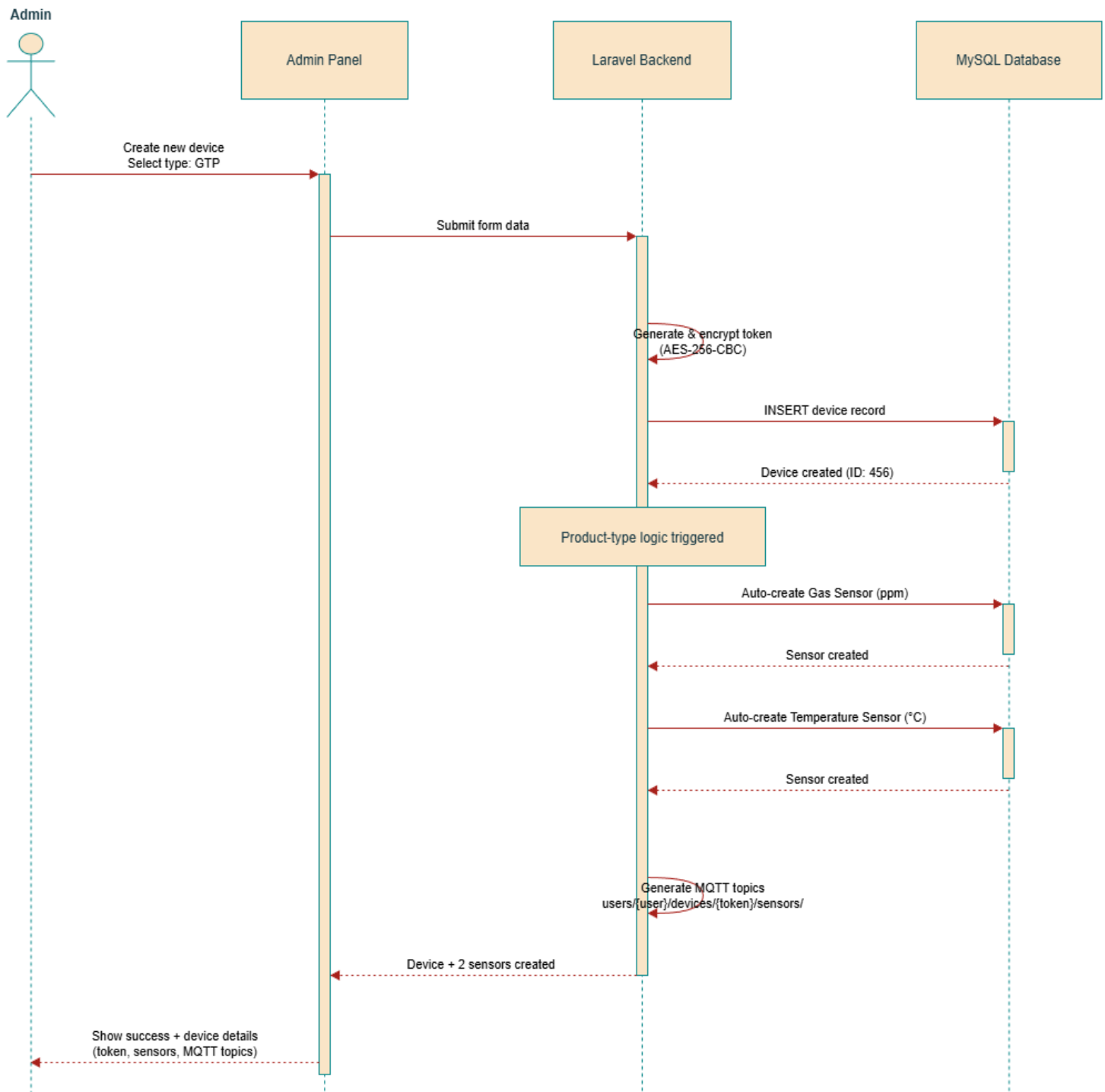


Figure 7: Device Creation with Product-Type Based Auto-Provisioning

5.3.3 Email Verification Reset on Admin Edit

When an admin modifies a user's email address through Filament, the UserObserver automatically resets the email_verified_at field to null, triggering a new verification workflow; the system sends an OTP verification email via Hostinger SMTP to the new address, and the user must verify by submitting the OTP code through the mobile app before the email is marked as verified.

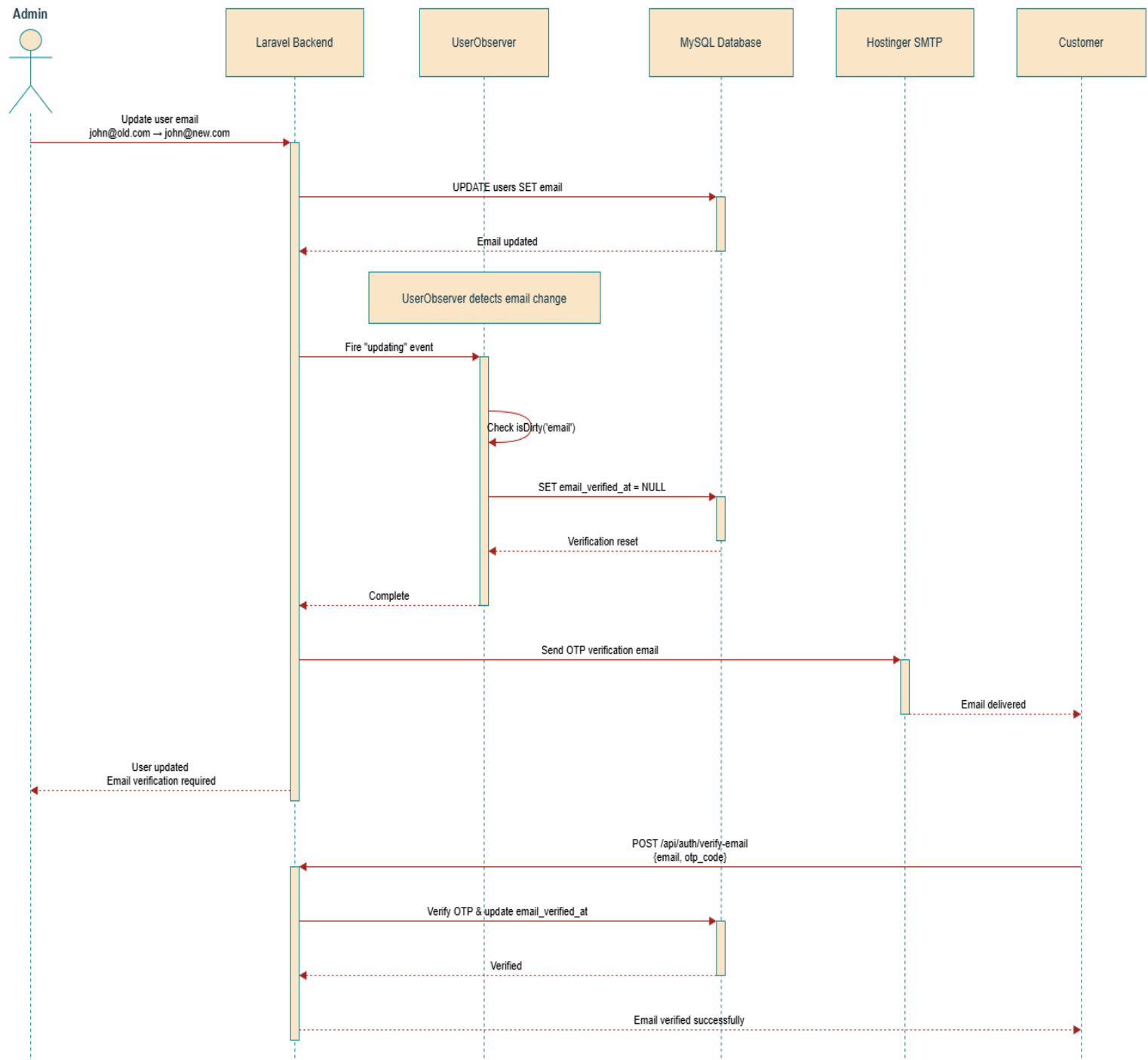


Figure 8: Email Verification Reset on Admin Edit

5.3.4 Async User Deletion with MQTT Cleanup & Retry Logic

When an admin deletes a user, the UserObserver immediately removes the database records (cascading to devices, sensors, controls) and dispatches a background queue job to clean up the user's MQTT namespace asynchronously, preventing UI blocking; if the MQTT broker is unreachable, the job automatically retries 3 times with exponential back off (60s, 120s), and logs failures for manual cleanup via artisan command.

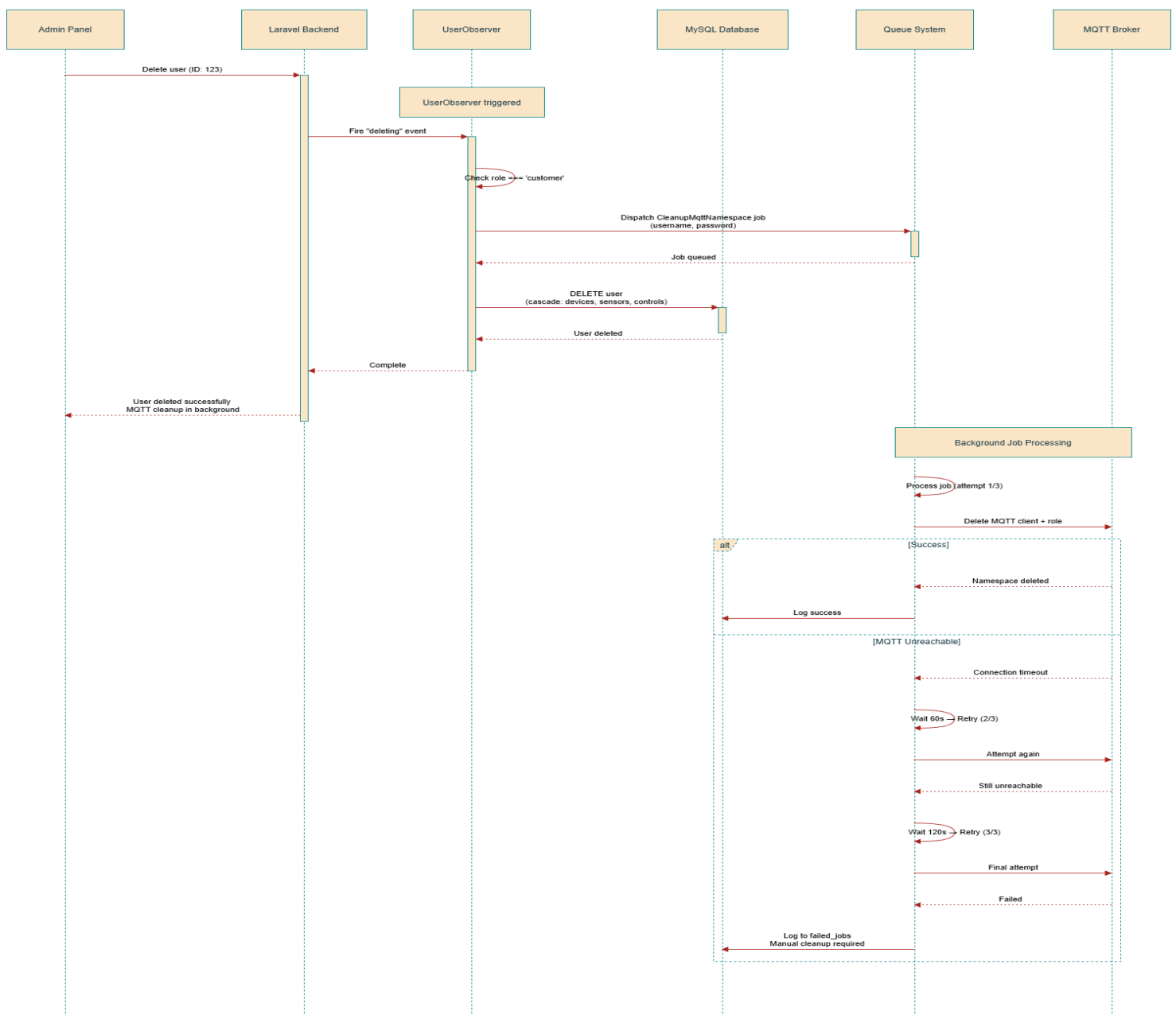


Figure 9: Async User Deletion with MQTT Cleanup & Retry Logic

5.4 Activity Diagram

This section models system processes through UML activity diagrams, visualizing workflow logic and decision paths to demonstrate operational efficiency.

5.4.1 Real-Time Sensor Data Flow

ESP32 devices continuously read sensor values and publish them to MQTT topics; the Flutter mobile app subscribes to these topics and receives real-time updates, displaying live sensor readings.

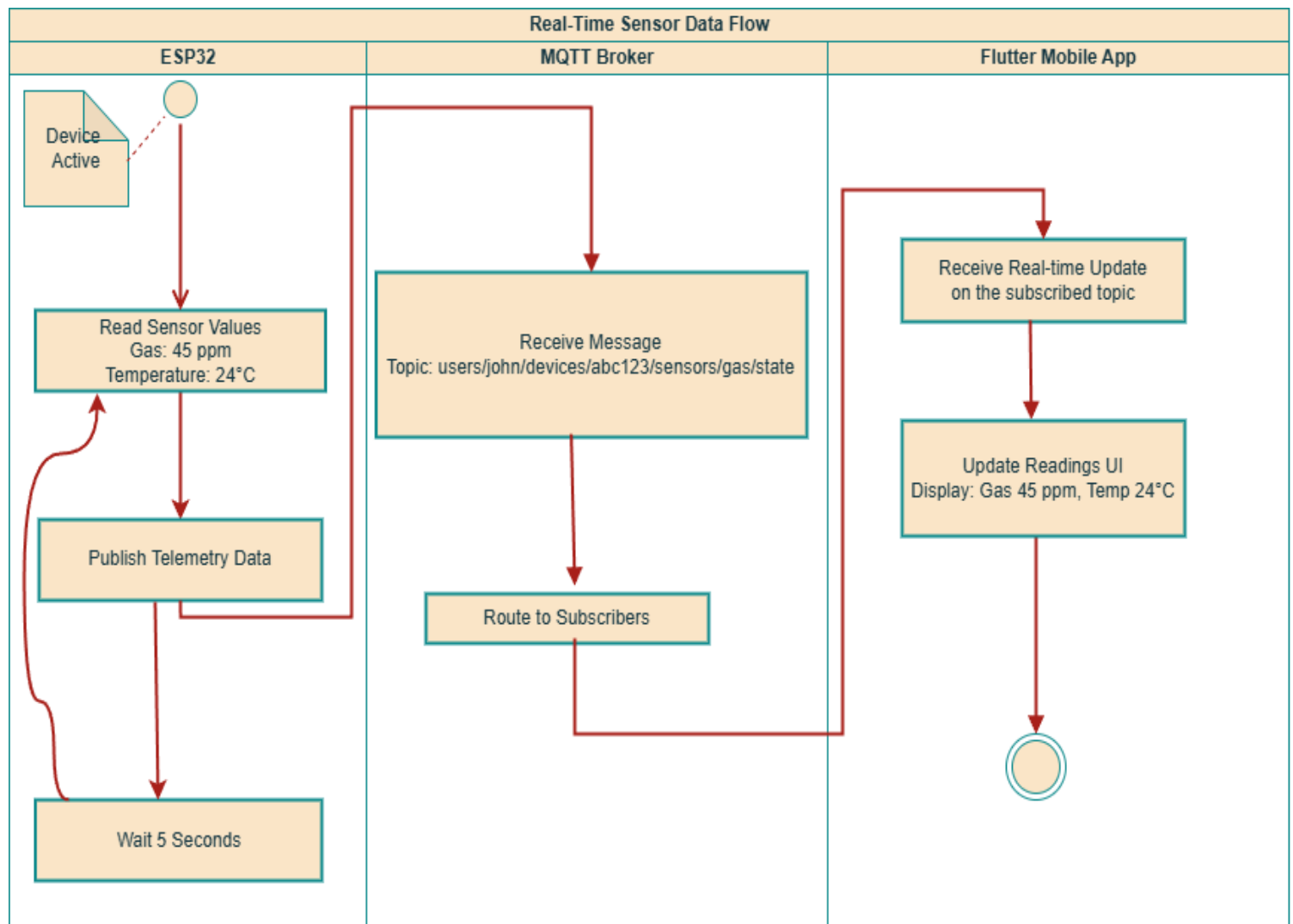


Figure 10: Real-time Sensor Data Flow

5.5 Hardware Design

This section presents the **hardware design** of the AuraHome system and explains how the selected electronic components and device distribution contribute to solving the challenges stated in the **Problem Statement**, especially the need for **reliable detection** and **fast response** in safety-critical home scenarios.

The hardware design focuses on: (1) **what components are used**, (2) **why these components were selected** over alternatives, and (3) how the overall hardware structure supports the system's key features (secure door access, night security, environmental safety, and voice-based emergency triggering).

5.5.1 ESP32 Node Architecture Overview

AuraHome is implemented using **four independent ESP32-based nodes**, where each node is designed as a **separate circuit** responsible for a single product/function. This modular approach allows each node to operate and be tested **individually**, improving maintainability and reducing the impact of failures (a fault in one node does not stop the entire system).

The four ESP32 nodes are:

1. **Door Lock Product (DLP)**

This node implements **Secure Door Access (MFA + Anti-spoofing)** by combining biometric authentication (face and fingerprint) with a keypad password. Anti-spoofing measures are included to reduce the risk of impersonation using printed or digital images.

2. **Door Guard Product (DGP)**

This node supports **Night Security Mode (Motion → Mask/Gun Detection)** using sensor-based motion gating to reduce false alarms (e.g., PIR + ultrasonic + LDR). When suspicious motion is confirmed, the system triggers AI-based analysis for **weapon detection** and **masked person detection**.

3. **Gas Temperature Product (GTP)**

This node provides **Gas and Temperature Safety Monitoring** by

continuously measuring gas levels and temperature. When predefined thresholds are exceeded, the node contributes to immediate alert generation to support early hazard detection.

4. Trigger Emergency Product (TEP)

This node enables **Wake Word Detection and Speech Emotion Recognition (SER)**. An INMP441 microphone listens for a wake word; once detected, the node records a short audio clip (5 seconds) and forwards it to the SER pipeline to help identify emergency-related emotional states (e.g., fear or distress) and trigger an emergency workflow.

Table 2: AuraHome ESP32 nodes, deployment locations, and functional roles

Node abbreviation	Node full name	Recommended node location	Node short description
GTP	Gas Temperature Product	Kitchen (preferred) or any indoor area where gas/heat hazards may occur	Monitors gas leakage and ambient temperature to support early hazard detection and trigger safety alerts when thresholds are exceeded.
DLP	Door Lock Product	At the main door and/or any secure door in the house	Provides secure door access control using multi-factor authentication (e.g., fingerprint, face verification, and keypad) and actuates the door lock accordingly.
DGP	Door Guard Product	Near the home entrance/door area	Enhances entrance security by confirming motion events and enabling detection of masked persons and weapons , reducing false alarms and triggering security actions.
TEP	Trigger Emergency Product	Any indoor room (e.g., living room, bedroom)	Listens for wake-word triggers , records a short audio clip, and supports emergency mode activation based on detected trigger words and emotion-related cues.

5.5.2 Local AI Deployment on Raspberry Pi 5 (Performance and Privacy)

All AI models used in AuraHome (e.g., face recognition and anti-spoofing, weapon/masked detection, and SER) are **deployed locally on a Raspberry Pi 5 installed inside the home**. This design choice improves **response time** (low latency inference) and supports **privacy**, as sensitive data is intended to remain within the home environment rather than being processed or stored in the cloud.

In addition, local deployment supports secure handling of sensitive biometric assets (e.g., authorized faces and fingerprints) and enables operation under home LAN conditions without requiring cloud processing of raw biometric/audio/video streams.

5.5.3 Per-Node Circuit Design and Component Selection

In this subsection, each of the four ESP32 circuits is described **independently**. For every circuit, we list the main electronic components and summarize (1) the **component name/type**, (2) its **operating voltage**, (3) its **role and why it was selected**, and (4) an **approximate current consumption** (typical values from common modules and datasheets; actual values may vary by vendor and configuration such as backlight brightness).

5.5.3.1 Door Lock Product (DLP) Circuit

This subsection describes the **Door Lock Product (DLP)** circuit and the electronic components used within it. For each component, we present its operating voltage, purpose within the circuit, justification for selection, and approximate current consumption (typical values; the exact value may vary depending on the specific module and configuration).

LCD 16×2 (I2C)

Operating voltage.

The LCD operates at **5 V**.

Estimated current consumption.

Typically **20–120 mA**, mainly depending on the backlight intensity.

Role in the circuit.

The LCD acts as the primary user interface at the door. It is used to guide the user through the authentication flow and display system feedback messages such as:

- “Enter password”
- “Password correct / incorrect”
- “Face recognized / not recognized”

Reason for selection.

An **I2C LCD** was selected because it requires only **four pins** (VCC, GND, SDA, SCL). This is important due to the limited available GPIO pins on the ESP32, compared to the parallel LCD version which consumes significantly more pins.

4×3 Matrix Keypad

Operating voltage.

The keypad is a passive switch matrix; it does not require a dedicated supply. It is scanned using ESP32 **3.3 V GPIO logic**.

Estimated current consumption.

Negligible during operation (typically < **1 mA average**, depending on pull-up/pull-down and scan rate).

Role in the circuit.

The keypad is responsible for:

- Entering the **door password** (as a second factor in MFA)
- Allowing **password updates** when needed

Reason for selection.

A 4×3 keypad provides a simple and reliable input method without extra hardware, and it integrates directly using GPIO row/column scanning.

Push Button (Start Trigger)

Operating voltage.

Connected to an ESP32 GPIO, therefore it operates at **3.3 V logic**.

Estimated current consumption.

~0 mA when idle, and typically < **1 mA when pressed** (through the pull resistor).

Role in the circuit.

The push button acts as a **start/activation trigger**. When the user arrives home, pressing this button initiates the authentication sequence (biometrics first, then password entry). This makes the interaction explicit and avoids unintended triggers.

Reason for selection.

A push button is a low-cost and robust method to allow intentional user-driven initiation of the access workflow.

Buzzer (Audio Feedback)**Operating voltage.**

Driven from the ESP32 at **3.3 V**.

Estimated current consumption.

Typically **10–30 mA** for common small active buzzers (varies by module).

Role in the circuit.

The buzzer provides interactive audio feedback and status indication, such as:

- Door opened
- Correct password
- Wrong password
- General success/failure cues

Reason for selection.

Audio feedback improves usability and accessibility, especially when the user cannot continuously rely on visual messages from the LCD.

Relay Module (Electronic Lock Control)**Operating voltage.**

The relay is controlled by the ESP32 via a **3.3 V control signal**, while the lock itself operates at **12 V**.

Estimated current consumption.

The relay module typically draws **70–90 mA** when energized (coil current; module-dependent).

Role in the circuit.

The relay module allows the ESP32 to control the **12 V electronic door lock** safely. The lock's power line is routed through the relay contact so that the ESP32 can enable or disable the lock current without being directly exposed to the 12 V load.

Reason for selection.

A relay provides electrical isolation between the low-voltage control side (ESP32) and the higher-voltage actuator side (12 V lock), making it a simple and reliable switching solution for the door lock mechanism.

5.5.3.2 Door Guard Product (DGP) Circuit

The **Door Guard Product (DGP)** circuit is designed to detect **suspicious activity near the house entrance**, particularly during late hours. It combines distance sensing and motion sensing to reduce false triggers, and it controls two external **12 V LED lights** through relay modules: one light is automatically enabled based on ambient darkness, and the other is enabled when suspicious movement is detected to increase deterrence.

*Ultrasonic Distance Sensor***Operating voltage.**

The ultrasonic sensor is powered at **3.3 V**.

Estimated current consumption.

Typically around **10–20 mA** during active ranging.

Role in the circuit.

This sensor is used to detect the presence of objects close to the door area during the defined night-time window. It provides an estimate of distance, which allows the node to determine whether a person/object is near the entrance.

Interface and wiring.

It uses **four pins**: **VCC**, **GND**, **TRIG**, and **ECHO**. The ESP32 triggers the measurement via TRIG and reads the pulse width on ECHO.

Reason for selection.

Ultrasonic sensing provides a simple and low-cost method for **proximity detection**, which is useful for confirming that something is near the door even if it is not moving significantly.

PIR Motion Sensor**Operating voltage.**

The PIR sensor operates at **5 V**.

Estimated current consumption.

Typically **50–100 μA idle** and around **< 1 mA** in common modules (overall consumption depends on the specific PIR board).

Role in the circuit.

The PIR sensor detects **movement** based on changes in infrared radiation. This complements the ultrasonic sensor: while ultrasonic can indicate that an object is close (even if relatively stationary), PIR is effective at detecting **motion**, including movement that may occur at a slightly larger distance.

Reason for selection.

Using PIR alongside ultrasonic improves reliability by combining two different sensing principles. This helps reduce false alarms and provides better confirmation that a real motion event is happening near the entrance.

LDR (Light-Dependent Resistor)**Operating voltage.**

The LDR is used as part of a voltage divider read by the ESP32 ADC at **3.3 V**.

Estimated current consumption.

Very low, typically **< 1 mA** depending on the chosen resistor value in the divider.

Role in the circuit.

The LDR measures ambient light level to determine whether it is dark enough to enable night lighting. When the light level falls below a chosen threshold (night condition), the circuit automatically turns on the **first 12 V LED**.

Reason for selection.

An LDR provides a simple and low-cost way to implement **automatic night-light behavior** without requiring complex scheduling logic or external time sources.

*Two Relay Modules (12 V LED Control)***Operating voltage.**

The relay modules are controlled by the ESP32 using **3.3 V control signals**, while the LEDs are powered from a **12 V supply**.

Estimated current consumption.

Each relay module typically draws **~70–90 mA** when energized (coil current; module-dependent).

Role in the circuit.

Two relay modules are used to control two separate 12 V LED lights:

- **Relay 1 (Ambient light control):** turns **LED 1** on/off based on the LDR (darkness detection).
- **Relay 2 (Deterrence light):** turns **LED 2** on when a suspicious motion event is detected (to deter the intruder and increase visibility).

Reason for selection.

Relay modules enable safe switching of **higher-voltage loads (12 V LEDs)** using a low-voltage ESP32 control signal, while keeping the power switching isolated from the microcontroller.

5.5.3.3 Gas Temperature Product (GTP) Circuit

The **Gas Temperature Product (GTP)** circuit is responsible for environmental safety monitoring inside the home. Its main purpose is to detect **abnormal temperature rise** (as an early fire indicator) and **gas leakage**, then provide immediate local alerts using audio and visual indicators.

*DS18B20 Temperature Sensor***Operating voltage.**

The DS18B20 operates at **3.3 V**.

Estimated current consumption.

Typically **~1 mA during temperature conversion**, and significantly lower while idle (commonly in the microamp range depending on sampling rate).

Role in the circuit.

This sensor measures ambient temperature to detect unusual increases that may indicate a fire hazard. The node continuously monitors temperature readings and triggers alerts when the temperature exceeds a predefined threshold.

Reason for selection.

DS18B20 is suitable for reliable temperature sensing and can provide stable measurements with simple wiring, making it appropriate for continuous safety monitoring in a home environment.

MQ-4 Gas Sensor

Operating voltage.

The MQ-4 module is supplied at **3.3 V** (as implemented in this system).

Estimated current consumption.

Typically **~100–200 mA**, because MQ-series sensors include an internal heater that consumes the majority of the power.

Role in the circuit.

The MQ-4 sensor is used to detect combustible gas leakage. When the measured gas concentration exceeds a defined threshold, the node generates a hazard event and activates local alerts.

Reason for selection.

MQ-4 is a widely used gas-sensing module suitable for domestic gas leakage detection scenarios and provides a practical approach for early warning in smart home safety systems.

Buzzer (Hazard Alert)

Operating voltage.

Driven by the ESP32 at **3.3 V**.

Estimated current consumption.

Typically **10–30 mA** (depends on the buzzer type/module).

Role in the circuit.

The buzzer provides an immediate audible alarm when a hazardous condition is detected (gas leakage or abnormal temperature rise). This local alert is important even if the user is not actively using the mobile application.

Reason for selection.

Audio alarms improve response time and awareness, especially for safety-critical events where immediate attention is required.

*Status LEDs (2 × 3.3 V)***Operating voltage.**

Each LED operates at **3.3 V** (with a suitable series resistor).

Estimated current consumption.

Typically **5–10 mA per LED**, depending on the resistor value and brightness.

Role in the circuit.

Two LEDs are used as simple status indicators to show the current state of the node (e.g., normal operation vs. warning/alarm). They provide quick visual feedback to the user when near the device.

Reason for selection.

LEDs are low-cost, low-power indicators that improve usability and allow immediate understanding of system status without requiring a screen.

5.5.3.4 Trigger Emergency Product (TEP) Circuit

The **Trigger Emergency Product (TEP)** circuit enables voice-based emergency triggering. Its main role is to continuously listen for a predefined **wake word**; once detected, the node records a short audio clip (5 seconds) to support the emergency analysis workflow. The circuit also provides immediate feedback to the user through a buzzer when the wake word is successfully detected.

*INMP441 Microphone (I2S)***Operating voltage.**

The INMP441 operates at **3.3 V**.

Estimated current consumption.

Typically around **1–2 mA** (module-dependent).

Role in the circuit.

The INMP441 is responsible for capturing audio input. It continuously listens for the **wake word**, and once the wake word is detected, it triggers the recording of a **5-second audio segment**.

Reason for selection.

INMP441 was selected because it is a digital I2S microphone suitable for ESP32-based audio capture. Using a digital microphone improves signal stability compared to analog alternatives and simplifies integration with the ESP32 audio pipeline.

*Buzzer (Wake Word Feedback)***Operating voltage.**

Driven by the ESP32 at **3.3 V**.

Estimated current consumption.

Typically **10–30 mA**, depending on the buzzer type.

Role in the circuit.

The buzzer provides an audible confirmation when the node detects the wake word. This feedback informs the user that the system has started recording the 5-second clip and that the emergency-trigger workflow has been initiated.

Reason for selection.

Audio feedback improves usability and reduces ambiguity, especially in emergency situations where the user needs clear confirmation without relying on a screen.

5.6.1 Security Objectives and Protected Assets

This subsection defines the **main security objectives** of AuraHome and identifies the **assets that must be protected**. The security design focuses on preserving **confidentiality, integrity, and controlled access** across the communication layer, user authentication, and locally stored sensitive data.

5.6.1.1 Protection of device credentials (device_token and MQTT credentials)

AuraHome treats **device_token** and **MQTT credentials (mqtt_username, mqtt_password)** as highly sensitive assets because they can be abused to impersonate devices or access private MQTT topics. The design protects these credentials in three places:

- **In transit:** credentials are exchanged only through encrypted channels (TLS/HTTPS) to prevent eavesdropping and credential theft.
- **At rest (database):** credentials are stored in encrypted form in the database to reduce the impact of database exposure.
- **On the mobile device:** MQTT credentials used by the app are stored in encrypted form (e.g., using AES-256-CBC in this design) to reduce the risk of leakage from local storage.

5.6.1.2 Protection of control and telemetry traffic (MQTT messages)

AuraHome exchanges two main MQTT traffic types: **telemetry** (sensor readings/events) and **control commands** (e.g., enabling night mode or triggering actions). The design protects these messages using **TLS on MQTT port 8883**, supported by certificates, to prevent message interception or tampering (MITM / modification / sniffing).

5.6.1.3 Securing user access to the mobile application (2FA/OTP and optional biometric login)

To reduce the risk of unauthorized account access, the security design supports stronger authentication mechanisms:

- **Two-Factor Authentication (2FA) (optional):** the user can enable 2FA using either **OTP via email** or an **authenticator application with backup codes** (depending on the configured method).
- **Email OTP verification workflow:** when a user's email is changed, the system enforces re-verification by sending an OTP code that must be submitted through the mobile app before marking the email as verified.
- **Device biometric login (optional):** the user may enable login using the phone's biometric authentication (fingerprint/face) where the biometric template remains securely stored by the mobile OS. This adds convenience without exposing biometric data to the server.

5.6.1.4 Topic isolation and authorization (per-user namespace + ACL rules)

AuraHome secures MQTT topic access by ensuring that each user/device operates inside an isolated **namespace** and that permissions are enforced through **ACL rules**.

This limits publish/subscribe operations to only the topics required for each user/device (least privilege).

In addition, the system supports automated provisioning where MQTT users and permissions are created and managed through Mosquitto's **Dynamic Security Plugin**, enabling per-user topic rules without manual broker configuration.

5.6.1.5 Privacy-first local storage and processing (Raspberry Pi 5)

AuraHome prioritizes privacy by keeping sensitive biometric and security-related data **inside the home**. Authorized **faces/fingerprints**, **door access attempts** (successful and failed), and security media (e.g., threat-related snapshots) are designed to be stored and processed **locally** on Raspberry **Pi 5**. AI inference is also deployed locally to reduce latency and avoid sending sensitive raw data to the cloud.

5.6.2 Replay Attack Mitigation

AuraHome considers **replay attacks** ineffective in practice because an attacker cannot simply capture a previous MQTT publish message and resend it to trigger the same action. First, control commands are delivered only through an authenticated **MQTT over TLS** connection, which requires valid **MQTT credentials** (username/password). Without the correct password, the attacker cannot establish a broker session and therefore cannot publish protected control topics. Even if the attacker attempts to reuse captured TLS traffic, it will not be accepted, because TLS data is bound to **session-specific encryption/integrity keys** that change per connection; replaying old, encrypted records in a new session will fail. In addition, AuraHome isolates devices using a **per-user namespace** enforced through **ACL rules**, which further prevents publishing to topics outside the authorized scope. However, if valid MQTT credentials are compromised, the attacker may be able to publish within the permissions granted to that account; therefore, the design relies on **least-privilege ACLs**, credential rotation. Device-token rotates each 5 minutes, the new rotated token will be requested from the client side (nodes) using different credentials to make the process more secure.

Chapter 6: Implementation

This chapter presents the implementation details of the AuraHome system. It explains how each system component was practically realized based on the design decisions discussed in previous chapters. The chapter focuses on the applied hardware and software implementations, highlighting the rationale behind selecting specific technologies, libraries, and communication mechanisms.

Each subsystem is described individually to provide a clear understanding of its functionality, internal logic, and interaction with other system components. The implementation process emphasizes modularity, scalability, and reliability, ensuring that each component can operate efficiently within the overall system while maintaining independence when required.

6.0 Common Hardware Implementation Concepts

This section describes implementation aspects that are common across all hardware circuits used in the AuraHome system.

The system is composed of multiple ESP32-based circuits, where each circuit represents an independent functional node responsible for a specific task, such as door access control, motion detection, environmental monitoring, or emergency triggering. Each circuit is designed to operate **fully independently** from the other circuits, without requiring direct hardware/software level dependency or synchronization. This modular architecture improves system robustness, simplifies debugging, and allows individual components to continue functioning even if other nodes are offline.

To ensure flexibility during real-world deployment, each ESP32 node supports **independent wireless network configuration**. In the event of a change in the home Wi-Fi network, such as a modified SSID or password, the network credentials of each circuit can be updated without requiring physical reprogramming or redeployment of firmware.

Wi-Fi credential configuration is performed wirelessly through a Bluetooth interface. Each ESP32 exposes a Bluetooth serial service that allows secure transmission of the new network parameters. The mobile application communicates with the ESP32 nodes

using the `flutter_bluetooth_serial` package, enabling the user to update the SSID and password directly from the mobile device. Once the new credentials are received, the ESP32 stores them locally and reconnects to the specified Wi-Fi network automatically.

This approach allows each circuit to be configured and maintained independently, enhances usability for non-technical users, and ensures adaptability to changes in the home network environment without interrupting the operation of other system components.

6.1 Door Lock Product (DLP) Implementation

The Door Lock Product (DLP) node is responsible for securing home entry through a multi-factor authentication (MFA) flow. This circuit integrates a local user-interaction interface (keypad + LCD) with an electronic locking mechanism (relay-controlled lock), while delegating AI-based identity verification tasks to the local AI gateway (Raspberry Pi). The DLP implementation was designed to balance usability and security by combining biometric verification (face/fingerprint) with a knowledge factor (PIN/password), and by adding an anti-spoofing layer to mitigate presentation attacks (e.g., printed photos or screen replay).

6.1.1 AI Models Used in DLP

The DLP subsystem relies on three main AI components that operate sequentially as part of the authentication pipeline:

- 1. Face Detection:**

This stage locates the face region within the captured frame before any further analysis. Accurate detection is critical because recognition and anti-spoofing performance depend strongly on the quality of the extracted face crop.

- 2. Face Recognition:**

After detecting and aligning the face, the system extracts a feature representation (embedding) and compares it against enrolled identities. This enables identity verification for authorized household members.

- 3. Anti-Spoofing (Presentation Attack Detection):**

To reduce the risk of unlocking the door using photos or replayed videos, the

system applies an anti-spoofing step that classifies the detected face as either real or spoof before accepting recognition results.

This modular decomposition (detect → recognize → spoof-check) follows the common structure of modern face recognition pipelines, and allows each component to be independently improved or replaced if needed.

6.1.2 Initial Dataset-Based Experiments and Limitations

Early experiments evaluated several publicly available **face anti-spoofing datasets** hosted on Roboflow. While these datasets allowed rapid prototyping, they exhibited practical limitations for real-world deployment. In many cases, the datasets were relatively small and were collected with limited diversity (e.g., the same subject and similar background/lighting conditions). Such characteristics can produce seemingly strong training performance while still yielding weak generalization to unseen environments and different users.

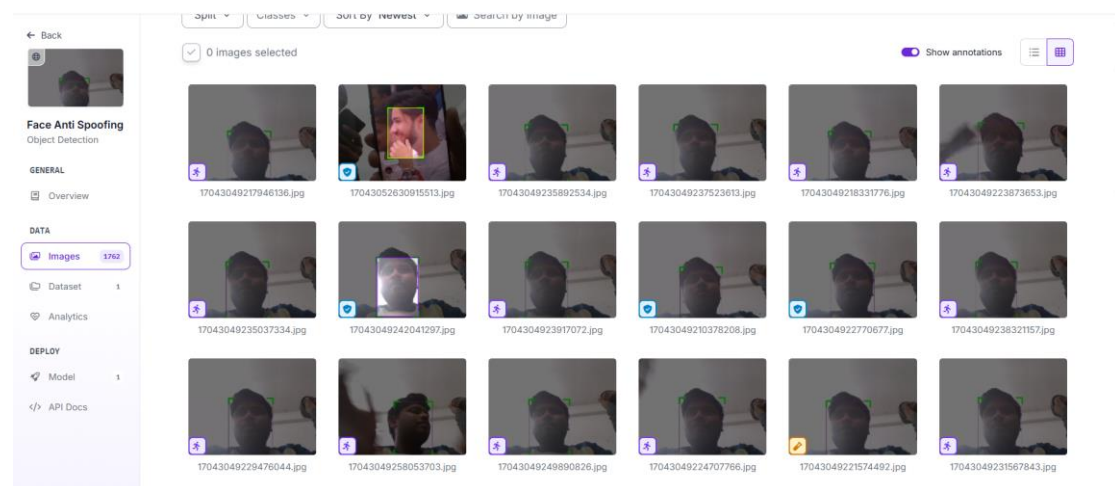


Figure 11. Roboflow dataset sample showing limited diversity

In addition, a subset of these datasets was tested using a YOLOv8-based approach. The training process showed fast convergence and high validation performance in a small number of epochs, indicating strong fitting to the dataset distribution. However, this outcome is not sufficient when the dataset does not represent real deployment variability (multiple identities, multiple lighting conditions, different cameras, and multiple spoof media types).

```

print("PT VAL mAP50-95:", val_metrics.results_dict["metrics/mAP50-95(B)"])
print("PT TEST mAP50-95:", test_metrics.results_dict["metrics/mAP50-95(B)"])
print("PT VAL mAP50 :", val_metrics.results_dict["metrics/mAP50(B)"])
print("PT TEST mAP50 :", test_metrics.results_dict["metrics/mAP50(B)"])

```

```

... Ultralytics 8.3.239 Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla T4, 15095MiB)
Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 1607.1±417.3 MB/s, size: 69.2 KB)
val: Scanning /content/valid/labels.cache... 135 images, 2 backgrounds, 0 corrupt: 100% 135/135 254.5Kit/s 0.0s

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	135	150	0.892	0.882	0.904	0.612
fake	66	77	0.846	0.93	0.887	0.667
real	71	73	0.938	0.834	0.921	0.556

```

Speed: 7.3ms preprocess, 5.4ms inference, 0.0ms loss, 2.6ms postprocess per image
Results saved to /content/evals_pt/on_val
Ultralytics 8.3.239 Python-3.12.12 torch-2.9.0+cu126 CUDA:0 (Tesla T4, 15095MiB)
val: Fast image access (ping: 0.0±0.0 ms, read: 2048.0±584.6 MB/s, size: 68.9 KB)
val: Scanning /content/test/labels.cache... 98 images, 1 backgrounds, 0 corrupt: 100% 98/98 186.4Kit/s 0.0s

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	98	169	0.966	0.883	0.946	0.749
fake	33	33	0.993	0.97	0.993	0.874
real	65	136	0.939	0.796	0.9	0.624

```

Speed: 4.5ms preprocess, 4.9ms inference, 0.0ms loss, 3.0ms postprocess per image
Results saved to /content/evals_pt/on_test
PT VAL mAP50-95: 0.6116551818240883
PT TEST mAP50-95: 0.7491105828490754
PT VAL mAP50 : 0.9042219945056935
PT TEST mAP50 : 0.9464563574042032

```

Figure 12. YOLOv8 validation/testing results (20 epochs) showing possible fit

This highlighted an important practical concern: high accuracy on a constrained dataset does not necessarily imply robust performance against spoofing attempts in realistic home environments.

6.1.3 DeepFace Adoption as a Practical Implementation Solution

To address the generalization and deployment challenges, the implementation transitioned to using DeepFace, an open-source library that is widely used in research and practical applications. DeepFace provides an end-to-end face analysis pipeline and supports multiple face detection and recognition backends through a unified API, which simplifies integration and experimentation. GitHub

A key advantage of DeepFace is its support for multiple recognition models (e.g., VGG-Face, FaceNet, ArcFace, and others) and multiple detectors (e.g., OpenCV, MTCNN, RetinaFace, MediaPipe, YuNet, etc.), enabling the DLP subsystem to select configurations suitable for local inference constraints and reliability requirements.

6.1.4 Source-Code Adaptations for Real-Time Decisions and Returned Outputs

During integration, two practical issues were observed in the library's real-time streaming workflow:

1. **Single-frame decision sensitivity:**

Making a spoofing decision based on a single frame can lead to unstable behavior due to noise, blur, motion, or sudden lighting changes.

2. **Limited access to structured outputs in streaming:**

For system integration, it is essential that the pipeline returns structured results (e.g., `is_real` flag, recognition identity, confidence indicators) so that the firmware/backend can decide whether to proceed with unlocking and how to log and notify events.

To improve robustness and integration, the implementation was adapted so that the decision is based on **multiple consecutive frames** (five frames) rather than a single frame, and the stream pipeline was modified to provide explicit returnable outputs suitable for system-level decision making. Notably, DeepFace’s own real-time pipeline describes processing logic that depends on focusing on a face across consecutive frames, which aligns well with this stabilization strategy.

6.1.5 Anti-Spoofing Decision Mechanism in DeepFace

When anti-spoofing is enabled, DeepFace performs a presentation attack detection step that classifies whether the captured face is **real or fake/spoof**. At a high level, face anti-spoofing systems commonly rely on learned visual cues that distinguish genuine faces from presentation attacks, such as texture artifacts, reflectance inconsistencies, moiré patterns from screens, and other imaging distortions that frequently appear in printed or replayed media.

In DeepFace, this capability is exposed through an `anti_spoofing` flag that augments face extraction and returns an `is_real` indicator for each detected face. This output is then used as a gating condition before accepting recognition as valid for authentication.

6.1.6 Selected Models and Backends in This Implementation

Based on accuracy–latency trade-offs and deployment constraints, the following configurations were adopted for the DLP subsystem:

- Face Detection Backend: **OpenCV** (fast and lightweight for real-time usage)
GitHub

- Face Recognition Model: **VGG-Face** (supported in DeepFace and widely used as a stable baseline)
- Anti-Spoofing: the **built-in anti-spoofing module** provided by DeepFace (enabled via `anti_spoofing`)

6.1.7 Hardware Connections of the DLP Circuit

Figure (DLP circuit diagram) below shows the hardware wiring of the **Door Lock Product (DLP)** node, where an **ESP32** acts as the main controller for user interaction (LCD and keypad), local feedback (buzzer), and door-lock actuation through a **relay module**. The relay is used to switch the **12 V electric lock** while maintaining electrical isolation between the ESP32 low-voltage control domain (**3.3 V logic**) and the higher-voltage actuator supply.

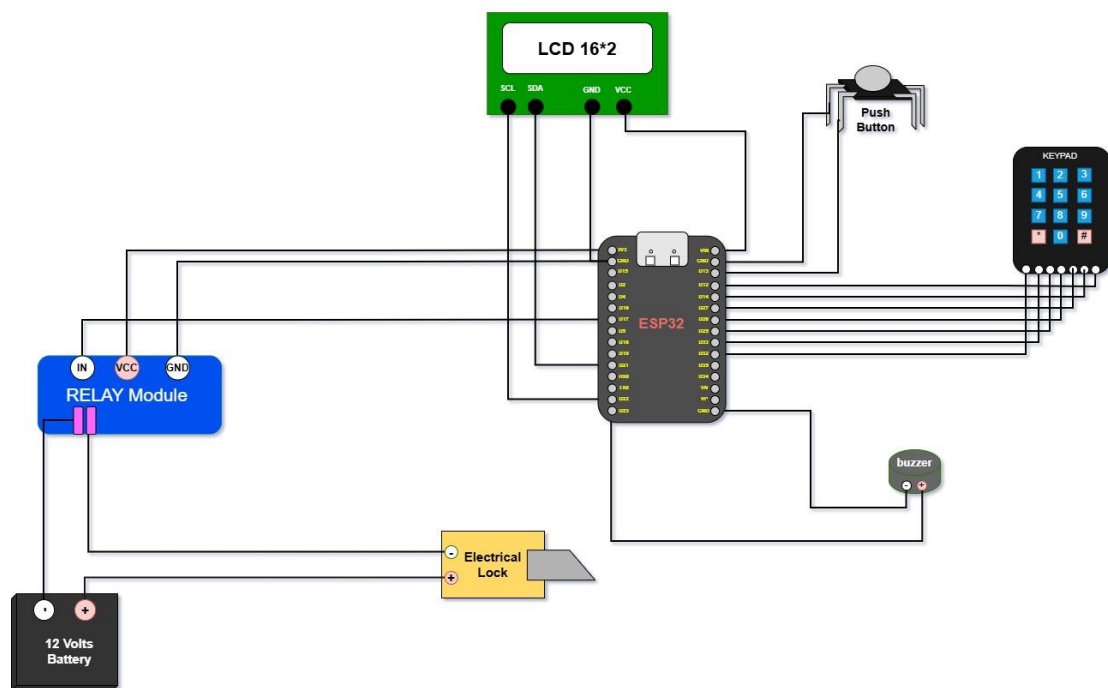


Figure 13. DLP circuit diagram

A **16×2 I2C LCD** is connected via the I2C bus to minimize GPIO usage. The bus lines are wired as **SCL → D22** and **SDA → D21**, with **GND → GND**. The LCD is powered from the ESP32 board's **5 V** rail by connecting **VCC → VIN (5 V)**. The **4×3 matrix keypad** is interfaced using seven ESP32 digital pins to support

software-based matrix scanning. The keypad lines are connected to **D32, D33, D25, D26, D27, D14, and D12**, enabling the firmware to detect pressed keys by sequentially driving and sampling the row/column lines.

The **start push button** is wired as a digital trigger between **D13 and GND**, allowing the user to explicitly initiate the authentication workflow. In addition, an **active buzzer** is connected for audible feedback, with **D23** used as the drive pin and the other terminal connected to **GND**, enabling distinct notification patterns (e.g., success/failure/status cues).

Finally, the **relay module** is connected on the control side as **IN → D17, VCC → 3.3 V**, and **GND → GND**. On the power side, the **12 V battery** supply is routed through the relay's contact terminals to control the electric lock, allowing the ESP32 to safely enable or interrupt the lock's power without directly exposing the microcontroller to the 12 V load.

6.1.8 DLP Workflow Algorithm

The DLP node implements a two-step authentication workflow that combines **face recognition** (performed on the Raspberry Pi camera pipeline) with a **keypad-based password** (handled by the ESP32). When the user arrives at the door, the LCD displays an idle prompt instructing the user to press the start button. Once the button is pressed, the user is required to present their face to the camera connected to the Raspberry Pi. If the face is successfully recognized, the LCD displays **“Known Face”**, a notification is sent to the mobile application, and the access attempt is logged so it can be later reviewed through the dashboard. Immediately after that, the LCD prompts the user that they have **10 seconds** to press the **“*”** key on the keypad to continue to the password stage.

If the user presses **“*”** within the 10-second time window, the system requests a password entry. The user enters a **6-digit** password and then confirms by pressing **“*”** again. If the entered password is valid, the ESP32 activates the relay to unlock the door for **3 seconds**, displays **“Door Unlocked”** on the LCD, and sends a second notification indicating that the door has been unlocked. After the 3 seconds elapse, the relay is deactivated and the door returns to the locked state. Conversely, if the password is

incorrect, the LCD displays an error message indicating an invalid password and a notification is also sent to the mobile application to report the failed password attempt.

If the user does not press “*” within the **10-second timeout**, the password stage is not enabled, and the user must restart the process by performing face authentication again (i.e., repeating the face verification step).

In addition to the standard unlock procedure, the system supports a secure password update mechanism. To change the password, the user must first pass face recognition successfully, then press “#” within the same **10-second** window. The system then prompts for the **current password** followed by the **new password**, where the new password must also be exactly **6 digits** to be accepted and stored.

6.2 Door Guard Product (DGP) Implementation (Timed Suspicious-Activity Detection)

6.2.1 Overview and Timed Activation Logic

The **Door Guard Product (DGP)** is responsible for enhancing entrance security by enabling *night security mode* after a configurable time. In this mode, the system focuses on detecting suspicious activity near the home entrance while reducing false alarms through **sensor-based gating**. Once a valid motion event is confirmed, the DGP triggers local AI inference (on the Raspberry Pi) to perform **masked-person detection** and **weapon detection**, then reports results to the user via real-time notifications.

6.2.2 Sensor-Gated “Suspicious Movement” Confirmation

To avoid triggering AI analysis from noisy or non-threatening events, DGP uses a confirmation strategy based on combining multiple sensing principles near the door area (e.g., motion and proximity), which improves reliability compared to relying on a single sensor. This design is intended to reduce false triggers while still capturing genuine movement events during the late-hours monitoring window.

6.2.3 Masked Person Detection Model Training and Evaluation

For masked-person detection, the implementation trained **YOLOv8s** using multiple dataset versions and training configurations. The following experiments were conducted:

- **Experiment 1 (YOLOv8s on AuraMask.v1):** Dataset size **5836 images** (5188 training, **648** validation). Training settings: **50 epochs, batch 16, imgsz 416**. Reported losses: train box **0.59308**, val box **0.73147**; train cls **0.2696**, val cls **0.29839**; train dfl **0.87318**, val dfl **0.93994**. Metrics: precision **0.99066**, recall **0.97798**, mAP50 **0.99291**, mAP50–95 **0.83661**.

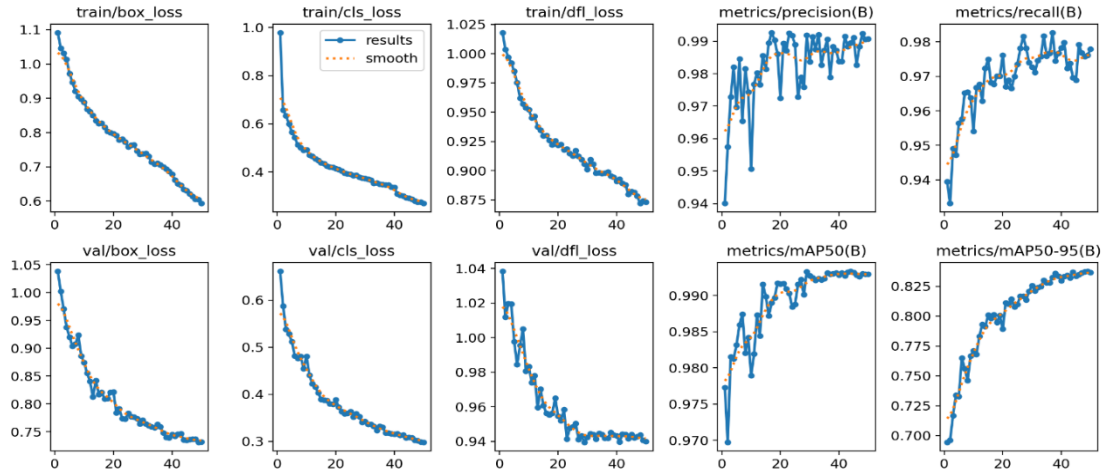


Figure 14. Masked Detection, Experiment 1

- **Experiment 2 (YOLOv8s on AuraMask.v6):** Dataset size **6591 images** (5106 training, **1485** validation). Training settings: **150 epochs, batch 4, imgsz 512, patience 20**, with **auto-stop at epoch 97**. Reported losses: train box **0.58727**, val box **0.68661**; train cls **0.26226**, val cls **0.68661**; train dfl **0.88632**, val dfl **0.97526**. Metrics: precision **0.99172**, recall **0.98952**, mAP50 **0.99383**, mAP50–95 **0.85738**.

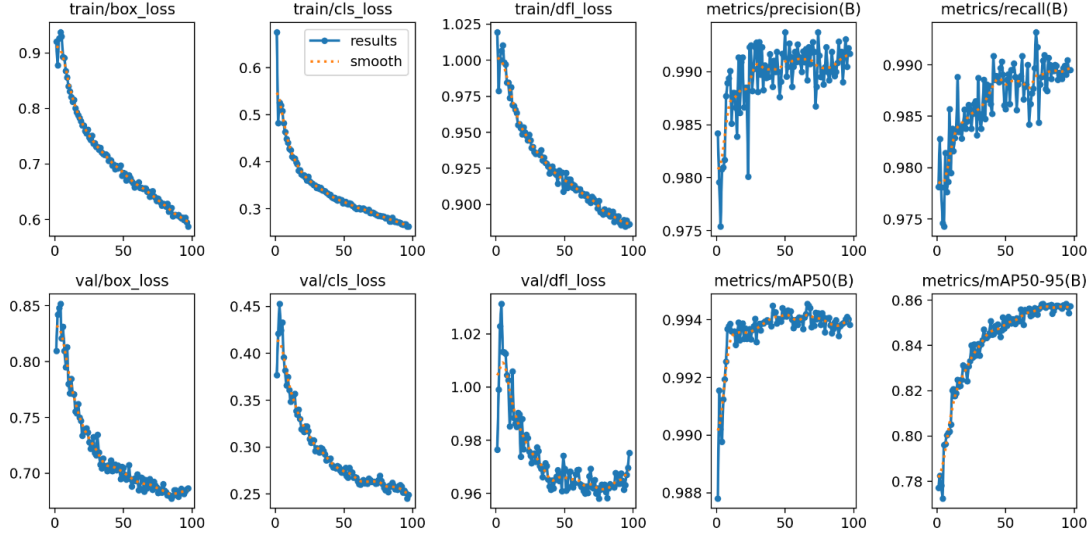


Figure 15. Masked Detection, Experiment 2

- **Experiment 3 (YOLOv8s on AuraMask.v7):** Dataset size **7607 images (6122 training, 1485 validation)**. Training settings: **150 epochs, batch 4, imgsiz 640, patience 20, with auto-stop at epoch 123**. Reported losses: train box **0.55477**, val box **0.67439**; train cls **0.25707**, val cls **0.24741**; train dfl **0.87195**, val dfl **0.92161**. Metrics: precision **0.98908**, recall **0.99241**, mAP50 **0.99279**, mAP50–95 **0.85852**.

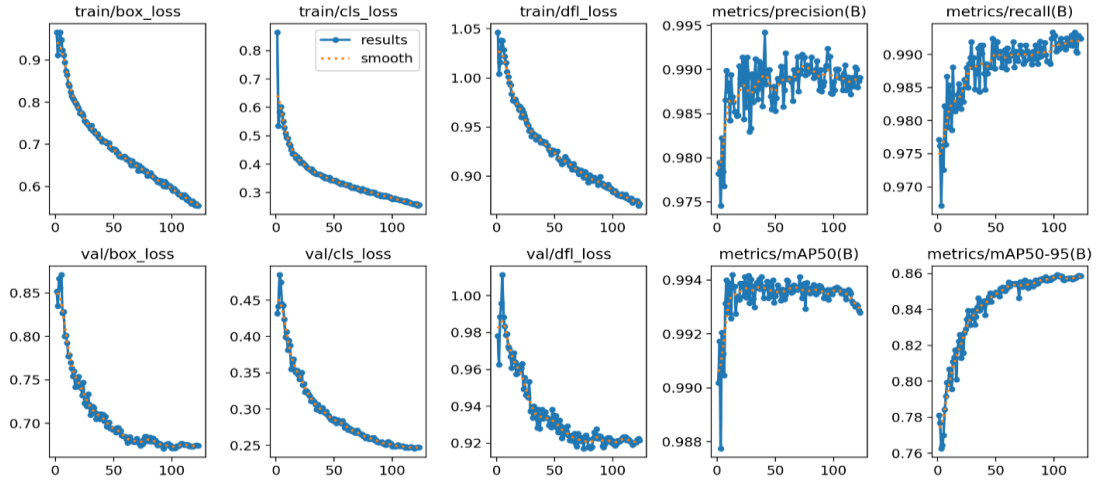


Figure 16. Masked Detection, Experiment 3

Based on the reported results, the strongest masked-person configuration (by **mAP50–95**) is **Experiment 3 (0.85852)**, closely followed by Experiment 2 (0.85738).

6.2.4 Weapon (Gun) Detection Model Training and Evaluation

For weapon detection, the implementation also trained **YOLOv8s** across multiple datasets and configurations. The experiments conducted are summarized as follows:

- **Experiment 1 (YOLOv8s on Youtube-GDD-relabelled.v1i)**: Dataset size **3012 images** (2101 training, 609 validations, 302 testing). Training settings: **90 epochs, batch 16, imgsz 416**. Reported losses: train box **0.61814**, val box **1.32438**; train cls **0.38055**, val cls **0.98927**; train dfl **0.9866**, val dfl **1.43473**. Metrics: precision **0.82818**, recall **0.69347**, mAP50 **0.75832**, mAP50-95 **0.51735**.

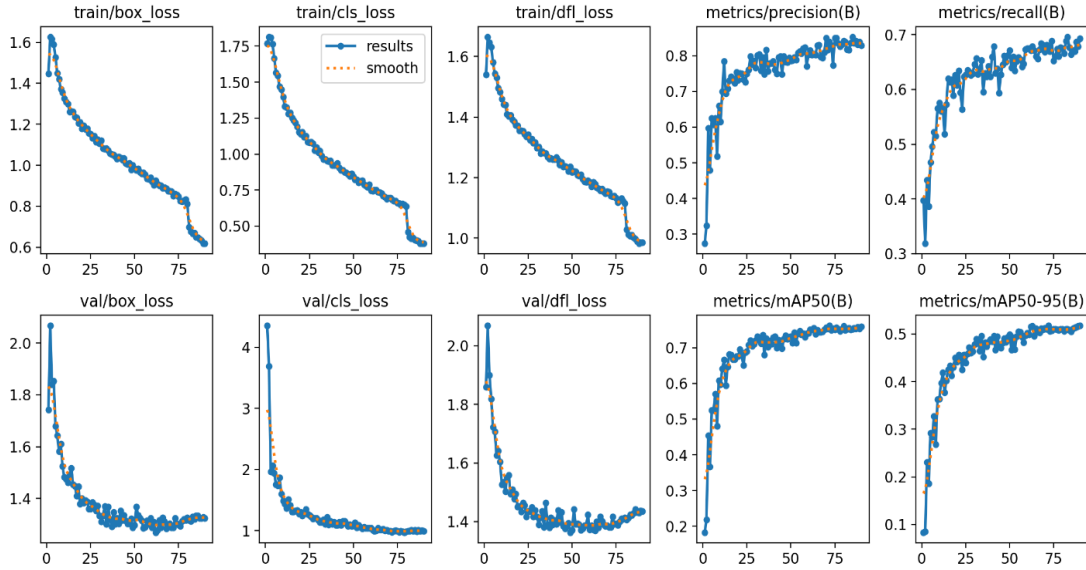


Figure 17. Gun Detection, Experiment 1

- **Experiment 2 (YOLOv8s on WeaponDetection.v5-weaponnet18k)**: Dataset size **17324 images** (13282 training, 4022 validation, 20 testing). Training settings: **100 epochs, batch 16, imgsz 416**. Reported losses: train box **0.56951**, val box **0.75839**; train cls **0.30806**, val cls **0.47798**; train dfl **0.9256**, val dfl **1.01494**. Metrics: precision **0.9142**, recall **0.8794**, mAP50 **0.93**, mAP50-95 **0.75881**.

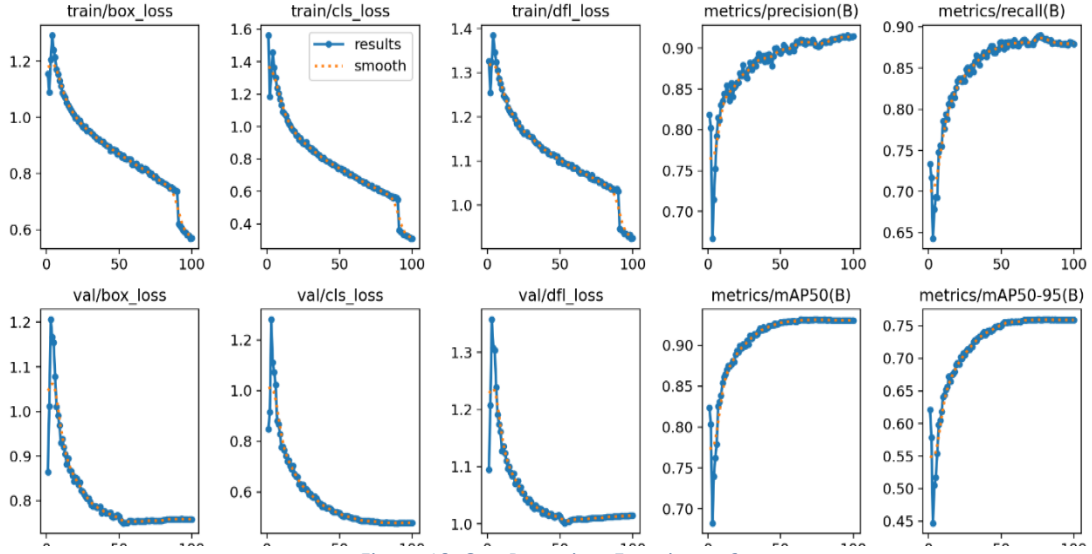


Figure 18. Gun Detection, Experiment 2

- **Experiment 3 (YOLOv8s on WeaponNet18K.AuraWeapons.Original.Grayscale.Exposure):** Dataset size **31690 images** (24496 training, 7174 validation, 20 testing). Training settings: **200 epochs, batch 2, imgsz 1280, patience 20, with auto-stop at epoch 134**. Reported losses: train box **0.89574**, val box **1.27072**; train cls **0.73799**, val cls **0.93514**; train dfl **1.23151**, val dfl **1.22801**. Metrics: precision

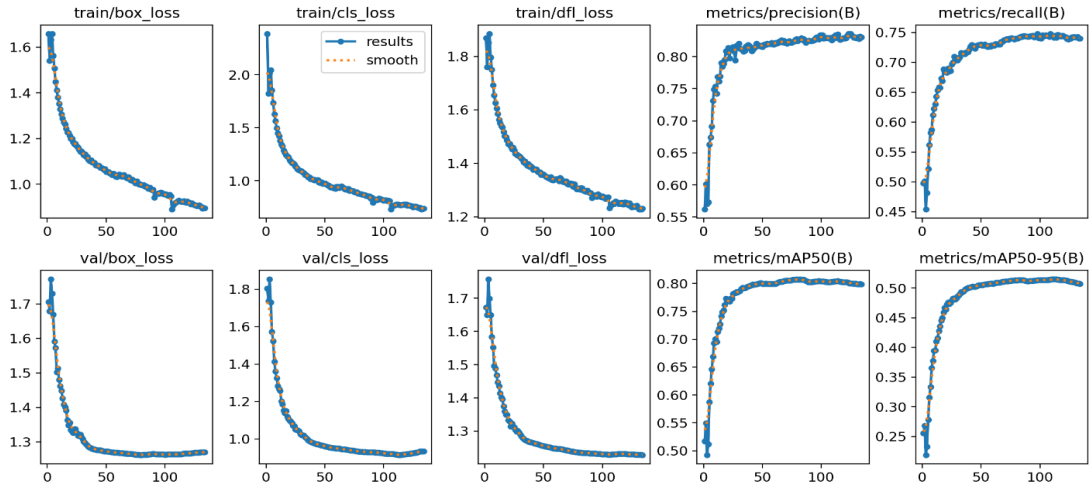


Figure 19. Gun Detection, Experiment 3

0.83026, recall 0.74013, mAP50 0.79842, mAP50–95 0.50754.

From the reported metrics, the best-performing weapon model (by **mAP50–95**) is **Experiment 2 (0.75881)**.

6.2.5 DGP's AI Models Local Inference

In operational mode, DGP activates at the configured night-time window and waits for a sensor-confirmed motion event. Once motion is confirmed, the system triggers **local inference** on the Raspberry Pi using the trained YOLOv8 models for **masked-person** and **weapon** detection. Detection outputs (e.g., bounding boxes/labels/confidence) are then used to generate real-time alerts to the user and to record the event for later review in the application. This workflow aligns with the system's privacy-first approach, as AI processing is designed to remain local rather than relying on cloud inference.

6.2.6 Hardware Connections of the DGP Circuit

The **Door Guard Product (DGP)** hardware is designed to support two main functions: (1) *automatic night lighting* based on ambient light level, and (2) *visual deterrence* through a flashing light when suspicious movement is detected. Since both loads are **12 V LEDs**, each LED is driven through a dedicated **relay module** in order to electrically isolate the ESP32 low-voltage control signals (**3.3 V logic**) from the 12 V power domain supplied by an external DC battery.

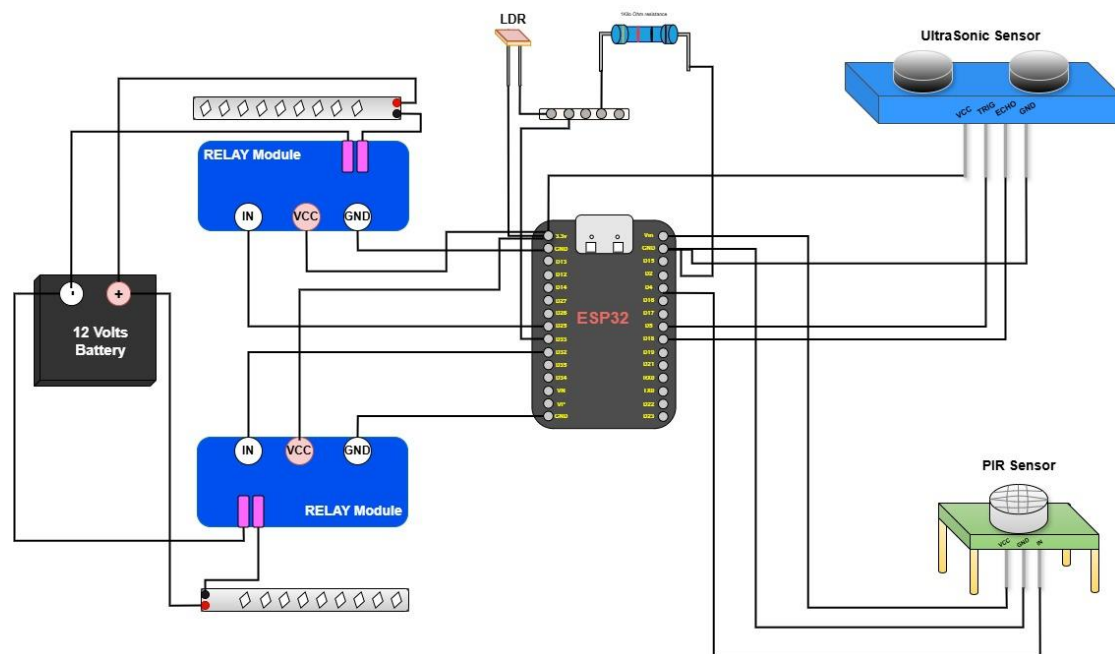


Figure 20. DGP Circuit Diagram

The first **12 V LED** is dedicated to night illumination and is activated automatically when the surrounding light intensity drops (i.e., when it becomes dark). This behavior

is achieved using an **LDR (Light Dependent Resistor)** configured as a **voltage divider** with a **1 k Ω resistor**, where the divided analog signal is read by the ESP32 on **D33**. Based on the measured level, the firmware determines whether the illumination threshold has been crossed and, if so, switches the corresponding relay to power the LED. The relay module controlling this LED is connected on the control side as **IN $\rightarrow$ D25**, with **VCC $\rightarrow$ 3.3 V** and **GND $\rightarrow$ GND**.

The second **12 V LED** is used as an active deterrent and is configured to generate a *flash* whenever the system confirms suspicious activity. Its corresponding relay module is wired as **IN $\rightarrow$ D32**, while the remaining relay control pins are supplied by **3.3 V** and **GND**. Upon a positive suspicious-event decision, the firmware toggles this relay to create a blinking pattern, thereby providing an immediate visible warning intended to discourage potential intruders.

For motion sensing, a **PIR sensor** is integrated as the primary trigger to detect human movement. The PIR output is connected to the ESP32 digital input **D4**, while the sensor is powered using **VCC = 5 V** and **GND**. To enhance confirmation and reduce false positives, an **ultrasonic sensor** is also used, with **TRIG $\rightarrow$ D5** and **ECHO $\rightarrow$ D18**, in addition to the required supply and ground connections. The combined use of PIR and ultrasonic sensing enables the DGP to validate motion events more reliably before initiating deterrence actions and subsequent system responses.

Finally, the overall actuation stage (both LEDs) is powered using an external **12 V DC battery**, which provides the required current and voltage for the 12 V lighting loads, while the ESP32 and sensing circuitry remain in the low-voltage domain to ensure safe operation and proper electrical separation.

6.2.7 DGP Workflow and Algorithm

The DGP node operates using two coordinated workflows: **automatic night lighting** based on ambient illumination, and **time-controlled suspicious-movement monitoring** followed by AI-based classification. First, the system continuously samples the **LDR** reading and compares it against a predefined **illumination threshold**. When the measured light intensity falls below this threshold (i.e., low-light/night

conditions), the controller activates the first 12 V LED through its relay, providing automatic entrance lighting without requiring user intervention.

In parallel, the suspicious-activity detection module is enabled according to a **user-defined start time** configured through the mobile application (e.g., the user may set the monitoring to start at **11:00 PM**). Before this configured time, the system remains in a standby state and does not evaluate suspicious-movement events. Once the start time is reached, DGP begins monitoring the surrounding area using two complementary sensing mechanisms to reduce false positives.

To mitigate erroneous triggering, DGP does not escalate the event based on a single detection. Instead, it accumulates “movement counts” using two definitions of movement. The first movement type is derived from the **ultrasonic sensor**, where a movement is registered when the measured distance becomes **less than 150 cm**. The ultrasonic sensor is assumed to be mounted at a sufficiently high position to reduce the likelihood of detecting small animals (e.g., cats). The second movement type is produced by the **PIR sensor**, where a movement is registered when the PIR output indicates motion. A suspicious scenario is confirmed only when the system observes **10 movement events** (configurable) within a **30-second window**. This rule-based gating stage significantly reduces false alarms by requiring repeated and consistent activity in a short period before invoking AI inference.

Once the suspicious-activity condition is satisfied, the system triggers the **AI models** to analyze the scene and classify the event. The output is mapped into one of four possible states: **Gun**, **Gun_Masked**, **Masked**, or **Movement_Only**. After detection, DGP immediately sends a notification to the mobile application indicating the detected state, and the event is stored persistently so that the user can review the complete history through the dashboard (including the detection outcome and time of occurrence).

6.3 Gas and Temperature Product (GTP) Implementation

6.3.1 Overview of GTP

The **Gas and Temperature Product (GTP)** is implemented as a lightweight embedded node intended for continuous environmental monitoring. Unlike other system

components, GTP does not rely on AI inference; instead, it uses direct sensor thresholding to detect hazardous conditions. The node monitors **ambient temperature** and **gas concentration** and provides immediate local feedback using two status LEDs and a buzzer, in addition to generating mobile notifications when abnormal conditions are detected. The design follows a clear fail-alert principle: normal conditions are indicated through a green status LED, while critical conditions activate a red LED, an audible buzzer, and a notification to the user.

6.3.2 Hardware Connections

GTP is built around an ESP32 and integrates one gas sensor module, one temperature sensor, two 3.3 V LEDs, and a buzzer. The **gas sensor** is connected using its analog output line, where **A0** → **D34** (ADC input). The digital output pin (**D0**) is not used in this implementation. The module is powered from the ESP32 supply rail by wiring **VCC** → **3.3 V** and **GND** → **GND**.

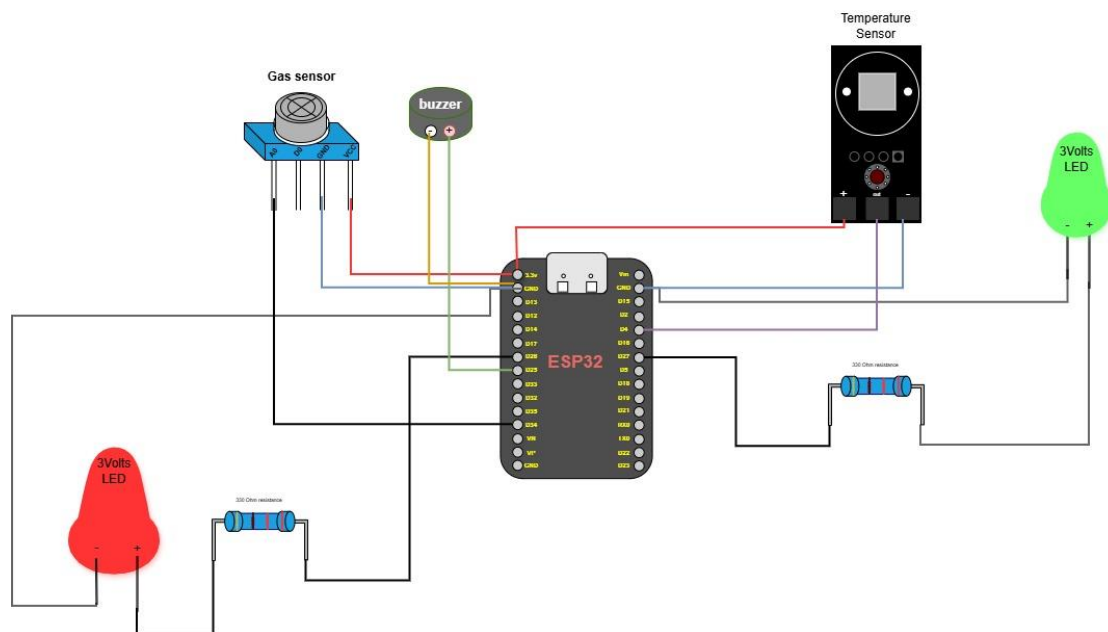


Figure 21. GTP Circuit Diagram

The **temperature sensor** is connected through its signal line as **OUT** → **D4**, with its supply pins wired as **VCC** → **3.3 V** and **GND** → **GND**. For local visual indication, the **red LED** (fault state) is driven from **D26** through a **330 Ω series resistor**, while the **green LED** (normal state) is driven from **D27** through a current-limiting resistor.

Finally, an audible alarm is provided using a buzzer connected as **D25** → **buzzer (+)** and the other buzzer terminal to **GND**.

6.3.3 Workflow and Decision Logic

During operation, GTP continuously samples both sensors and compares their readings against predefined safety thresholds. Under normal conditions, the **green LED remains ON**, while the red LED and buzzer remain OFF. If the measured temperature rises above **50°C**, the system transitions to an alarm state: it turns ON the **red LED**, activates the **buzzer**, and sends a notification to the mobile application to inform the user of the overheating condition. To avoid rapid toggling due to minor fluctuations around the boundary, the system applies hysteresis: the temperature alarm remains active until the temperature drops below **45°C**, at which point the system clears the alarm and returns to the normal state.

Similarly, gas concentration is monitored using threshold-based triggering with hysteresis. When the gas reading exceeds **1000** (sensor units as defined by the module's analog scale), the system raises a gas alarm by enabling the **red LED**, activating the **buzzer**, and sending a notification to the mobile application. The alarm condition persists until the reading falls below **900**, after which the gas alarm is cleared and the system resumes normal indication. This dual-threshold approach improves stability and reduces false alarms caused by transient spikes.

6.4 Trigger Emergency Product (TEP) Implementation

6.4.1 Overview

The **Trigger Emergency Product (TEP)** is an audio-based node that enables emergency triggering through a predefined wake word ("**AuraHome**"). The node continuously listens to the environment, and once the wake word is detected it initiates the recording/analysis workflow and reports outcomes to the system through MQTT, enabling real-time monitoring and alerting in the mobile application.

6.4.2 Hardware and I2S Microphone Interface

TEP is built around an **ESP32-D0WD-V3** and an **INMP441 I2S digital microphone**, with a **buzzer** used for immediate user feedback after a successful wake-word trigger. Power is provided through **USB 5V or a LiPo battery**.

The I2S interface is configured with the following GPIO mapping: **WS (LRCL) → GPIO25**, **SD (DOUT) → GPIO33**, and **SCK (BCLK) → GPIO32**. The buzzer is connected to **GPIO2**. The microphone wiring includes **VDD → 3.3V**, **GND → GND**, and **L/R → GND (Left channel)**.

6.4.3 Wake-Word Detection

Wake-word detection is implemented using a lightweight custom algorithm suitable for embedded execution. The pipeline combines **Zero Crossing Rate (ZCR)**, **energy calculation**, and **correlation-based pattern matching** against the expected “AuraHome” template, followed by a **normalized correlation coefficient** decision.

The implementation uses fixed parameters and thresholds, including: **WAKE_WORD_SAMPLE_SIZE = 8000** (≈ 0.5 s at 16 kHz), **VOICE_THRESHOLD = 500000**, **WAKE_WORD_THRESHOLD = 0.80**, **MIN_VOICE_ENERGY = 300000**, and **MIN_ZCR_THRESHOLD = 30**. Once the constraints are met, the wake word is confirmed, and the buzzer provides immediate confirmation to the user.

Reported limitations include reduced robustness under background noise above **60 dB**, sensitivity to accent variation, and potential false triggers from acoustic similar words.

6.4.4 MQTT Integration

TEP is integrated with the system via **MQTT** using configurable broker parameters (server/port/credentials and **DEVICE_TOKEN**) and a topic structure supporting **Subscribe (control)** and **Publish (notification)**.

This enables (1) **remote triggering** of recording from the mobile app without speaking “AuraHome”, (2) **real-time status monitoring** by receiving emotion results, and (3) **emergency alerting** based on specific emotions such as fear or anger.

6.4.5 SER Datasets and Preprocessing

Speech Emotion Recognition (SER) training relies on a combined dataset of acted English speech from **RAVDESS**, **CREMA-D**, **TESS**, and **SAVEE**, totaling **16,402 samples** and consolidated into **7 emotion classes** with **121 actors**.

The preprocessing pipeline consists of **audio loading**, **MFCC extraction**, **feature aggregation**, and **normalization**. Optional augmentation techniques were explored, including time stretching ($\pm 10\%$), pitch shifting (± 2 semitones), additive noise (SNR 30–40 dB), and time shifting (± 0.5 s).

Table 3. SER datasets summary

Dataset	Samples	Emotions	Language	Actors	Type
RAVDESS	1,440	8	English	24	Acted
CREMA-D	7,442	6	English	91	Acted
TESS	2,800	7	English	2	Acted
SAVEE	480	7	English	4	Acted
TOTAL	16,402	7	English	121	Acted

6.4.6 SER Models and Experiments

To benchmark Speech Emotion Recognition (SER) performance under different architectural choices, multiple experiments were conducted with varying model variants and training durations, including: **Original Baseline (80 epochs)**, **Ensemble of four models (85 epochs)**, **184-feature Simple CNN (80 epochs)**, **MFCC-Improved model (71 epochs)**, **Strong Regularization model (100 epochs)**, **No Regularization model (96 epochs)**, and a **Balanced model (96 epochs)**. Based on these trials, the selected configuration for this implementation phase is the **Balanced CNN**, which consists of **four Conv1D blocks followed by two dense layers**, using convolution filters **256 $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ 64**. Regularization is applied through **dropout = 0.3** in convolutional blocks and **0.4** in dense layers, along with **L2 regularization = 0.0001**. The model is trained with a **learning rate of 0.0001** and **batch size of 64**, while overfitting is controlled using **early stopping** (patience = **10** epochs, monitoring validation loss) and **ReduceLROnPlateau** scheduling (factor = **0.5**, patience = **5** epochs).

The learning behavior of the Balanced CNN is illustrated in **Figure 22**, which shows a steady improvement in both training and validation accuracy as training progresses, accompanied by a consistent decrease in training and validation loss until convergence. The best observed **validation accuracy** reached **74.17%**, and the model achieved a **training accuracy of 77.44%**. In addition, a preliminary evaluation on the internal held-out split used during model development reported an accuracy of **72.81%**. This value is included here as an internal reference for the trained model’s generalization within the same data distribution; however, the **final testing stage** of the project will evaluate the system on a **different dataset** and is therefore reported separately in the dedicated testing chapter.

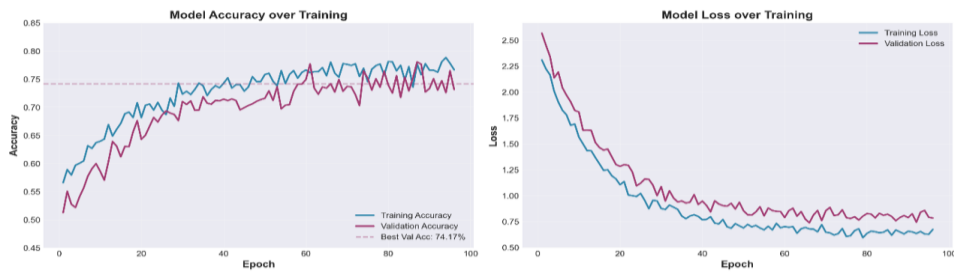


Figure 22. Balanced CNN Accuracy and Loss over Training

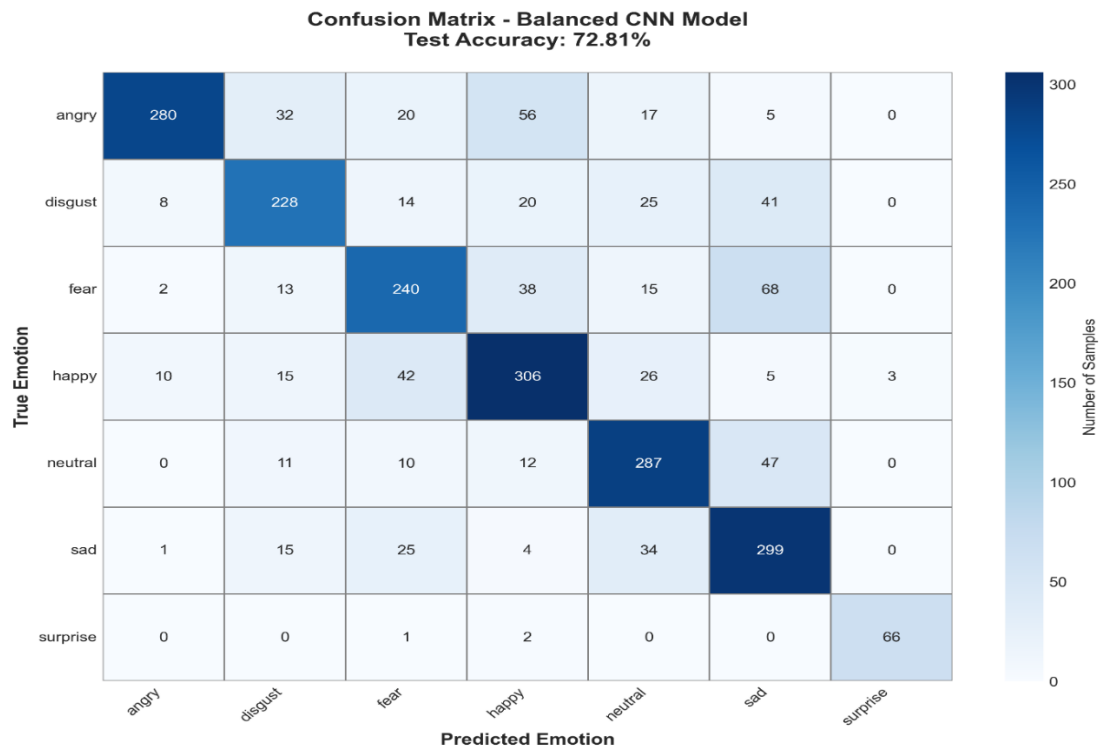


Figure 23. Balanced CNN Confusion Matrix

6.4.7 End-to-End Workflow

Operationally, TEP runs in an always-listening mode where the ESP32 captures audio via the INMP441 and applies the wake-word detection logic. Upon detecting “AuraHome”, the node confirms the trigger using the buzzer and proceeds to the recording/processing stage. In addition to voice triggering, the same recording stage can be initiated remotely through MQTT control messages from the mobile application. After inference, the emotion result is published through MQTT for real-time monitoring and emergency alerting use cases.

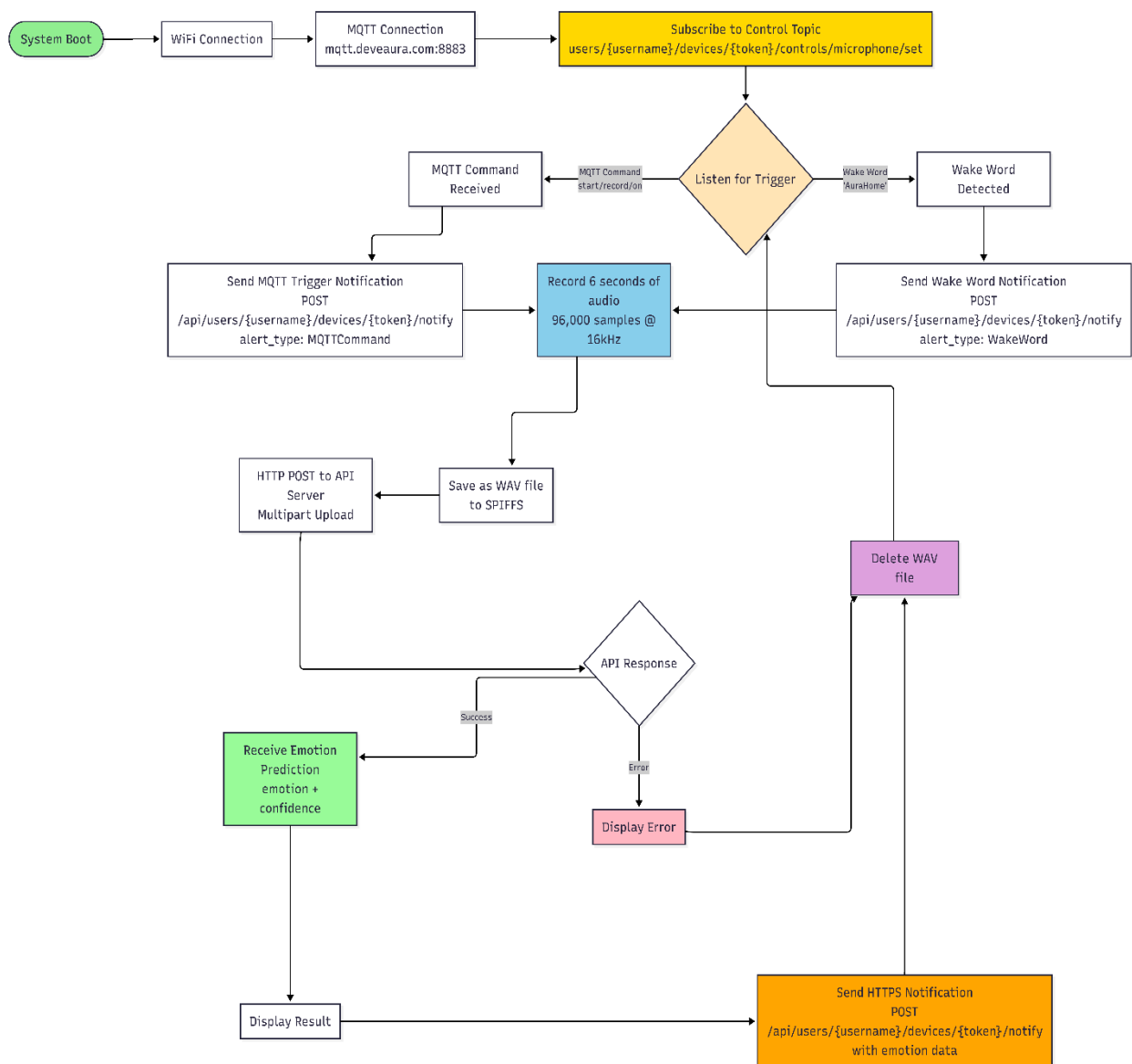


Figure 24. TEP end-to-end workflow showing system boot, Wi-Fi and MQTT initialization

6.5 Backend Development

The backend was developed to serve as the core infrastructure for AuraHome, a sophisticated IoT smart home platform designed to manage device communication, ensure secure data handling, and provide real-time functionality. This implementation establishes a scalable architecture capable of supporting ESP32-based devices, local Raspberry Pi gateways, and mobile applications, while integrating advanced features such as two-factor authentication, MQTT broker management, subscription billing, and real-time push notifications. The primary objective was to create a secure, maintainable, and highly available system that bridges the gap between edge computing on local networks and cloud-based user management.

6.5.1 Technologies Used

The backend leverages a modern and robust technology stack carefully selected to meet the unique requirements of an IoT platform:

- **Laravel 12:** A cutting-edge PHP framework renowned for its elegant syntax, comprehensive feature set, and developer-friendly ecosystem. Laravel was chosen for its built-in support for RESTful API development, robust middleware system, queue management, and seamless integration with modern tools. The framework's Eloquent ORM, validation system, and event broadcasting capabilities streamlined development significantly.
- **Filament 4:** A powerful admin panel framework built on Laravel and Livewire, providing a modern interface for system administrators and technicians to manage users, devices, subscriptions, and system configurations. Filament's resource-based architecture enabled rapid development of CRUD operations with advanced filtering, bulk actions, and custom pages.
- **MySQL:** A reliable and battle-tested relational database system integrated seamlessly with Laravel's Eloquent ORM. MySQL stores all critical application data including user accounts, subscription plans, device configurations, sensor readings, and audit logs. The database schema was designed with proper indexing and relationships to ensure optimal query performance.
- **Laravel Sanctum:** Handles API token-based authentication for mobile applications and third-party integrations. Sanctum provides a lightweight solution for issuing access and refresh tokens, supporting secure authentication flows without the complexity of OAuth2. The implementation includes token expiration, revocation, and ability-based authorization.
- **Spatie Permission:** A flexible role-based access control (RBAC) package that manages user permissions and roles. The system defines three primary roles: Admin, Technician, and Customer, each with specific capabilities. Permission guards ensure that only authorized users can access sensitive operations.
- **php-mqtt/client:** A PHP library for communicating with MQTT brokers, enabling the backend to interact with Mosquitto Dynamic Security plugin. This library handles connection management, subscription handling, and message publishing for device namespace management and user credential provisioning.

- **Firestore Cloud Messaging (FCM):** Facilitates push notifications to mobile devices through Google's FCM service. Supported by the google/apiclient library, FCM enables both targeted notifications to specific users and topic-based broadcasts for device alerts, subscription updates, and system announcements.
- **Supervisor:** A process control system used in production to manage Laravel queue workers and scheduled tasks. Supervisor ensures that background jobs run continuously, automatically restarting workers if they fail, and maintaining system reliability for critical operations like MQTT namespace cleanup and notification delivery.
- **Custom Encryptor Service:** A specialized encryption service separate from Laravel's default encryption, using an independent ENCRYPTION_KEY for encrypting sensitive device data. This dual-key approach ensures that device tokens, MQTT credentials, and sensor metadata remain protected even if the application key is compromised.

6.5.2 Architectural Design

The backend architecture implements the **Service-Repository Pattern**, a sophisticated design approach that promotes separation of concerns, maintainability, and testability. This pattern divides the application logic into three distinct, well-defined layers:

Three-Layer Architecture

1. **Controllers (HTTP Layer)**

Controllers serve as the entry point for all HTTP requests, responsible exclusively for request handling and response formatting. They perform the following duties:

- Receive and validate incoming requests using Laravel's Form Request classes
- Delegate business logic to service layer
- Format responses using the ApiResponse trait for consistency
- Handle HTTP status codes and error responses

Controllers remain thin and focused, containing no business logic or database queries. This separation ensures that HTTP concerns don't bleed into domain logic.

2. **Services (Business Logic Layer)**

Services encapsulate all business logic and orchestrate complex operations involving multiple repositories. Each service manages a specific domain:

- **AuthService:** Handles user authentication, token management, and 2FA workflows
- **SubscriptionService:** Manages subscription lifecycle, approval processes, and billing
- **DeviceService:** Coordinates device registration, configuration, and status updates

- **MosquittoDynamicSecurityService:** Manages MQTT user credentials and ACL permissions
- **TwoFactorAuthService:** Implements authenticator app and email-based 2FA
- **FirebaseService:** Sends push notifications and manages FCM topics

Services are defined through interfaces (App\Contracts\Services*) ensuring loose coupling and enabling easy substitution for testing or alternative implementations. They handle transactions, error recovery, and cross-cutting concerns like logging and event dispatching.

3. Repositories (Data Access Layer)

Repositories provide an abstraction layer between business logic and data persistence. Each repository corresponds to a specific model and handles:

- CRUD operations with optimized queries
- Complex filtering and searching
- Relationship eager loading to prevent N+1 queries
- Query result caching where appropriate

Like services, repositories implement interfaces (App\Contracts\Repositories*) defined in the contracts namespace. This abstraction allows the data source to change (e.g., switching from MySQL to PostgreSQL) without impacting higher layers.

Dependency Injection and Interface Binding

All service and repository interfaces are bound to their concrete implementations in RepositoryServiceProvider.php using Laravel's service container. The binding uses the singleton() method to ensure only one instance exists per request:

```
// AuthService - Handles authentication business logic
$this->app->singleton(
    \App\Contracts\Services\AuthServiceInterface::class,
    \App\Services\AuthService::class
);
```

Figure 25. Singleton Binding

This approach provides several critical benefits:

- **Testability:** Interfaces can be mocked in unit tests

- **Flexibility:** Implementations can be swapped without changing dependent code
- **SOLID Principles:** Adheres to Dependency Inversion Principle
- **Type Safety:** Controllers and services type-hint interfaces, ensuring contracts are enforced

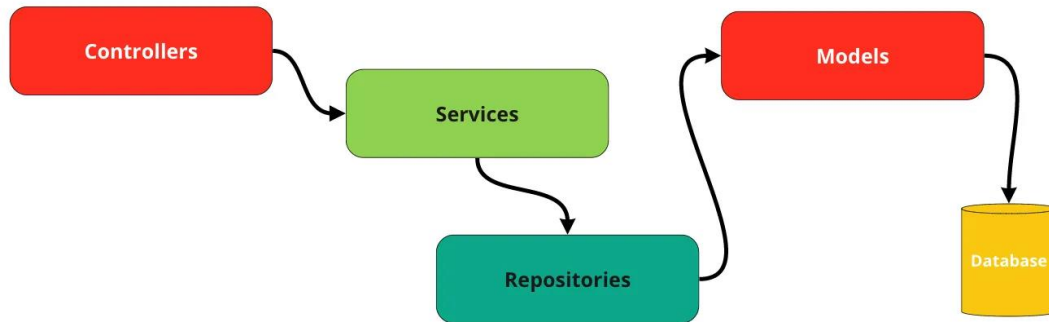


Figure 26. Service-Repository Pattern

6.5.3 Security Measures

Security was paramount in the backend design, given the sensitive nature of IoT device control and user data. The implementation includes multiple layers of defense to protect against common vulnerabilities and ensure data confidentiality, integrity, and availability.

Multi-Token Authentication System

AuraHome implements a sophisticated dual-token authentication mechanism using **Laravel Sanctum**:

1. **Access Token:** Short-lived token used for API requests. Minimizes exposure window if compromised.
2. **Refresh Token:** Long-lived token used exclusively to obtain new access tokens. Cannot be used for API requests, reducing attack surface.

Upon successful login, users receive both tokens. When the access token expires, the mobile app automatically uses the refresh token to obtain a new access token without requiring re-authentication. This provides security without sacrificing user experience.

Two-Factor Authentication (2FA)

One of the standout security features is the comprehensive **Two-Factor Authentication** system supporting multiple methods:

1. **Email-Based OTP (One-Time Password)**

When enabled, users must enter a 6-digit code sent to their email after providing correct credentials. The OTP is:

- Generated using cryptographically secure random functions
- Valid for 10 minutes
- Single-use only (consumed after verification)
- Rate-limited to prevent brute force attacks

Implementation leverages the OtpService which stores codes in the database with expiration timestamps and validation counters.

2. **Authenticator App (TOTP - Time-based One-Time Password)**

Users can configure Google Authenticator, Authy, or similar apps:

- Backend generates a secret key using the pragmarx/google2fa-laravel package
- QR code URL is provided for easy scanning
- TOTP codes rotate every 30 seconds
- Recovery codes (10 single-use backup codes) are generated and encrypted

The system requires users to verify a code before 2FA is fully enabled, ensuring proper setup.

Dual-Key Encryption Architecture

AuraHome implements a **two-key encryption strategy** for enhanced security:

1. **Laravel's APP_KEY**: Used for session encryption, cookie signing, and general application security (standard Laravel usage).
2. **Custom ENCRYPTION_KEY**: A separate encryption key dedicated exclusively to sensitive device data:
 - Device tokens (unique identifiers for ESP32 devices)
 - MQTT passwords
 - Device metadata (configuration, credentials)

The custom Encryptor service (in App\Services\Encryptor.php) uses Laravel's AES-256-CBC cipher but with the independent key. This ensures that even if APP_KEY is compromised, device credentials remain protected.

Role-Based Access Control (RBAC)

The system defines three distinct roles with hierarchical permissions:

- **Admin:** Full system access including user management, device oversight, subscription approval, system configuration, and audit log review.
- **Technician:** Limited to device management, technical support, and viewing users. Cannot approve subscriptions or modify users.
- **Customer:** Access to own devices, sensors, controls, subscription management, and profile settings.

Permissions are enforced at multiple levels:

- **Route Middleware:** Blocks unauthorized access before reaching controllers
- **Policy Classes:** Fine-grained authorization for specific model operations
- **Filament Resources:** Admin panel automatically respects permissions

Password Security

- **Strong Password Requirements:** Custom StrongPassword validation rule enforces:
 - Minimum 12 characters
 - At least one uppercase letter
 - At least one lowercase letter
 - At least one number
 - At least one special character
- **Password Hashing:** Uses bcrypt with automatic salt generation.
- **Password Reset Control:** Admin-controlled flag password_reset_allowed prevents unauthorized password resets. Users must contact support to enable reset capability, preventing social engineering attacks.

Audit Logging

All administrative and technician actions are automatically logged using the HasAuditLog trait. Audit logs capture:

- User who performed the action
- Timestamp of operation
- Action type (create, update, delete)
- Affected model and record ID
- IP address and user agent
- Changes made (before/after values for updates)

This provides an immutable record for compliance and forensic analysis.

6.5.4 MQTT Integration

The **MQTT broker** is central to AuraHome's IoT communication, facilitating real-time bidirectional messaging between devices and the cloud backend. The system uses **Mosquitto** with the **Dynamic Security plugin**, providing programmatic management of users, roles, and access control lists (ACLs).

Mosquitto Dynamic Security Plugin

Unlike traditional MQTT configurations requiring manual file editing, the Dynamic Security plugin enables the backend to:

- Create MQTT users programmatically
- Define per-user namespaces and permissions
- Dynamically update ACLs without broker restart
- Enforce topic-level access control
- Audit MQTT authentication and authorization

The backend communicates with the Dynamic Security plugin via special MQTT control topics:

- **Command Topic:** \$CONTROL/dynamic-security/v1
- **Response Topic:** \$CONTROL/dynamic-security/v1/response

User Namespace Architecture

Each AuraHome user is assigned a dedicated MQTT namespace following the pattern:

➔ `users/{username}/devices/{device_token}/{category}/{id}`

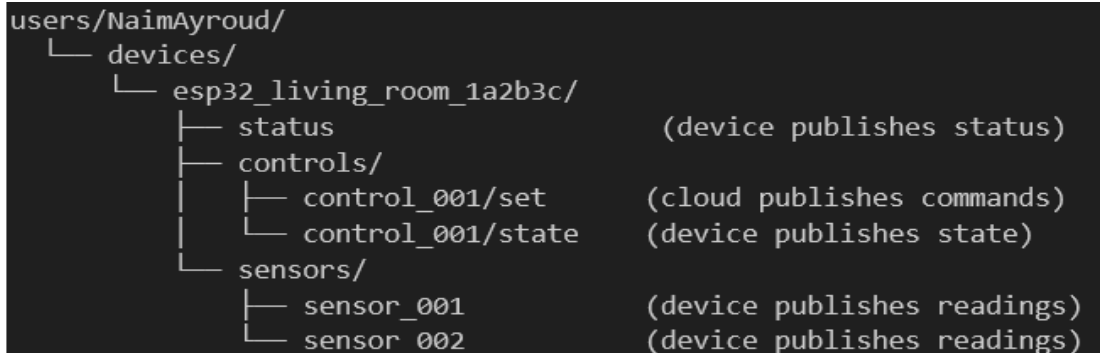


Figure 27. MQTT Topic Structure Example

MQTT Credential Lifecycle

The backend manages MQTT credentials throughout the user lifecycle:

1. User Registration

When a new user is created in Filament's CreateUser.php page (in the afterCreate hook), the system:

- Generates a secure MQTT password
- Creates MQTT user via Dynamic Security API
- Creates a role: {username}_role with ACL permissions
- Assigns the role to the MQTT user
- Encrypts and stores MQTT credentials in the database

Note: Creation happens in Filament's afterCreate hook (NOT observers) because roles/permissions aren't available during model observation.

2. User Deletion

When a user is deleted, the UserObserver dispatches a background job CleanupMqttNamespace which:

- Deletes the MQTT user from Mosquitto
- Removes the user's role
- Cleans up any orphaned ACLs

Using a queued job prevents blocking the HTTP response and allows retry logic if the MQTT broker is temporarily unavailable.

MosquittoDynamicSecurityService

The MosquittoDynamicSecurityService (located in App\Services\MosquittoDynamicSecurityService.php) encapsulates all MQTT Dynamic Security operations:

Key Methods:

- createUserRole(string \$username): Creates role with namespace ACL
- createMqttUser(string \$username, string \$password): Registers MQTT user
- assignRoleToUser(string \$username, string \$roleName): Links user to role
- deleteUserRole(string \$username): Removes role and ACLs
- deleteMqttUser(string \$username): Unregisters MQTT user
- executeCommand(array \$commands): Generic command executor with response handling

The service uses the php-mqtt/client library to establish connections, subscribe to response topics, and publish commands. It implements timeout handling and response parsing to ensure reliable communication with the broker.

ACL Configuration Example:

```
[
    'acltype' => 'publishClientSend', // Permission type
    'topic' => "users/{username}/#", // Topic pattern (# = multi-level wildcard)
    'priority' => 0, // ACL priority
    'allow' => true // Allow or deny
]
```

Figure 6.X+3:ACL Configuration Example

MQTT Topic Generation

Models generate their MQTT topics programmatically:

- **DeviceModel:** Implements topicBase() returning users/{username}/devices/{device_token}
- **Control Model:** Implements setTopic() appending /controls/{control_id}/set

- **Sensor Model:** Implements readTopic() appending /sensors/{sensor_id}

This ensures consistent topic structure and simplifies publishing/subscribing logic.

6.5.5 Deployment and Production Setup

The backend is deployed on **Hostinger's KVM4 VPS** plan, providing dedicated resources including:

- 4 CPU cores
- 8 GB RAM
- 200 GB NVMe SSD storage
- Full root access for custom configurations

Server Configuration

The production environment runs:

- **Ubuntu 22.04 LTS:** Stable Linux distribution with long-term support
- **Nginx:** High-performance web server with reverse proxy for Laravel
- **PHP 8.3 FPM:** Fast PHP processor with optimized settings
- **MySQL 8.0:** Database server with InnoDB engine
- **Mosquitto 2.0.18:** MQTT broker with Dynamic Security plugin
- **Supervisor:** Process manager for queue workers and schedulers

Laravel Queue System

Background jobs are essential for non-blocking operations:

- **MQTT Namespace Cleanup:** Asynchronously deletes MQTT users and roles
- **Email Sending:** Queues OTP emails and notification emails
- **Push Notifications:** Batches FCM delivery to avoid rate limits
- **Subscription Expiration:** Checks and updates expired subscriptions

6.6 Mobile Application implementation

The mobile application serves as the primary user interface for AuraHome, providing a sophisticated yet intuitive platform for managing IoT devices, monitoring sensor data, and controlling smart home systems in real-time. Developed using Flutter, the application implements a robust cross-platform solution that maintains native performance while ensuring a consistent user experience across Android and iOS devices. The implementation prioritizes security, scalability, and maintainability through a clean architecture approach, comprehensive state management, and seamless integration with the Laravel backend and MQTT broker.

6.6.1 Technologies Used

The mobile application leverages a carefully selected technology stack designed to meet the demanding requirements of real-time IoT control and secure user management:

- **Flutter 3.x:** Google's cross-platform UI framework enabling development of native Android and iOS applications from a single Dart codebase. Flutter was chosen for its excellent performance, expressive UI capabilities, hot reload development workflow, and extensive ecosystem of packages.
- **Dart Programming Language:** The foundational language for Flutter development, providing strong typing, null safety, and modern language features including `async/await`, streams, and functional programming constructs.
- **State Management & Architecture:**
 - **flutter_bloc (Cubit):** Implements the BLoC (Business Logic Component) pattern for predictable state management. Cubits provide a simplified approach to managing application state with clear separation between business logic and UI.
 - **fpdart:** Functional programming library enabling robust error handling through `TaskEither` monads, eliminating null checks and providing type-safe error propagation throughout the application.

- **GetIt:** Dependency injection container managing service lifetimes and enabling loose coupling between components through interface-based programming.
- **Code Generation & Immutability:**
 - **freezed:** Generates immutable classes with union types, eliminating boilerplate code for data classes, state classes, and API models while ensuring type safety.
 - **json_serializable:** Automates JSON serialization/deserialization for API communication, maintaining snake_case to camelCase conversion.
 - **build_runner:** Coordinates all code generation tasks, producing .freezed.dart, .g.dart, and .drift.dart files.
- **Networking & Real-Time Communication:**
 - **Dio:** Feature-rich HTTP client with interceptor support, request/response transformation, and comprehensive error handling for RESTful API communication.
 - **mqtt_client:** Native MQTT client library enabling real-time bidirectional communication with Mosquitto broker for device control and sensor data streaming.
- **Local Data Persistence:**
 - **Drift:** Type-safe SQL database library with compile-time query validation, migration support, and reactive streams for offline-first functionality.
 - **SharedPreferencesWithCache:** High-performance key-value storage for user preferences and cached authentication state.
 - **FlutterSecureStorage:** Hardware-backed encrypted storage for sensitive data, including authentication tokens and MQTT credentials.
- **Firebase Integration:**
 - **Firebase Cloud Messaging (FCM):** Handles push notifications for device alerts, subscription updates, and real-time system events.
- **Monitoring & Debugging:**

- **Sentry:** Production error tracking and performance monitoring with automatic crash reporting, breadcrumb tracking, and user feedback integration.
- **logger:** Structured logging system integrated with Sentry for comprehensive debugging capabilities.
- **UI & Design:**
 - **flutter_screenutil:** Responsive design system ensuring consistent UI scaling across different screen sizes based on a 390×844 reference design.
 - **lucide_icons_flutter:** Comprehensive icon library providing consistent visual language throughout the application.
 - **flutter_animate:** Declarative animation framework enabling smooth transitions and micro-interactions.

6.6.2 Architectural Design

The application implements **Clean Architecture** principles, establishing a clear separation of concerns through a three-layer structure organized by features. This architectural approach ensures maintainability, testability, and scalability while adhering to SOLID principles.

Feature-Based Modular Structure

The codebase is organized into self-contained feature modules, with each feature implementing the complete three-layer architecture:

Core Features:

- **auth:** User authentication, login, OTP verification, and 2FA workflows
- **home:** Dashboard, quick actions, and system overview
- **device:** Device management, configuration, and status monitoring
- **notifications:** Push notification handling and in-app notification center
- **settings:** User preferences, theme management, and account settings
- **splash:** Application initialization and authentication state verification

Each feature exports its components through a barrel file (e.g., `auth.dart`), encapsulating implementation details and providing clean public APIs.

Three-Layer Architecture

1. Domain Layer

The innermost layer contains pure business logic with zero external dependencies:

- **Entities:** Immutable data classes representing core business concepts, defined using `@freezed` for type safety and union type support
- **Repository Interfaces:** Abstract contracts defining data operations without implementation details
- **Use Cases:** Single-responsibility classes encapsulating specific business operations, each accepting repository interfaces via constructor injection

2. Data Layer

Implements domain layer contracts and manages data operations:

- **Models:** Data transfer objects with JSON serialization support, including `.toEntity()` conversion methods
- **Data Sources:**
 - **Remote:** API communication via Dio, handling HTTP requests, and response parsing
 - **Local:** Drift database operations for offline storage
 - **MQTT:** Real-time device communication through MQTT client
- **Repository Implementations:** Concrete implementations of domain repository interfaces, coordinating between multiple data sources

3. Presentation Layer

Handles UI rendering and user interaction:

- **Cubits:** State management units extending `Cubit<State>` from `flutter_bloc`, managing UI state and coordinating use case execution
- **States:** Freezed union types representing all possible UI states (initial, loading, success, error)
- **Pages:** Top-level screens registered in the routing system

- **Widgets:** Reusable UI components extracted using the “part of” pattern for related widgets

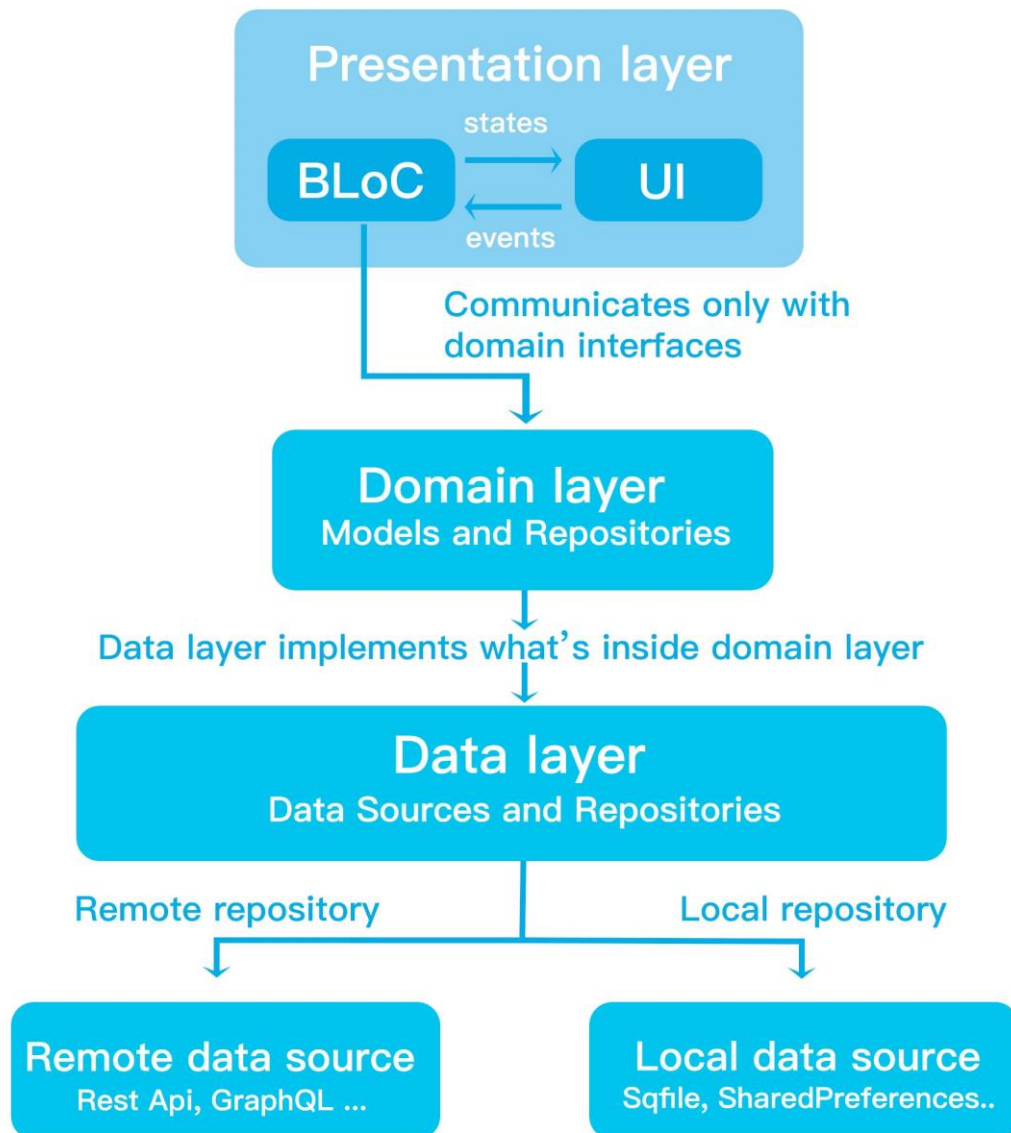


Figure 28. Clean Architecture Design Pattern [14]

Dependency Injection System

All dependencies are managed through **GetIt** service locator:

- **Singletons:** Core services like Dio HTTP client, SharedPreferences, FlutterSecureStorage, and global Cubits (ThemeCubit, SessionCubit)

- **Factories:** Use cases, repositories, and data sources created on-demand
- **Feature Modules:** Each feature registers its dependencies in separate part files (e.g., `auth_dependencies.dart`)

This approach enables constructor injection throughout the application, facilitating testing through mock implementations and ensuring loose coupling.

6.6.3 Security Implementation

Security is paramount in the mobile application, given its role in controlling physical IoT devices and handling sensitive user credentials.

Multi-Token Authentication Flow

The application implements a sophisticated dual-token system integrated with Laravel Sanctum:

- **Access Token:** Short-lived JWT used for API authorization, attached to requests via `AuthInterceptor`
- **Refresh Token:** Long-lived token stored securely for obtaining new access tokens without re-authentication

Token Refresh Mechanism:

The `AuthInterceptor` automatically handles token expiration:

1. Detects 401 Unauthorized responses
2. Queues pending requests
3. Attempts token refresh using refresh token
4. Retries queued requests with a new access token
5. On refresh failure, clears the session and redirects to login

Secure Token Storage

All authentication tokens and MQTT credentials are stored in **FlutterSecureStorage**, which utilizes:

- **Android:** AES encryption backed by Android Keystore
- **iOS:** Keychain Services with hardware-backed encryption

User profile data is cached in **SharedPreferencesWithCache** for performance, while sensitive credentials remain exclusively in secure storage.

Two-Factor Authentication Support

The application fully supports the backend's 2FA login system:

- **Email OTP:** 6-digit code entry screen with countdown timer and resend functionality
- **Authenticator App (TOTP):** QR code scanning and manual code entry for Google Authenticator/Authy integration
- **Recovery Codes:** Secure display and storage of single-use backup codes

6.6.4 Real-Time Device Communication

MQTT Integration Architecture

The application establishes direct MQTT connections to the Mosquitto broker, enabling real-time bidirectional communication with ESP32 devices:

Connection Flow:

1. User authenticates via REST API
2. Backend provides MQTT credentials (username, encrypted password)
3. Application connects to the broker using the `mqtt_client` library
4. Subscribes to user's namespace: `users/{username}/devices/+ /sensors/+`
5. Publishes control commands to:
`users/{username}/devices/{token}/controls/{id}/set`

MqttService Implementation:

- Manages persistent WebSocket connection with auto-reconnect
- Implements connection state monitoring
- Provides typed streams for sensor data updates
- Handles connection lifecycle (connect, disconnect, reconnect)

Device MQTT Data Source:

- Subscribes to device-specific topics on feature initialization
- Parses incoming sensor readings into typed models

- Publish control commands with QoS 1 guarantee
- Maintains topic generation logic matching backend namespace structure

Real-Time Sensor Updates

Sensor data streams are consumed by DeviceCubit, updating the UI in real-time:

- **Stream Subscription:** Cubit subscribes to MQTT data source streams
- **State Emission:** New sensor readings trigger state updates
- **UI Reactivity:** BlocBuilder widgets automatically rebuild on state changes
- **Error Handling:** Connection failures emit error states with retry mechanisms

6.6.5 Routing and Navigation

The application uses **go_router** for declarative, type-safe navigation:

Key Features:

- **Deep Linking:** Support for URL-based navigation and universal links
- **Shell Routes:** Wraps authenticated screens with SessionCubit for automatic session verification
- **Custom Transitions:** Sliding and fading page transitions via TransitionFactory
- **Route Guards:** Automatic redirection based on authentication state

Session Management:

SessionCubit wraps the main application shell, monitoring the authentication state:

- Loads cached user data on app launch
- Validates token expiration
- Listens for authentication state changes
- Triggers logout and navigation on session expiration
- Displays SessionExpiredDialog on authentication failures

Chapter 7: Results & Testing

In this chapter, we present the testing methodology and the results of the proposed system. Each AI component is evaluated independently to verify its correctness, accuracy, and reliability before full integration. We also describe the dataset used and the testing environment, supported by screenshots and practical examples. In addition, server-side testing from the software perspective is included to assess performance, stability, and response behavior under real usage conditions.

7.1 Speech Emotion Recognition (SER) Testing and Results

This section presents the testing procedure and results of the Speech Emotion Recognition (SER) module. The model was evaluated using two datasets with different languages and characteristics, namely **MSP-IMPROV** (English) and **emoDB** (German). This evaluation provides insights into both **within-domain performance** (English training domain) and **cross-lingual generalization** when testing on a different language.

7.1.1 Datasets Description

MSP-IMPROV (English, Training Dataset):

The MSP-IMPROV (MSP-IMProvisation) dataset contains naturalistic emotional speech collected from improvised interactions. It includes approximately **8,438 utterances** from **12 actors** (6 male, 6 female), covering four primary emotions: **anger**, **happiness**, **sadness**, and **neutral**, in addition to other emotional states. The dataset is considered suitable for training and evaluating models on realistic conversational emotions, since it provides spontaneous expressions with diverse speaking styles and varying emotional intensity.

emoDB (German, Cross-Lingual Evaluation):

The Berlin Database of Emotional Speech (emoDB) is a German emotional speech dataset containing **535 utterances** from **10 professional actors** (5 male, 5 female). It

includes seven emotion classes: **anger, boredom, disgust, fear, happiness, sadness, and neutral**. In this project, emoDB is used as a cross-lingual test set to evaluate the model’s ability to generalize to **German speech**, where linguistic and acoustic patterns differ from the English training data.

7.1.2 Results

The model performance was assessed using standard classification metrics, including **Accuracy, Precision, Recall, and F1-Score**. The SER module achieved its best performance on **MSP-IMPROV**, reaching an accuracy of **77.45%** with **0.783 precision, 0.775 recall**, and an **F1-score of 0.775**. However, when evaluated on **emoDB**, performance decreased significantly, with an accuracy of **31.59%**, **0.478 precision, 0.316 recall**, and an **F1-score of 0.27**.

Table 4. SER performance metrics on MSP-IMPROV and emoDB

Dataset	Accuracy (%)	Precision	Recall	F1-Score
MSP-IMPROV	77.45%	0.783	0.775	0.775
emoDB	31.59%	0.478	0.316	0.27

7.1.3 Discussion

The results show that the SER model performs strongly on **MSP-IMPROV**, which aligns with the English training domain and includes a simpler primary emotion set. In contrast, the model’s performance drops on **emoDB**, which indicates limited cross-lingual generalization. This reduction is expected due to the language shift from English to German, the difference in emotion class definitions, and variations in speaking style and recording conditions between the two datasets.

7.2 Gun Detection Testing

This section evaluates the performance of the **Gun Detection** module used in the DGP pipeline. The objective of this testing is to measure how accurately the model detects guns in images and to validate its behavior using both quantitative metrics and visual analysis.

7.2.1 Dataset and Testing Setup

The gun detection model was tested on **1076 images** collected from the same dataset that was adopted in **Experiment 3** in the Implementation chapter. Using the same dataset ensures that the evaluation is consistent with the system's operating conditions and reflects realistic scenarios from the project environment.

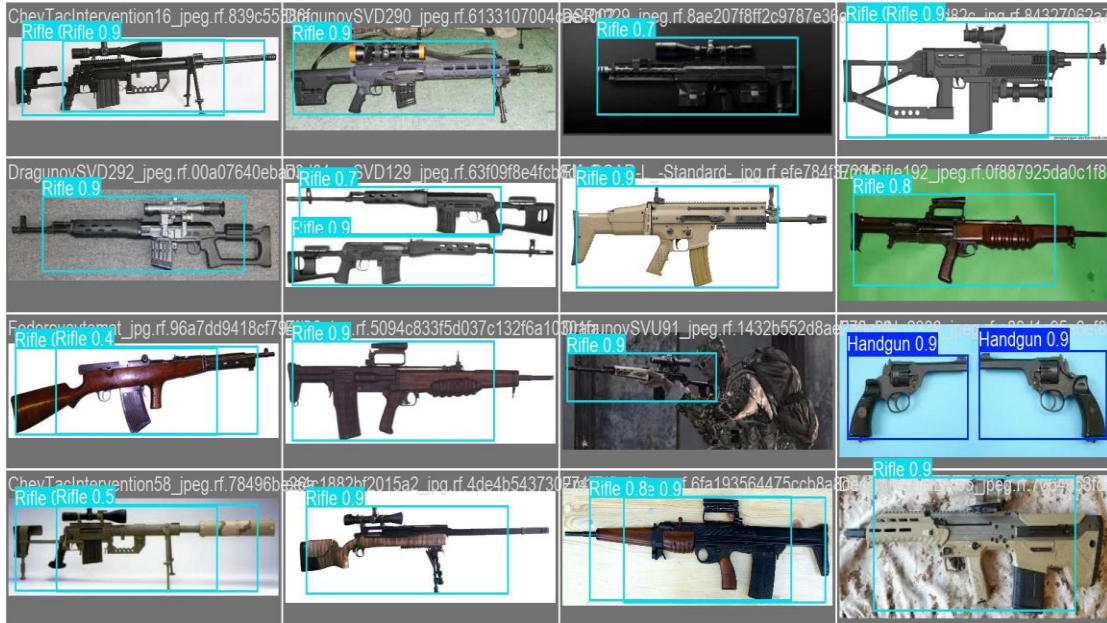


Figure 29. Sample detections from Gun Detection test dataset

7.2.2 Quantitative Results

The performance of the gun detector was evaluated using standard object detection metrics, including **Precision**, **Recall**, **mAP@0.5 (mAP50)**, and **mAP@0.5:0.95 (mAP50–95)**. The results obtained are:

- **Precision:** 0.838
- **Recall:** 0.835

- **mAP50:** 0.872
- **mAP50–95:** 0.521

These results indicate strong detection capability. The close values of precision and recall show that the model maintains a balanced behavior between minimizing false positives and detecting most true gun instances. Moreover, the **mAP50** value reflects high accuracy under a moderate IoU threshold, while the lower **mAP50–95** value is expected because this metric is stricter and penalizes imperfect bounding-box localization across multiple IoU levels.

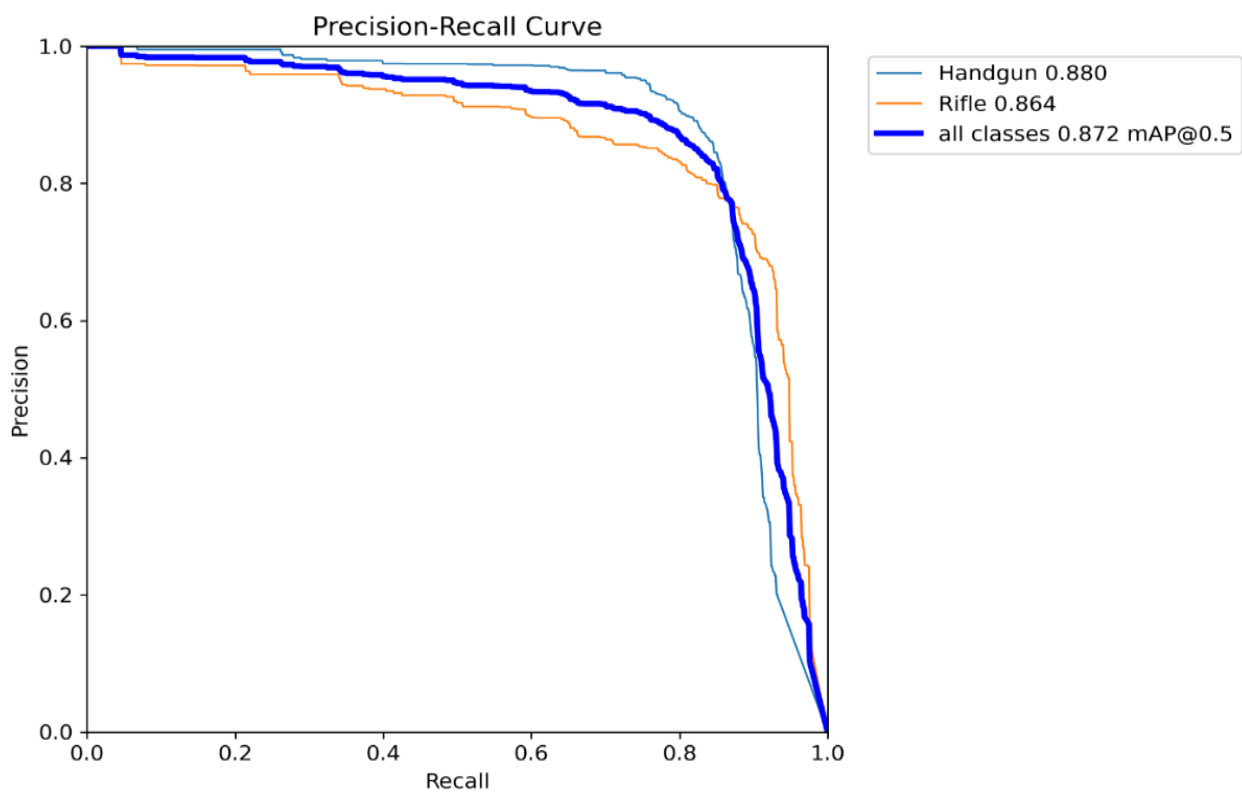


Figure 30. Precision–Recall curve for Gun Detection Core (test set)

7.2.3 Discussion

Overall, the gun detection model demonstrates reliable performance on the project test set. The achieved **precision (0.838)** suggests that most predicted gun detections are correct, meaning the model produces relatively few false alarms. At the same time, the **recall (0.835)** indicates that the detector is able to capture the majority of gun instances in the dataset, which is essential for safety-related use cases.

The **Precision–Recall curve** further confirms stable behavior across different confidence thresholds, showing that the model preserves good precision while maintaining high recall. The difference between **mAP50 (0.872)** and **mAP50–95 (0.521)** highlights that the detector is more consistent at identifying the presence of

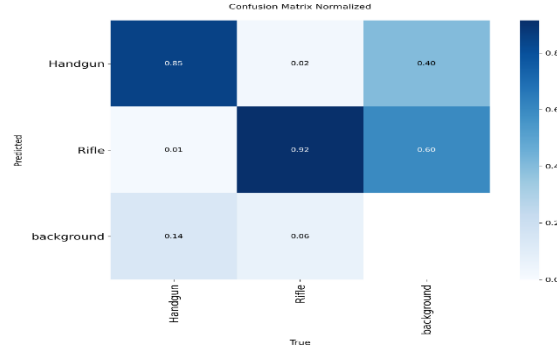


Figure 31. Normalized Confusion Matrix for Gun Detection Core

guns than at achieving perfectly tight bounding boxes under strict localization requirements, which is a common observation in object detection tasks.

7.3 Masked People Detection Testing

This section evaluates the performance of the **Masked People Detection** module within the DGP pipeline. The objective of this testing is to measure how accurately the model detects masked individuals in images and to validate its behavior using quantitative metrics and visual analysis.

7.3.1 Dataset and Testing Setup

Masked people detection testing was applied on **399 images** taken from the same dataset that was used in the Implementation chapter, specifically in **Experiment 3**, to ensure consistent evaluation conditions and comparable results across the system components.



Figure 32. Sample detections from Masked People Detection test dataset

7.3.2 Quantitative Results

The performance of the masked-people detector was assessed using standard object detection metrics, including **Precision**, **Recall**, **mAP@0.5 (mAP50)**, and **mAP@0.5:0.95 (mAP50–95)**. The results obtained are:

- **Precision:** 0.935
- **Recall:** 0.740
- **mAP50:** 0.856
- **mAP50–95:** 0.614

These results indicate that the model achieves very high precision, meaning that most predicted masked-person detections are correct and false positives are limited. However, recall is relatively lower, which suggests that the model may miss some masked individuals in challenging cases, such as partial occlusion, small or distant faces, extreme head poses, low-resolution frames, or crowded scenes. The **mAP50** score confirms strong detection under a moderate IoU requirement, while **mAP50–95** reflects stricter localization performance across multiple IoU thresholds.

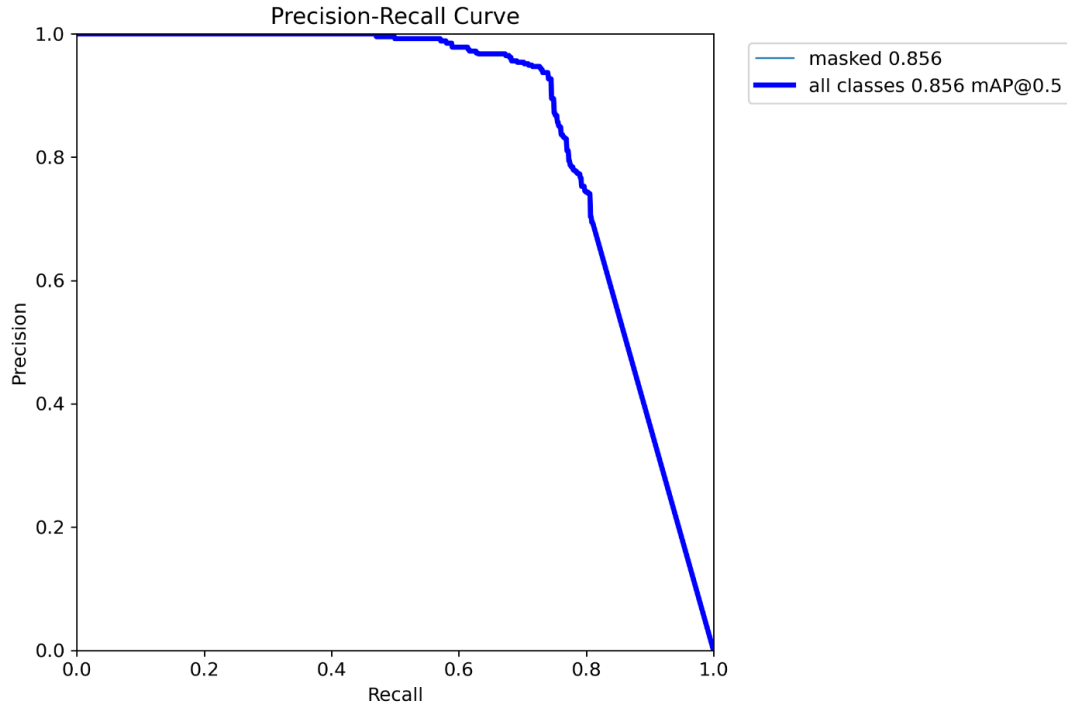


Figure 33. Precision-Recall Curve for Masked People Detection Core (test set)

7.3.3 Discussion

Overall, the masked-people detector demonstrates reliable detection capability on the project dataset, especially in terms of precision. The high precision value (**0.935**) indicates that the model is conservative and tends to output detections when it is confident, which is desirable in many surveillance and safety applications to reduce false alarms. On the other hand, the recall value (**0.74**) shows that some masked cases may not be detected, which can happen when the mask is partially visible, the person is far from the camera, or facial regions are occluded.

The gap between **mAP50 (0.856)** and **mAP50-95 (0.614)** suggests that while the model is effective at recognizing masked individuals, performance decreases under stricter localization constraints. This is expected in real-world detection tasks where bounding boxes can be affected by occlusions and crowded environments.

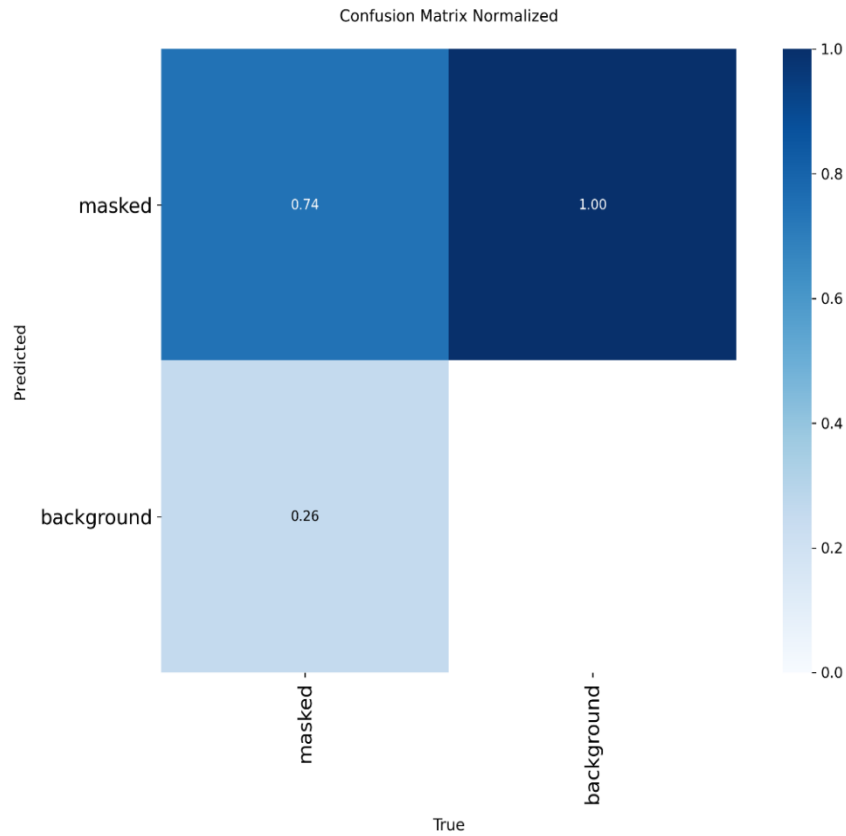


Figure 34. Normalized Confusion Matrix for Masked People Detection Core

7.4 Face Anti-Spoofing Testing

7.4.1 DeepFace Library Adoption and Baseline Performance

The face anti-spoofing component was implemented by directly adopting the **DeepFace** library and its built-in anti-spoofing capability, without training or fine-tuning a custom model. Therefore, the achieved performance reflects the intrinsic behavior of the DeepFace implementation under the same deployment conditions and camera inputs. In this context, the evaluation focuses on verifying practical robustness in the project environment rather than reporting a newly trained model's accuracy.

7.4.2 Test Dataset and Experimental Validation

To validate real-world reliability, face anti-spoofing was tested on approximately **150 diverse samples** collected under varied conditions. The test set included children and adults, females with and without hijab, subjects with and without glasses, with and without hats, different hairstyles including bald subjects, and also visually similar subjects (for example, two similar-looking siblings) to increase the difficulty of the scenarios. The evaluation was conducted under both good and poor lighting conditions, and spoof attempts were examined using different presentation media, including mobile screen images and printed photographic images. **Qualitative samples from the conducted tests are illustrated in Figure 34**, highlighting representative bona fide cases and spoof attempts under different acquisition and presentation conditions.

7.4.3 Results and Discussion

Across the tests conducted, the anti-spoofing module achieved an overall accuracy of approximately **95%**. The results indicate that the component is generally robust against common spoofing attempts, particularly when the face region is clear and lighting is adequate. Missed detections, when they occur, are typically expected in challenging cases such as poor illumination, partial occlusion, or low-quality capture where texture and depth cues become less distinguishable. Overall, the observed performance supports the suitability of the DeepFace anti-spoofing module for the project's access-control workflow, while highlighting that environmental conditions remain a key factor influencing reliability.

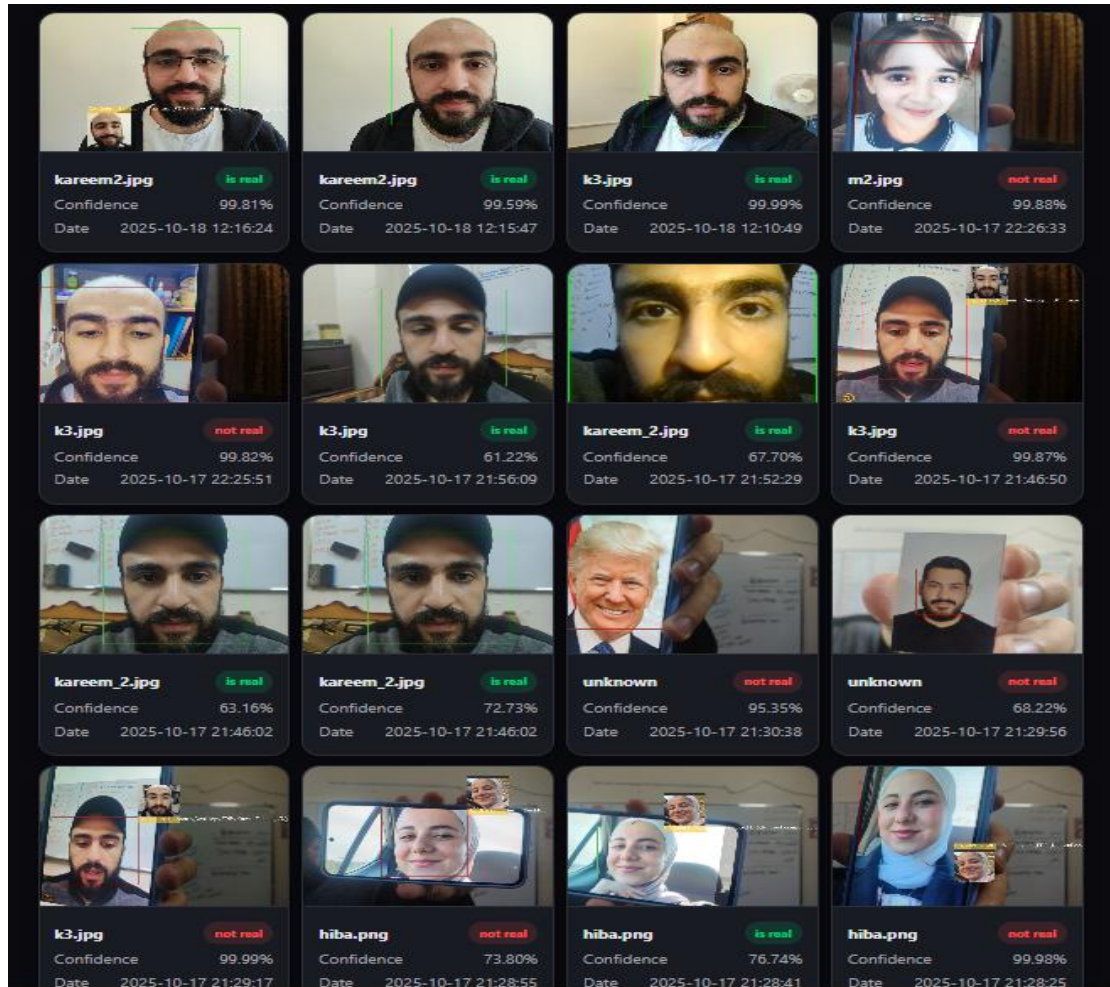


Figure 35. anti-spoofing test cases under different lighting and presentation conditions.

7.5 Server Testing

In this section, we conduct performance testing on the Hostinger KVM4 VPS plan using Apache JMeter. The testing includes baseline testing, load testing, and stress testing to evaluate the server's capacity to handle IoT device traffic and API requests.

7.5.1 Server Specifications

The Hostinger KVM4 VPS plan is a virtual private server offering designed for medium-to-large-scale online projects. It features modern hardware, including:

- **RAM:** The Hostinger KVM4 VPS plan offers 8 GB of RAM, suitable for medium-scale IoT applications with concurrent device connections and user requests.

- **CPU:** The plan includes 4 vCPU cores, powered by modern Intel Xeon or AMD Epyc processors, ensuring robust processing power for MQTT message handling and API request processing.
- **Storage:** The plan provides 200 GB of NVMe SSD storage, known for its fast data access speeds, ideal for database operations and log storage.
- **Bandwidth:** The plan offers 16 TB of data transfer bandwidth and a maximum network speed of 300Mbps, supporting high traffic needs from IoT devices and mobile applications.

This setup ensures the server can handle significant loads from multiple ESP32 devices, concurrent mobile app users, and real-time MQTT communication, making it an ideal candidate for performance testing.

7.5.2 JMeter Tool Details

Apache JMeter is an open-source, Java-based tool widely used for load testing and performance measurement of web applications, servers, and APIs. It simulates multiple users sending requests to the server, allowing for the analysis of key performance metrics such as response times, throughput, and error rates. JMeter's flexibility, extensive documentation, and strong community support make it a popular choice for performance testing. For these tests, JMeter is configured to send HTTP requests to the AuraHome API endpoints, simulating real-world user interactions including authentication, device registration, and control commands.

7.5.3 Baseline Testing

The baseline test simulates 50 users accessing the server over a ramp-up period of 60 seconds. This test establishes the server's performance under normal load conditions, representing a typical number of concurrent API requests from mobile applications and device status updates.

Test Configuration:

- Number of Threads (users): 50
- Ramp-Up Period: 60 seconds
- Loop Count: 1

Results:

The table below summarizes key performance metrics, including: **Samples** (total requests), **Average** (mean response time), **Median** (middle value of response times), **Max** (maximum response time), **Error rate** (percentage of failed requests), and **Throughput** (rate of requests processed per second). These metrics highlight the server's efficiency, reliability, and capacity under light load.

Table 5. Baseline Test Metrics

Label	#Samples	Average	Median	Min	Max	Error%	Throughput
Baseline Test	50	342 ms	328 ms	285 ms	548 ms	0.00%	0.83/sec

The graph shows a stable response time throughout the test, indicating consistent performance under normal load with minimal variation. The server handles authentication and device API requests efficiently with no errors.

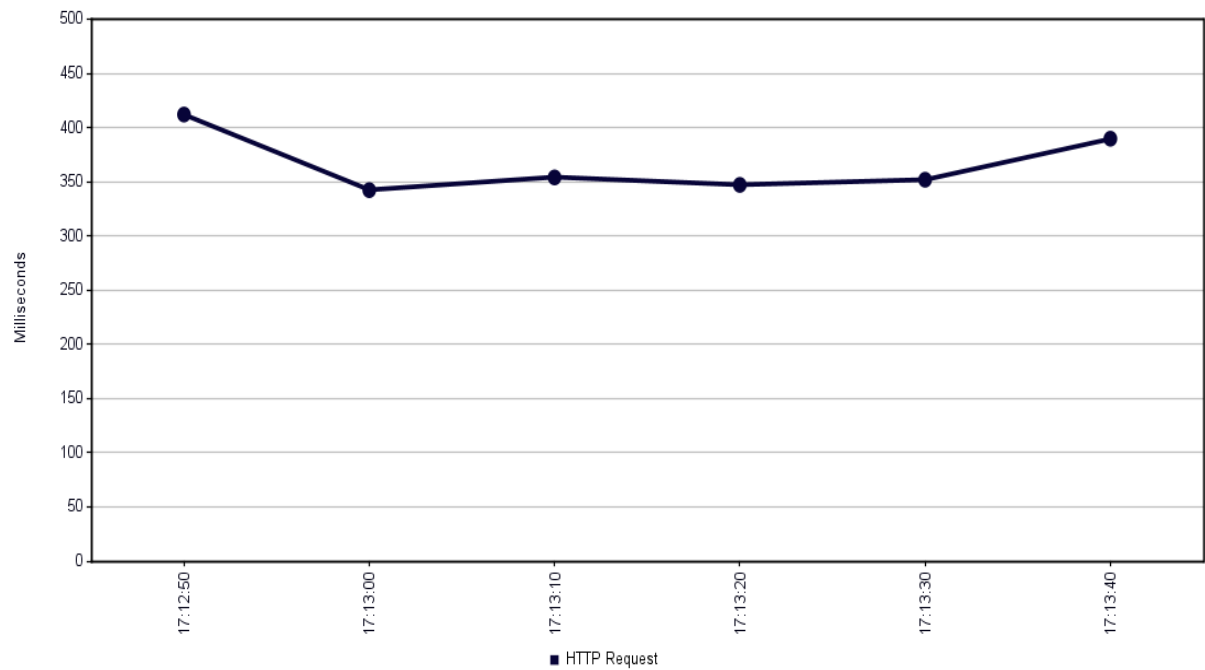


Figure 36. Response Time over Test Duration for Baseline Test

7.5.4 Load Testing

The load test increases the number of users to 500 with a ramp-up period of 60 seconds to assess the server's performance under heavier load, simulating peak usage hours with multiple concurrent device operations and user sessions.

Test Configuration:

- Number of Threads (users): 500
- Ramp-Up Period: 60 seconds
- Loop Count: 1

Results:

The table below summarizes key performance metrics.

Table 6. Load Test Metrics

Label	#Samples	Average	Median	Min	Max	Error%	Throughput
Load Test	500	421 ms	378 ms	291 ms	2,687 ms	0.00%	8.1/sec

The graph indicates a moderate increase in response time as the number of concurrent users grows, with some variability during peak moments but no critical failures. The server handles the increased load adequately, demonstrating good scalability for IoT device management operations.

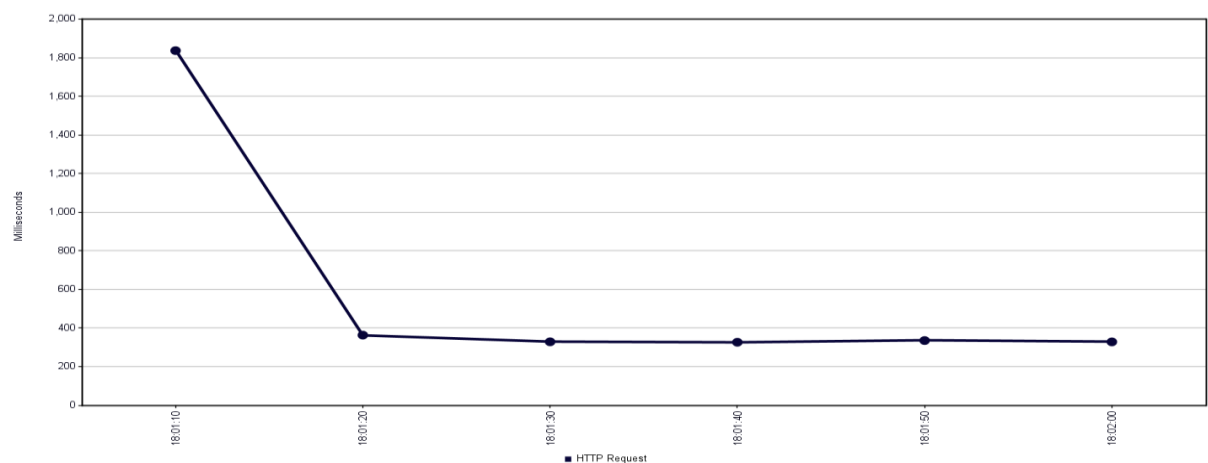


Figure 37. Response Time over Test Duration for Load Test

7.5.5 Stress Testing

The stress test pushes the server to its operational limits by simulating 1500 users with a 60-second ramp-up, aiming to identify the breaking point or performance degradation under extreme conditions. This represents a scenario where hundreds of IoT devices are simultaneously sending data and users are actively controlling devices.

Test Configuration:

- Number of Threads (users): 1500
- Ramp-Up Period: 60 seconds
- Loop Count: 1

Results:

The table below summarizes key performance metrics.

Table 7. Stress Test Metrics

Label	# Samples	Average	Median	Min	Max	Error %	Throughput
Stress Test	1500	14,832 ms	8,946 ms	124 ms	61,458 ms	0.533%	15.8/sec

The graph shows significant spikes in response time and some request failures (8 failed requests out of 1500), indicating the server is under severe stress and approaching its operational limits. The 0.533% error rate suggests the server can handle approximately 1400-1450 concurrent users before performance degrades significantly. This provides valuable insights for capacity planning and scaling strategies.

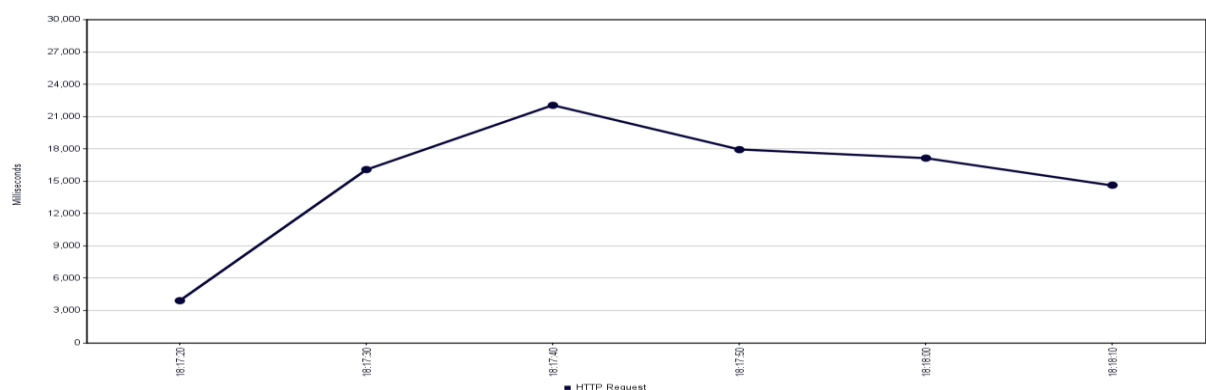


Figure 38. Response Time over Test Duration for Stress Test

7.5.6 Performance Testing Summary

The performance testing results demonstrate that the Hostinger KVM4 VPS effectively handles AuraHome's IoT workload under normal and elevated conditions:

- **Baseline Performance:** Excellent response times (342ms average) with zero errors under normal load (50 concurrent users)
- **Load Handling:** Good scalability with 500 concurrent users, maintaining sub-500ms response times with no failures
- **Stress Threshold:** Server begins showing performance degradation around 1500 concurrent users with 0.533% error rate
- **Recommended Capacity:** Safely supports 500-800 concurrent users with optimal performance

These results confirm that the current infrastructure is well-suited for the expected user base while providing headroom for growth. For scaling beyond 1000 concurrent users, horizontal scaling strategies (load balancing, database replication) should be considered.

Chapter 8: Conclusion and Future Work

8.1 Conclusion

This project demonstrates the feasibility of developing an integrated smart home security and monitoring system by combining an **MQTT broker** as a lightweight messaging backbone with AI-based analytics and cloud services such as **Firebase**. The proposed architecture enables real-time event detection and notification by connecting heterogeneous components, including sensors and embedded hardware, AI inference modules, and a cloud backend for synchronization and user-facing alerts. The obtained results confirm that MQTT-based communication, coupled with reliable AI components and a scalable cloud platform, can provide an effective foundation for a unified system capable of monitoring safety and security events and supporting timely responses.

8.2 Challenges

During the development and validation of the system, several practical and technical challenges were encountered:

1. **Hardware availability constraints:** Some required hardware components were difficult to obtain, which affected testing continuity and delayed certain integration stages.
2. **System integration complexity:** Achieving stable interoperability between hardware, software services, and AI modules required extensive debugging, interface alignment, and iterative refinement.
3. **Limited access to suitable datasets:** Obtaining high-quality datasets with appropriate size and diversity for training AI models was challenging. Building such datasets typically requires dedicated resources, structured collection pipelines, and financial support.

4. **High implementation costs:** The overall cost of the project is relatively high for undergraduate students, especially when external funding is not available.
5. **Dependency on stable internet connectivity:** In environments with unstable internet access, notification delivery may be delayed, which can temporarily reduce system responsiveness and affect time-critical alerts.
6. **Multidisciplinary skill requirement:** The project required practical expertise in electronics and hardware troubleshooting in addition to software and AI development. This posed a challenge during implementation, since the team's primary background was in computer engineering rather than specialized electronics.

8.3 Limitations

The current version of the system has several limitations that should be considered:

1. **Scalability of MQTT and user management:** As the number of users and connected devices increases, the system will require more advanced solutions for MQTT scalability, session handling, and structured user management to maintain reliability and avoid performance bottlenecks.
2. **Limited security evaluation scope:** Although functional validation and basic security measures were applied, the system has not yet undergone an industry-level security assessment such as extensive penetration testing, formal threat modeling, or external security auditing.
3. **Limited commercial-grade generalization:** The AI components were validated on available datasets and project-specific testing samples. However, the dataset diversity and coverage are not at a commercial scale, which may limit generalization across unseen environments, camera types, and real-world operating conditions.

8.4 Future Work

Several enhancements can be considered to further improve performance, reliability, and automation:

1. **Improving AI model accuracy:** Enhance detection robustness through better model architectures, additional training data, and optimized preprocessing to reduce false positives and false negatives.
2. **Advanced security assessment:** Extend security validation through deeper testing, broader threat modeling, and penetration testing to increase confidence in system resilience.
3. **Automated physical actuation:** Extend the system from notification to automated response, such as activating a fire extinguisher when a fire is detected or turning on a ventilation fan when gas leakage is detected.
4. **Voice assistant integration:** Integrate with a voice assistant to support voice-based control of doors and home functions, enabling a more complete smart home experience through natural voice commands.

References

- [1] Cedalo, “Pro Mosquitto broker and Rust application,” Cedalo.com, Jan. 2025. [Online]. Available: <https://cedalo.com/wp-content/uploads/2025/01/Pro-Mosquitto-broker-and-Rust-application.png>
- [2] Vision Platform, “Object detection example,” *VisionPlatform.ai*, Jan. 2024. [Online]. Available: <https://visionplatform.ai/wp-content/uploads/2024/01/object-detection.png>
- [3] Lockly Visage Zeno Series, Lockly, [Online]. Available: <https://lockly.com>. [Accessed: 12-Nov-2025] .
- [4] Eufy Video Smart Lock E330, eufy, [Online]. Available: <https://www.eufy.com>. [Accessed: 12-Nov-2025] .
- [5] Nest Protect, Google Nest Support, [Online]. Available: <https://support.google.com/googlenest>. [Accessed: 12-Nov-2025] .
- [6] Minut Point Smart Alarm, Security.org, [Online]. Available: <https://www.security.org/home-security-systems/>. [Accessed: 12-Nov-2025] .
- [7] Ambient.ai Weapon Detection, Ambient.ai, [Online]. Available: <https://ambient.ai/>. [Accessed: 12-Nov-2025] .
- [8] S.P. Shanthi and V. Manjula, “Weapon detection with FMR-CNN and YOLOv8 for enhanced crime prevention and security,” *Scientific Reports*, vol. 15, Article no. 26766, Jul. 2025, **doi: 10.1038/s41598-025-07782-0**.
- [9] C. Dewi, D. Manongga, Hendry, E. Mailoa, and K. D. Hartomo, “Deep Learning and YOLOv8 Utilized in an Accurate Face Mask Detection System,” *Big Data and Cognitive Computing*, vol. 8, no. 1, p. 9, Jan. 2024, **doi: 10.3390/bdcc8010009**.

- [10] Y. Li, Y. Zhang, and X. Wang, “Improved Speech Emotion Recognition Using Transfer Learning on Spectrograms,” *IEEE Transactions on Affective Computing*, vol. 13, no. 2, pp. 450–460, 2022, **doi: 10.1109/TAFFC.2022.3149956**.
- [11] Z. Zhao, Y. Zhang, and M. Zhou, “Speech Emotion Recognition Using Deep 1D & 2D CNN LSTM Networks,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, Brighton, UK, May 2019, pp. 7405–7409, **doi: 10.1109/ICASSP.2019.8683065**.
- [12] C. M. Prakaashana *et al.*, “Measuring the potential risk of re-identification of imaging research participants from open-source automated face recognition software,” *NeuroImage*, vol. 320, p. 121476, Oct. 2025, **doi: 10.1016/j.neuroimage.2025.121476**.
- [13] U. Ihsan, N. Z. Jhanji, H. Ashraf, F. Ashfaq, and F. A. Wicaksana, “A Real-Time Intelligent Surveillance System for Suspicious Behavior and Facial Emotion Analysis Using YOLOv8 and DeepFace,” *Engineering Proceedings*, vol. 107, no. 1, art. 59, Sep. 2025, **doi: 10.3390/engproc2025107059**.
- [14] DBestech, “Flutter Clean Architecture With TDD | BLoC | Cubit,” *DBestech*, Jul. 7, 2023. [Online]. Available: <https://www.dbestech.com/tutorials/flutter-bloc-clean-architecture-and-tdd>. [Accessed: Jan. 15, 2026].

ملخص

أوراهوم هو نظام أمن وسلامة للمنزل الذكي قائم على إنترنت الأشياء، يدمج التحكم بالدخول، ومراقبة التسلل، واستشعار المخاطر البيئية، وإرسال الإشعارات الفورية ضمن معمارية واحدة متكاملة. يعتمد النظام على عقد مبنية على ESP32 مع بوابة طرفية Raspberry Pi 5 لتنفيذ المعالجة محلياً، إضافة إلى طبقة سحابية تتضمن Laravel Backend وقاعدة بيانات MySQL ووسيط رسائل Mosquitto MQTT لتحقيق اتصال آمن بأسلوب النشر والاشتراك. كما يوفر تطبيق Flutter إمكانية المراقبة والتحكم عن بعد، بينما تُستخدم Firebase Cloud Messaging لإرسال التنبيهات بزمن شبه فوري.

تم دمج عدة مكونات ذكاء اصطناعي وتقييم كل منها بشكل مستقل. حقق نظام التعرف على المشاعر من الصوت دقة **77.45%** على مجموعة **MSP-IMPROV** قيمة (**F1 0.775**) ودقة **31.59%** على **emoDB** قيمة (**F1 0.27**)، مما يشير إلى محدودية التعميم عبر اللغات. أما كشف السلاح فتم اختباره على **1,076** صورة وحقق **Precision 0.838** و **Recall 0.835** و **mAP@0.5:0.95 0.521** و **mAP@0.5 0.872** وتم اختبار كشف الأشخاص المقنعين على **399** صورة وحقق **Precision 0.935** و **Recall 0.740** و **mAP@0.5 0.856** و **mAP@0.5:0.95 0.614** كما تم اعتماد ميزة كشف التزييف الوجهي من مكتبة DeepFace دون تدريب مخصص، وحقت دقة تقريبية **95%** على نحو **150** عينة متنوعة ضمن ظروف إضاءة مختلفة ومحاولات تزييف متعددة. وأظهرت اختبارات الضغط على السيرفر أداءً مستقراً تحت عدد مستخدمين متزامنين متوسط، بينما كشفت اختبارات الإجهاد عن حدود السعة، ما يدعم الحاجة لاحقاً لتحسينات تتعلق بالتوسع والأداء.

الجامعة العربية الدولية

الجامعة العربية الدولية AIU جامعة سورية خاصة أُحدثت عام 2005، خططها الدراسية والوثائق الصادرة عنها معتمدة ومصدقة من قبل وزارة التعليم العالي في الجمهورية العربية السورية.

تعمل الجامعة على تحقيق الأهداف الآتية:

- إعداد جيل متميز من الخريجين الجامعيين القادرين على تلبية الحاجات النوعية للمجتمع والنهوض به.
- الإسهام في البحوث العلمية النظرية والتطبيقية التي تخدم أغراض التنمية الوطنية، ويتم العمل على حث الأساتذة والعاملين الأكاديميين على البحث العلمي والمشاركة في المؤتمرات والندوات التي تنظم الأبحاث.
- تحقيق الشراكة مع الجامعات العربية والأجنبية المرموقة بهدف التطوير والتحديث المستمرين للعمل الأكاديمي والقيام ببحوث علمية مشتركة.
- استقطاب الكفاءات الأكاديمية والبحثية المتميزة عن طريق توفير البيئة المناسبة لعملها.

الجامعة العربية الدولية من الجامعات السورية الأولى التي جرى تأسيسها وافتتاحها، وقد تمكنت من اجتذاب الكفاءات التعليمية والبحثية والإدارية المتميزة، لإنشاء صرح متكامل من النواحي الأكاديمية والتنظيمية والإدارية. وتمكنت من تخريج كوادر من المبدعين والمتميزين من خلال توفير بيئة تعليمية تركز إلى مقومات نوعية ومادية فريدة منها:

- الخطط الدراسية الحديثة والمتطورة المستندة إلى نظام الساعات المعتمدة.
- الأطر التعليمية المنتقاة بعناية كبيرة.
- المختبرات العلمية الحديثة، ومختبر للمكتبات الإلكترونية.
- المحفزات المادية والمعنوية للطلبة.
- تطبيق طرائق التدريس التفاعلي.
- التوجيه والإرشاد الأكاديمي والتربوي.
- مجموعة كبيرة من اتفاقيات التعاون العلمي مع جامعات محلية وإقليمية ودولية ذات سمعة مرموقة.
- اتفاقيات ومذكرات تفاهم متعددة مع العديد من مؤسسات المجتمع المدني.
- الحرم الجامعي اللائق والمزود بكافة المرافق العلمية والرياضية والترفيهية، والذي نشجعك على زيارته والتعرف على مزاياه.
- الأنشطة والأندية الطلابية بمختلف أنواعها: الرياضية والثقافية والعلمية والاجتماعية.

في الجامعة العربية الدولية سنوات الحياة الجامعية هي وقت للاستثمار في مستقبل الطالب. فالمعارف والخبرات التي يحصلها في قاعة المحاضرات والمختبرات ستساعده في تطوير ذاته، وستمنحه أسباب النجاح في التخصص الذي اختاره، والنشاط الطلابي الذي يمارسه سيساعده في توسيع أفقه، وفعاليات التدريب والأندية والرياضة ستمكنه من تطوير مواهبه، ولربما تساعده في اكتشاف مواهب جديدة.

ليستثمر وقته وذهنه وروحه في جامعتنا كي يجني فوائد عمله والوقت الذي كرسه في السنين القادمة. ونحن سوف نكون بجانب طلبتنا في كل خطوة على دربهم.



الجامعة العربية الدولية

كلية الهندسة المعلوماتية والاتصالات

مشروع التخرج

نظام ذكي لأمن وسلامة المنزل يعتمد على إنترنت الأشياء والذكاء
الصنعي

تم تقديمه إلى

قسم الهندسة المعلوماتية

تقديم

محمد نعيم عيروض

محمد حسين حامد

عبد الكريم شاهين

إسلام العبد

أنس السعودي

بإشراف

المهندس عبد الله السردى

الدكتور أحمد محسن

كانون الثاني 2026

